

—— // THE CLI FLEET // ——

The CLI Fleet

First Edition

THE CLI FLEET

the tools · the ports · the compendium

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Before You Begin	2
The shape of the fleet — shared architecture and the port method	4
The covered fleet — tools with their own books	10
Isofrs — a Rust reimplementation of Isof	16
nmaprs — a Rust port of nmap’s scan engine and CLI dialect	23
temprs — a stack-structured temporary-file manager for the shell	30
iftoprs — iftop rebuilt on ratatui and pcap	37
htoprs — the interactive process viewer, ported function-for-function into Rust	45
storageshower — a real-time disk-usage TUI in Rust	52
zcolorizer — the log stream, repainted in real time	59
grcrs — a two-binary Rust port of the Generic Colouriser	66
zpwrrchrome-host and zwire-host — the browser-adjacent command line	73
Matching the upstream — the parser-compatibility layer, tool by tool	78
The output contract — TTY-awareness, ANSI, and the cyberpunk chrome	81
The theme engine — the 31-palette lineage and shift_lighter	83
Talking to the kernel — procfs, libproc, pcap, raw packets, sysctl	85
Measuring the fleet — benchmark and test methodology, honestly	88
Packaging the fleet — binaries, completions, man pages, release	91
Appendix A — the complete command & flag reference	94
Appendix B — the palette and theme catalog	100
Appendix C — the provisional-numbers ledger	102

Before You Begin

This is the book about the command-line tools — the whole fleet of them, in one place. The MenkeTechnologies stack has grown a large surface of terminal programs: a compiled shell, four language runtimes that share one bytecode VM, a handful of from-source Unix ports, a set of colourising filters, and a pair of browser-adjacent native hosts. Several of those already have books of their own. Many do not. This compendium is the map to all of them and the full text for the ones that were, until now, documented only by their man pages and their source.

It is deliberately the CLI-side twin of *Rewriting the Desktop in Rust*, the companion that covers the graphical fleet — the ported Tauri applications and their embeddable engines. Where that book follows the desktop programs, this one follows the things you run in a terminal and pipe into each other. Together the two books account for the shipped tools end to end: the ones with a window and the ones with a prompt.

The structure follows from that goal. After this preface comes a short chapter on the shape of the fleet — the architecture, dependencies, and porting method the tools share, pulled out once so the per-tool chapters can stay focused. Then an index: the seven CLI tools that already carry a dedicated reference manual or companion volume — `zshrs`, `strykelang`, `awkrs`, `vimlrs`, `elisprs`, `powerliners`, and `ztmux` — are situated in the fleet and pointed at their own books, not re-documented here. Everything after that is a full chapter for a tool that had no book: the Unix reimplementations (`lsofrs`, `nmaprs`, `iftoprs`, `htoprs`), the shell temporary-file stack manager (`temprs`), the disk-usage explorer (`storageshower`), and the two colourisers (`zcolorizer` and `grcrs`) — closing with `zpwrichrome-host` and `zwire-host`, the two native-messaging host binaries that finish the catalog's CLI-tools shelf. Each chapter is written to be read front to back and grounded in the program's own source — the README, the crate manifest, the binary entry points — rather than in recollection.

This book is honest about foundations, because most of the fleet is a port and the honesty is the point. When a tool reimplements an existing Unix program — `lsof`, `nmap`, `iftop`, `htop`, `grc` — the chapter names the upstream plainly. The original work in those cases is the *engineering* of the Rust port: the architecture, the binary suite, the parity against the original, the drop-in compatibility with the world the upstream tool lived in. Where a tool is not a port — the log colouriser with its own theme grammar, the disk explorer, the two native hosts — the chapter says that too, and spends its pages on the design rather than on a comparison. No tool in this book is dressed up as more than it is; each is documented as what it is.

Where a number is provisional — a version string, a test count, a benchmark — the text marks it rather than rounding it into a claim. Versions move, suites grow, and a book that pretends otherwise goes stale the day it prints. The counts here are the counts the sources reported at the time of writing.

THE CLI FLEET – what this book contains

Front matter

The shape of the fleet shared architecture + the port method

Part I – the covered fleet (index only; see each tool's own book)

zshrs · strykelang · awkrs · vimlrs · elisprs · powerliners · ztmux

Part II – the uncovered fleet (full chapters)

lsofrs	list open files	(ports lsof)
nmaprs	network scanner	(ports nmap)
temprs	shell temporary-file stack manager	
iftoprs	per-connection bandwidth	(ports iftop)
htoprs	process monitor	(ports htop)
storageshower	disk-usage explorer	
zcolorizer	real-time log colouriser	(own theme grammar)
grcrs	generic colouriser	(ports grc)
zpwrchrome-host	browserpass-native port + extension actions	
zwire-host	universal local host	(sysinfo/fs/exec/pty/kv)

Part III – reading across the fleet (synthesis + reference)

Matching the upstream	the parser-compatibility layer, tool by tool
The output contract	TTY-awareness, ANSI, and the cyberpunk chrome
The theme engine	the 31-palette lineage and shift_lighter
Talking to the kernel	procfs, libproc, pcap, raw packets, sysctl
Measuring the fleet	benchmark and test methodology, honestly
Packaging the fleet	binaries, completions, man pages, release
Appendix A	the complete command & flag reference
Appendix B	the palette and theme catalog
Appendix C	the provisional-numbers ledger

The tools in Part II are what most of this book is about. Read the shape-of-the- fleet chapter for the architecture they share, Part I as the index that keeps the fleet whole, Part II as the coverage that was missing, and Part III as the cross-grain synthesis — the shared parser, output, theme, benchmark, and packaging machinery pulled together — closing with three reference appendices that consolidate every command, palette, and provisional number the per-tool chapters state in passing.

The shape of the fleet — shared architecture and the port method

Eight command-line tools get full chapters in this book, and before any of them is opened on its own terms it is worth reading them sideways — across the grain — for what they have in common. They were built by the same organization, on the same durable-dependency rules, with the same house conventions for how a shipped Rust CLI should behave, and they divide cleanly into two kinds: reimplementations of a named Unix program, and originals that answer a problem the author first solved somewhere else. This chapter pulls the shared machinery out once — the substrate split, the dependency floor, the terminal stack, the argument parsers, the kernel interfaces, the licences, and the parity discipline — so that each per-tool chapter can spend its pages on what makes that tool specific rather than re-deriving the parts they all agree on. Every claim here is grounded in the crate manifests and source of the eight tools it covers: `lsotrs`, `nmaprs`, `iftotrs`, `htotrs`, `storageshower`, `zcolorizer`, `grcrs`, and `temprs`. Where a pattern is only true for some of them, the text says which; a shared convention that four tools follow and four do not is not a fleet-wide law, and the value of reading across the grain is lost the moment a convenient generalization papers over the exceptions.

The substrate split: ports and originals

The first cut divides the eight into reimplementations and originals, and the division is not cosmetic — it changes what “done” means for the tool. Five of them reimplement a specific, named Unix program: `lsotrs` rebuilds `lsot` (by way of `lsotng`, the modernized fork of Vic Abell’s original), `nmaprs` rebuilds Nmap’s scan engine and CLI dialect, `iftotrs` rebuilds `iftop`, `htotrs` rebuilds `htop`, and `grcrs` rebuilds `grc`, the Generic Colouriser. For each of these there exists an upstream whose behaviour is the specification of record, and the engineering question is fidelity: does the port produce the output the original would have produced, accept the flags the original accepted, consume the same data files, and interoperate with the same scripts. The design of the tool is not up for invention. The option grammar, the columnar layout, the network selector, the colourfile format — these belong to the upstream, and the Rust work is the faithful translation plus whatever memory-safety, cross-architecture, and vendorability the rewrite buys.

The other three are originals, and they are honest about it. `storageshower` is a disk-usage TUI that is not a port of `ncdu`, `du`, or `df` — its own repository carries the bash predecessor it descends from under `old_scripts/`, a 2018-vintage `storageShower.sh` dispatching to a ~31 KB `storageshowerCyberpunk.sh`; the Rust crate is the reimplementations of the author’s own concept as a compiled, threaded, tested binary, and it arrives at whatever it shares with `ncdu` independently rather than by porting anyone’s code. `zcolorizer` is subtler: its manifest calls it a “ccze/pygments port” and it does carry a full copy of ccze’s C source under `vendor/ccze/` as a build-time reference, faithfully reproducing ccze’s plugin-then-word-colorizer architecture — but its 47-module format library, its themeable colour model, and its three-layer rules/themes/engine decomposition are original work extending a 2003 tool into formats (systemd, JSON, Kubernetes, Postgres) that did not exist when ccze was written. `temprs`, the shell temporary-file stack, is the cleanest original: there is no `temp(1)` it descends from. What it ports is the *observable behaviour* of several coreutils — its `--head`, `--tail`, `--wc`, `--grep`, `--diff`, `--cat` reproduce what `head`, `tail`, `wc`, `grep`, `diff`, and `cat` do — but addressed by stack index or tag name instead of by path, and the engineering that matters is the atomic, locked, null-delimited master record underneath, not the reinvention of a diff.

Port-honest, mechanically – what the word means here

“Port-honest” is the house rule that governs how the reimplementations describe themselves, and it has a mechanical meaning rather than a rhetorical one. Naming the upstream is the first obligation: `lsotrs` names `lsot`, `nmaprs` names Nmap in its own `Cargo.toml` description and points anyone needing “authoritative Nmap NSE/OS DB behavior” back at Nmap, `iftotrs`’s manifest calls it an “iftop clone in Rust,” `htotrs`’s README leads with “the C source is the spec,” and `grctrs` names `gcr` and its upstream version. The second obligation is to state where the port stops. `nmaprs` does not embed the NSE Lua runtime and says so; its OS detection is partial and its version detection skips Perl-only regex constructs, all documented in a capability matrix rather than glossed over. `htotrs` deliberately does not port the hand-rolled allocator families (`xMalloc`, `Vector.c`’s raw `Object**` machinery, `Hashtable.c`’s open-addressing table) because reimplementing them byte-for-byte would mean writing an allocator in `unsafe` — precisely the bug class the port exists to eliminate — so `Vec` and `HashMap` stand in, and the omission is documented, not hidden. The third obligation is the accept-and-ignore contract: `lsotrs` parses and silently discards a whole set of upstream flags (`-b`, `-O`, `-D`, `-K`, and their `+` counterparts) that have no meaning in the Rust implementation, so that scripts passing them keep running rather than erroring, even though the behaviour behind them is absent. That is the correct trade for drop-in substitution, and calling it “honored syntactically, not behaviorally” is the port-honest way to describe it.

The dependency floor

The manifests share a deliberately foundational dependency set — the meta rule is to prefer crates that will still build cleanly years from now over fashionable ones, and it shows. Across the eight, the recurring names are `serde` (six of eight), `clap` (six), `regex` (five, plus `fancy-regex` in the one case that needs backtracking), `libc` (five), `dirs` for config-path resolution (four), `toml` (four), and `crossterm/ratatui` for the four with a terminal UI. The extremes bracket the range: `grctrs` depends on exactly two crates — `fancy-regex 0.14` and `libc 0.2` — while `nmaprs`, which has to build raw packets and speak TLS and SOCKS, pulls the widest set (`tokio` with `full`, `pnet/pnet_sys` for raw packet construction, `rustls` on the `ring` backend, `hickory-resolver` for DNS, `tokio-socks` for proxies, `crc32c/crc32fast` for SCTP checksums, and `cc` to compile one C++ FFI object). `temprs` sits near the minimal end with `clap` plus `fs2` for advisory file locking. None of the eight depends on a memory-mapping crate — there is no `memmap/memmap2` anywhere in the set — which is worth stating plainly because it is easy to assume a systems tool memory-maps its input. These tools stream: `grctrs`, `zcolorizer`, and `temprs` read stdin line-buffered, and the rest gather their data from kernel APIs and pseudo-file systems, never from an `mmap`’d file.

The release profiles rhyme. Six of the eight tune for a lean shipped binary with `lto = true` and `strip = true`, most adding `opt-level = 3` and `codegen-units = 1`, and three (`nmaprs`, `temprs`) also set `panic = "abort"`. Editions split evenly: `lsotrs`, `iftotrs`, `storageshower`, and `temprs` are on the 2024 edition; `nmaprs`, `htotrs`, `zcolorizer`, and `grctrs` are on 2021. Per the meta rule, local development builds with a plain `cargo build`; the tuned profile is what the installed binary carries, not what a developer waits on.

The TUI stack: ratatui and crossterm, and one deliberate refusal

Four of the eight are interactive terminal applications, and all four reach for the same pair: `ratatui` (0.30 across the board) for widget layout and `crossterm` (0.29) for the terminal backend. `lsotrs`, `iftotrs`, `htotrs`, and `storageshower` all carry both in their manifests. The event-loop shape is consistent enough to describe once. A single-threaded render/input loop owns the terminal: it draws one frame with `ratatui`, then polls `crossterm` for input within a bounded tick budget, and it never does expensive work inline. The data collection runs elsewhere — on background OS threads that write into `Arc<Mutex<...>>` shared state the UI reads from. `storageshower` spawns a `spawn_bg_collector` thread that refreshes disks and system stats on a three-second cadence independent of the user-facing refresh rate; `iftotrs` runs a dedicated

`packet-capture` thread and a `proc-lookup` thread feeding a poison-tolerant `FlowTracker`; `lsofrs` gives every live mode the same keybindings, alternate-screen handling, and atomic frame rendering through a shared `TuiMode` trait. The loop cadence is explicit where measured — `iftoprs` runs a 33 ms tick (roughly 30 fps), `storageshower` polls `crossterm` with a 200 ms timeout — but the structure is the same: drain events, maybe rotate, snapshot, draw, poll.

`htoprs` is the deliberate refusal, and it is instructive. It carries `ratatui` in its manifest too, but the ported htop core does not draw through it — the constraint on `htoprs` is that its output be byte-for-byte identical to htop, whose C source renders through ncurses, so the ported draw code drives `crossterm` directly and transcribes `CRT.c`'s `ColorElements` enum and every `CRT_colorSchemes` entry verbatim to match htop's colours glyph for glyph. `ratatui` appears only in the gate-exempt `src/extensions/` tree — `finder.rs`, `panels.rs`, `overlay.rs` — where `htoprs` adds capability htop never had. So even the one shared library is used two different ways within a single crate, and reading the manifest without reading the source would mislead you into calling `htoprs` a `ratatui` app in the same sense the other three are.

The four TUI tools also share a themeing lineage that is worth tracing because it is real rather than coincidental. The 256-colour palette set — thirty-odd named schemes running from `NeonSprawl` through `Zaibatsu` — originates in `storageshower` (30 `ColorMode` variants), was carried into `iftoprs` (31, adding the parity palette `Iftopcolor`), and into `lsofrs` (31, asserted by an enum-length check) and `htoprs` (31 six-colour palettes). `zcolorizer` inherits the same set explicitly: its 31 generated palettes are, per its own source, “ported verbatim from the shared MenkeTechnologies theme set (`iftoprs`, itself derived from `storageshower`),” plus a 32nd hand-written `ccze-classic` for parity with the tool it colours after. A six-index cube-arithmetic helper, `shift_lighter`, moves with the palettes from `iftoprs` into `zcolorizer`. This is a genuine shared component, not merely a shared aesthetic.

Argument parsing: clap, and the two tools that refuse it

Six of the eight parse their command line with `clap` 4 — `lsofrs`, `nmaprs`, `iftoprs`, `storageshower`, `temprs`, and `zcolorizer` (behind a `cli` feature so the library can be embedded without pulling clap in). But two of the reimplementations refuse clap on purpose, and the reason is the same in both cases: fidelity to a parser that predates clap's grammar.

`htoprs` does not use clap; it calls the real `libc::getopt_long` symbol — the same one htop's own `getopt_long` fills — so that its option-parsing behaviour is not merely similar to htop's but produced by the identical library routine. `grcrs` does not use clap either; its `parse_opts` hand-mirrors `getopt`, including the two subtleties a clap parser would get wrong. First, abbreviated long options: `resolve_long` reproduces getopt's rule that an exact name wins, a *unique* prefix resolves, and an ambiguous prefix is rejected — so `--conf=name` resolves to `--config`, `--col=off` to `--colour`, but `--std` is refused because it could be `--stdout` or `--stderr`. Second, option-scanning that stops at the first non-option token and treats everything after -- as the command even if it looks like a flag. Two other tools do their own pre-parsing even while keeping clap downstream: `lsofrs` runs a hand-written `normalize_lsof_args` pass *before* clap sees anything, because lsof's grammar is not getopt-clean (the `+L/+d/+D` prefix family, attached arguments like `-o9` and `-sTCP:LISTEN`), and `nmaprs` runs an `argv_expand` pass that splits Nmap's glued tokens (`-sS`, `-Pn`, `-PS80`, `-T4`) apart before clap's derive parser, which expects separated options. The pattern here is not “everyone uses clap” — it is “everyone matches the upstream parser, using clap where clap can express the grammar and refusing or pre-processing it where it cannot.”

Talking to the kernel: procfs, libproc, pcap, and raw packets

The system tools gather their data straight from kernel interfaces, and none of them shells out to another program to do it — the anti-subprocess discipline is a shared value. The Linux path is hand-parsed `procfs` everywhere it appears: `lsofrs` walks `/proc/<pid>/fd` and reads the `cwd/root/exe`

symlinks; `iftoprs` parses `/proc/net/tcp`, `/proc/net/tcp6`, `/proc/net/udp`, `/proc/net/udp6`, `/proc/[pid]/fd/`, and `/proc/[pid]/comm` to attribute flows to processes; `storageshower` reads `/proc/mounts` and `/proc/diskstats`. Notably, none of them uses the `procfs` crate — the parsing is done by hand against the files. The macOS path is direct libproc: `lsofrs` declares `proc_listpids` and `proc_pidinfo` as raw `extern "C"` and calls them against `repr(C)` structs matched to the kernel headers; `iftoprs` uses the `libproc` crate to read each process's open sockets via `pidfdinfo/SocketFDInfo` (its manifest comment notes this replaces a runtime `lsof`); `storageshower` calls `getmntinfo` and reads IOKit block-storage byte counters through `libc`. `htoprs` carries the full per-OS backend tree (`darwin/`, `linux/`, `freebsd/`, `openbsd/`, `netbsd/`, `dragonflybsd/`, `solaris/`) that htop itself carries, and on Linux pulls three pure-Rust, no-FFI helper crates — `libmedium` for hwmon sensors, `neli` speaking generic-netlink for the TASKSTATS delay-accounting path, and `zbus` speaking D-Bus directly for the systemd meter — deliberately keeping C libraries out of the build.

Two tools stand apart on the wire. `iftoprs` is the only one that uses `pcap` (2.4) — it opens the interface with a 256-byte snaplen (enough for L2–L4 headers, never payload), decodes Ethernet, BSD loopback, Linux cooked capture, and raw IP link types, and pushes decoded packets across a `tokio::sync::mpsc` channel off the capture thread. `nmaprs` is the only one that constructs raw packets, via `pnet/pnet_sys`, and it is also the only one with a genuinely async core (`tokio` with `full`); interestingly, its default TCP-connect scan does *not* use async sockets but a worker pool of OS threads running blocking `connect_timeout`, falling back to tokio tasks only when a SOCKS/HTTP-CONNECT proxy state machine is needed. Raw capture and raw scanning both need privilege, and both tools degrade gracefully: `nmaprs` detects `geteuid()` and silently falls back from SYN to connect scan when unprivileged, while `iftoprs`'s capture loop treats device failures as transient and retries with exponential backoff.

Only two of the eight carry a `build.rs`, and for opposite reasons. `nmaprs` uses it to compile one C++ object (`c/expr_match.cpp`) via the `cc` crate for its OS-fingerprint expression matcher. `htoprs` uses it as an anti-drift gate — described below — that runs on every build.

Licensing

Every tool in the fleet is licensed MIT, and the Rust in each is original code. `lsofrs`, `nmaprs`, `iftoprs`, `htoprs`, `storageshower`, `zcolorizer`, `grcrs`, and `temprs` each declare `license = "MIT"` in their manifests, ship a single `LICENSE` file with the MIT text, and carry the same MIT badge and MenkeTechnologies copyright. Where a tool reimplements a Unix program — `lsof`, `nmap`, `iftop`, `htop`, `grc` — it studies the upstream's *behaviour* to reach parity, and some keep the upstream source under a `vendor/` tree purely as a read-only reference for that work; the binary is built from the tool's own `src/`, never from `vendor/`. There is no copyleft anywhere in the CLI fleet and no per-tool licence divergence to track — the whole shelf is uniformly, permissively MIT.

Parity methodology, tool by tool

The most important thing to *not* overstate is the parity discipline, because it varies enormously across the eight and only one of them enforces it at build time. It is tempting to describe “the fleet's parity harness” as a shared thing; it is not.

`htoprs` has the strictest apparatus, and it is a build gate rather than an aspiration. Its source splits into `src/ported/<file>.rs` (one Rust module per htop C file, every free `fn` carrying a `/// Port of` doc comment — 1,930 such citations) and `src/extensions/<name>.rs` (htoprs-original, gate-exempt). The gate in `build.rs` runs on every `cargo build`, `test`, and `check`, extracts every free `fn` name under `src/ported/`, and checks it against a snapshot of htop's real C function names at `tests/data/htop_c_fn_names.txt`; a function with no upstream counterpart *fails the build*, and it fires even under `cargo test --test X` so a reimplementation cannot be routed around it. On top of that sits a byte-diff oracle: the `tests/parity/` suite runs the same inputs through the reference C `htop` and through `htoprs` and compares output byte-for-byte

(skipping gracefully when no htop 3.5.x is installed, so CI stays green), and `scripts/gen_port_report.py` derives the coverage figure from source at run time, counting a `todo!()` body as stubbed, never ported.

```
cargo build          # runs the port-purity fn-name gate
cargo test --test parity # diff htoprs vs reference htop 3.5.x
python3 scripts/gen_port_report.py # regenerate port_report.html
```

`grcrs` has the next-strongest discipline: a byte-for-byte oracle without a build gate. Its 68 tests include 27 end-to-end tests in `tests/cli.rs` that spawn the built binaries and assert on the exact coloured bytes, with expectations cross-checked against the reference Python `grcat` — covering `count=stop/once/more/block`, `unchanged/previous` colours, `replace-then-colour`, `</>` literals, CRLF stripping, and the invalid-UTF-8 round-trips that pin its `surrogateescape` reproduction. `nmaprs` documents parity as a capability matrix in its README (which port-state mappings each scan technique produces) backed by contract tests over the port database, scanflags, OS-DB parsing, and target expansion, plus repo-hygiene guards. `zcolorizer` keeps ccze's C source vendored as the reference and pins its precedence model (`earlier_rule_wins_overlap`) and input-sanitization behaviour with engine tests, but it colours new formats ccze never saw, so full byte-parity is not even the goal for most of its surface.

The remaining four have no automated parity oracle at all, and it would be wrong to imply otherwise. `lsofrs` reproduces lsof's option grammar, type vocabulary (`FileType`, `TcpState`), and output formats (`-F`, `-t`, `+L`, `-o`, `-s`) and asserts them with its own tests, but it does not diff against a running `lsof`. `iftoprs` ships a parity palette (`Iftopcolor`) and mirrors iftop's keybinds and the flow-table/rolling-window model, but its 2s/10s/40s columns are *summed byte counts over trailing one-second slots scaled to bits*, not slot-averaged bytes/sec — a difference the chapter states precisely rather than calling it “identical to iftop.” `storageshower` and `temprs`, being originals, have nothing upstream to be parity-tested against; their large test suites pin *their own* invariants (symlink-cycle termination in `storageshower`, master-record tag alignment and round-trips in `temprs`), which is a different kind of discipline entirely. The counts each project reports are large — on the order of a thousand or more `#[test]` annotations in several of them — but every per-tool chapter marks those as provisional grep counts of test *attributes*, not results of a `cargo test` run, and this chapter does the same: treat a test count as a count of intentions until a `test result:` line confirms it.

The shared surface: binaries, completions, man pages, config

Below the architecture, the eight share a set of packaging conventions, again with honest exceptions. The short-alias binary convention gives frequently-typed tools a two- or three-character name from the same source: `lsofrs` also builds `lsf`, `nmaprs` also builds `nms`, `temprs` also builds `tp`, all byte-identical to their long form. `grcrs` builds two *functionally distinct* binaries rather than an alias — `grc` the launcher and `grcat` the colouriser — mirroring the same split upstream `grc` uses so the colouring logic stays a standalone filter you can drive from a pipeline. The remaining four (`iftoprs`, `htoprs`, `storageshower`, `zcolorizer`) ship a single binary.

Six of the eight ship a checked-in zsh completion under `completions/_<tool>` with a `#compdef` header — `lsofrs`, `nmaprs`, `iftoprs`, `htoprs`, `storageshower`, and `temprs` — several of them kept in lockstep with the parser (`htoprs`'s tracks the getopt table in `commandline.rs`; `temprs`'s is *live*, reading the running stack's master file so tab-completion offers the actual indices and tags present at the moment you press Tab). `zcolorizer` and `grcrs` ship no completion in-tree — `grcrs` relies on the upstream `grc.zsh/grc.sh/grc.fish` shims instead. The same six also ship the paired man-page convention: a short `<tool>.1` and a comprehensive `<tool>all.1` modelled on `zshall(1)`, folding invocation, keybinds, themes, and architecture into one grep-able document. `grcrs` ships per-binary `grc.1` and `grcat.1` instead of an “all” page, and `zcolorizer` ships no `man/*.1` in the tree searched here.

Persisted configuration splits by whether the tool is interactive. The three TUI tools that keep user preferences use a TOML dotfile in `$HOME` — `~/.lsofrs.conf`, `~/.iftoprs.conf`, `~/.storageshower.conf` — auto-saved on every in-app toggle with no explicit save step. `htoprs` breaks the pattern by storing JSON

under `~/.config/htoprs/prefs.json`, and `zcolorizer` uses TOML but under XDG at `$XDG_CONFIG_HOME/zcolorizer/config.toml`. The three non-interactive tools (`nmaprs`, `grcrs`, `temprs`) keep no preference file at all — `grcrs` reads the upstream `grc.conf` data files, but those are shared config, not per-user state, and `temprs` keeps its stack on disk rather than a settings file.

How they sit in the shell

None of these tools is a shell plugin, and none reaches into a shell's internals — they are ordinary `argv/stdin/stdout` Unix filters, which is exactly why they compose under any POSIX shell including `zshrs`, the compiled shell at the centre of the wider fleet. What makes them compose *cleanly* is a shared TTY-awareness that every one of them implements: colour and interactive chrome are gated on whether the relevant descriptor is a terminal, and disabled the moment output is piped. `lsOfrs` drops its colours and cyberpunk headers and prints plain script-safe text when stdout is not a TTY, and its live modes never enter the alternate screen non-interactively — they print one snapshot and exit. `grcrs` initialises its colour default from `libc::isatty(1)`. `zcolorizer` auto-disables colour off a TTY unless `-C` forces it. `nmaprs`'s styled help suppresses escapes on a non-TTY or under `NO_COLOR`. `temprs` branches on `stdin().is_terminal()` so `CMD | tp` reads the pipe while a bare `tp` waits for typed input. Two tools go further and emit structured data for downstream tooling: `iftoprs --json` streams NDJSON — one object per refresh — that a `jq` filter or a dashboard can consume, and `nmaprs` writes Nmap-compatible XML (`-oX`) and grepable (`-oG`) formats. In practice that turns the fleet into pipeline stages:

```
ps aux | tp -w procs          # push tagged onto the temprs stack
tp -g python procs && echo hit # grep the tagged entry, compose in &&
journalctl -f | zcolorizer    # repaint a live log stream
sudo iftoprs --json | jq -c '.flows[] | {dst, sent_2s}'
kill "${lsf -t -c nginx}"     # bare PIDs feed another command
```

The convention that ties them together is not a framework — it is the agreement that a shipped CLI tool should be terse on stderr in the `<tool>: <thing>: <reason>` shape, quiet on a pipe, structured when asked, and drop-in against whatever it reimplements. Under `zshrs` they are dispatched the same way any external command is; the shell provides the completions, the history, and the pipeline plumbing, and the tools provide clean, TTY-aware, script-safe behaviour on the other end of it. That is the whole shared contract, and it is why the per-tool chapters can now assume it and move on to what each tool does that no other does.

The covered fleet — tools with their own books

This compendium is the single index to every command-line tool in the MenkeTechnologies stack, and it hands a full chapter to each tool that lacks a book of its own. Seven tools do not need that treatment here, because each already has a dedicated reference manual or companion volume in MenkeTechnologiesPublications. This chapter situates those seven inside the fleet — what each one is, what it ports, where it sits in the runtime picture, and exactly which book carries it in full — so nothing is duplicated and nothing is lost. Read this as the map; read the named books for the territory.

Six of the seven share one substrate worth naming up front: `fusevm`, a bytecode virtual machine with a three-tier Cranelift JIT. `zshrs`, `strykelang`, `awkrs`, `vimlrs`, and `elisprs` all lower their source to `fusevm` bytecode and execute on the same engine rather than each carrying a bespoke VM. `ztmux` is the outlier — a from-source terminal-multiplexer port with no language-runtime role. `powerliners` is the other outlier — a statusline renderer that both consumes the fleet (it ships segments that read the rkyv and JIT caches of `zshrs`, `strykelang`, and `awkrs`) and stands entirely on its own Rust binary suite.

```
FLEET INDEX — tools documented in their own volumes
tool          port / origin          dedicated book
----          -
zshrs         zsh (Bourne lineage)    zshrs — zsh in Rust (ref.)
strykelang    Perl 5 superset         strykelang — the Stryke lang (ref.)
awkrs         AWK (POSIX/gawk/mawk)   awkrs — AWK in Rust (ref.)
vimlrs        Vimscript (nvim eval)   vimlrs — Vimscript in Rust (ref.)
elisprs       Emacs Lisp              elisprs — Emacs Lisp/fusevm (ref.)
powerliners   powerline-status (Py)   powerliners — powerline port (comp.)
ztmux         tmux (C)                ztmux — tmux port (companion)
```

zshrs — the compiled shell at the center of the fleet

`zshrs` is a drop-in `zsh` replacement written in Rust, and it is the tool everything else in the stack is built to run inside. Its README states the codebase at 658k+ lines across 490 source files in a two-crate workspace (`zshrs` runtime plus `zshrs-daemon`), with the runtime split mechanically into a strict 1:1 port directory (`src/ported/` — 106 files including its `builtins/`, `zle/`, and `modules/` subtrees, each function mapping to a real `Src/<x>.c` `zsh` function and carrying a `/// Port of <cname>() from Src/<file>.c:NNNN` marker, enforced by `tests/port_purity.rs`) and a non-port extensions directory (`src/extensions/`) for features `zsh`'s C does not have. The design premise is that since the Bourne shell in 1970 every Unix shell has been an interpreter; `zshrs` compiles every command to `fusevm` bytecode and executes it on the VM. Its lexer and parser are direct ports of `Src/lex.c` and `Src/parse.c`, and the layered `ZshProgram` → `ZshList` → `ZshSublist` → `ZshPipe` → `ZshCommand` AST is preserved verbatim; only the bytecode compiler is original Rust, and it targets the same `Op` enum that `strykelang` emits, so the two frontends share fused superinstructions and the Cranelift JIT path.

Execution runs through the four tiers the README lays out. A warm script whose source is unchanged hits its rkyv image and skips lex, parse, and compile entirely — it mmaps a `fusevm::Chunk` and runs it; loops and branches take the block-JIT path to native code through Cranelift; straight-line arithmetic takes the linear-JIT path; and everything else — builtins, I/O, string work — falls to the jump-table interpreter with fused superinstructions. The README's figure for that spread is a 717 ms cold run against a 7 ms warm run on

the same script. Around that core the extensions layer supplies the pieces zsh forks for: a persistent worker pool of [2–18] threads (sized in `~/.zshrs/zshrs.toml`) that replaces `fork + exec`; 23 coreutils commands (`cat`, `head`, `tail`, `wc`, `sort`, `find`, `date`, `mktemp`, `seq`, `tr`, ...) run in-process as zero-fork builtins; parallel primitives (`async/await`, `pmap`, `pgrep`, `peach`, `barrier`) that compile their body to a `fusevm::Chunk` once and substitute a `{}` placeholder per iteration rather than forking N times, drawing a VM from a pool that preserves allocations across iterations; AOP `intercept` before/after/around advice; and 193 registered ZLE widgets. Bytecode images live under `~/.zshrs/images/` as an `index.rkyv` plus mmap-ready shards, with a read-only SQLite mirror (`catalog.db`, `history.db`) beside them for `dbview` and ad-hoc SQL inspection only — never consulted for cache hit/miss or execution. The binary also carries an LSP server, a DAP debug adapter, and a JetBrains plugin that drives both. The README's test-corpus table totals 869 tests across seven suites — 392 construct-corpus, 160 no-tree-walker behavioral pins, 87 parser-shape probes, 70 upstream `.ztst` files, and three smaller suites (provisional, as reported in the README, not independently run here). Its role in the fleet is the host: the runtime the entire CLI environment dispatches through.

```
zshrs --lsp                # language server over stdio
zshrs --dap HOST:PORT      # DAP debugger, TCP connect-back
pmap 'gzip {}' *.log       # parallel map, VM-run, zero fork
intercept before git \
{ echo "git $INTERCEPT_ARGS" >> -/git.log }
dbview autoloads _git      # read-only browse of the mirror
```

Covered in full by its own book: **zshrs — zsh in Rust (reference manual)**.

strykelang — the Perl-compatible parallel language

strykelang (binary `stryke`, also installed as `s`) is a Perl 5 compatible interpreter in Rust with native parallel primitives. Per its README it uses NaN-boxed runtime values (`StrykeValue` is a `u64` carrying immediates — `undef`, an `i32`, or raw `f64` bits — or tagged `Arc<HeapObject>` pointers for big ints, strings, arrays, hashes, refs, regexes, and channels), a three-tier regex stack (Rust `regex` → `fancy-regex` for backreferences → `pcre2` for PCRE-only verbs), a match-dispatch bytecode VM (`vm.rs`) with a Cranelift block-and-linear-sub JIT (`jit.rs`), and rayon work-stealing across all cores for every parallel builtin. It parses and executes Perl 5 scripts directly — `-e` is optional when the first argument is not a file on disk and looks like code — through a context-sensitive lexer (regex-vs-division, heredocs, interpolation) and a recursive-descent-plus-Pratt parser, then lowers everything to bytecode; the project's stated policy is to add new VM opcodes rather than new AST node kinds when extending the language.

Where it meets the rest of the fleet is the `fusevm` bridge (`fusevm_bridge.rs`). **strykelang**'s own block JIT handles pure-integer segments, while string-bearing segments — comparisons, concat, `length/index`, `sprintf` with a literal format, mixed chains such as `length($s) >= $min` — are lowered to `fusevm`'s Cranelift block JIT through an abstract-stack analyzer with per-slot kind inference. A per-sub cache (`sub_fusevm_meta`) runs the eligibility detectors once for a hot sub and afterward hits an `OnceCell<Arc<fusevm::Chunk>>` for O(1) dispatch. On top of Perl it adds a wide surface: pipe-forward `|>` and thread-macro `->` syntax, `pmap/pgrep/psort/preduce` plus streaming variants (`pmaps/pgreps` over lazy, bounded-memory iterators), `varsync` shared state backed by `Arc<Mutex>` with atomic compound ops, `fan/fan_cap`, channels with a Go-style `pselect`, structs/enums/classes/traits, an `s_web` framework, distributed `pmap_on` over an SSH worker pool (`cluster`), and AI primitives. The standard library is spread across a large family of `builtins_*` and `math_wolfram_*` modules in the source tree; the README reports 10,487 primaries in its `%b` namespace and a 44 MB single static binary (both provisional as stated). It shares the `fusevm` opcode set and JIT path with **zshrs** — the two frontends target the same `Op` enum. In the stack **stryke** is the scripting layer — the one-liner and data-processing language that embeds inside **zshrs** — and it is the origin of the unit-test assertion framework (`zassert_*`) that **zshrs** ports as builtins.

```

val @doubled = @data |> pmap $_ * 2, progress => 1 # parallel map
val @evens   = @data |> pgrep $_ % 2 == 0        # parallel filter
varsync $counter = 0
fan 10000 { $counter++ }                        # always 10000
range(0, 1e9) |> pmaps { expensive(_) } |> take 10 # streaming

```

Covered in full by its own book: [strykelang — the Stryke language \(reference manual\)](#).

awkrs — AWK as a bytecode engine

[awkrs](#) runs pattern → action programs over input records like POSIX [awk](#), GNU [gawk](#), and [mawk](#), with a CLI that accepts the union of those three option sets. Its README positions it as POSIX awk plus the gawk extensions that appear in real scripts — [BEGINFILE/ENDFILE](#), [coprocess](#) [|&](#), CSV mode ([-k/--csv](#)), [PROCINFO/SYMTAB/FUNCTAB](#), [@include/@load/@namespace](#), [/inet/tcp|udp](#) client sockets, and MPFR big numbers via [-M](#) (backed by the [rug/gmp-mpfr-sys](#) crates, which is why the build wants a C compiler and [make](#)). The engine is a fused-superinstruction stack bytecode VM with peephole fusion — [print \\$N](#) collapses to [PrintFieldStdout](#), [i++/++i/i = i + 1](#) to [IncrSlot](#), [s += \\$N](#) to [AddFieldToSlot](#), and [\\$1 ", " \\$2](#) to [ConcatPoolStr](#) — and scalars get [u16](#) slot indices so reads and writes are flat-array indexing instead of HashMap lookups. Fields are stored as [\(u32, u32\)](#) byte ranges into the record with owned strings only on assignment, and [print \\$N](#) under the default [FS](#) scans raw bytes straight out of the mmap'd file to find the Nth whitespace field.

A default-on [fusevm](#) offload path (the crate pins [fusevm](#) 0.14.5 with the [jit](#), [jit-disk-cache](#), and [aot](#) features) lowers eligible numeric chunks — arithmetic, comparisons, compound assignments, [int](#), the transcendentals, the bitwise builtins, and trapping division/modulo — to [fusevm::Ops](#) and block-JIT-compile hot loops on their first run, persisting native code to [~/.cache/fusevm-jit](#), the same on-disk JIT cache [zshrs](#) uses; the README measures 14–110× over the bytecode interpreter on pure-numeric accumulator loops and reports it bit-for-bit identical to [AWKRS_FUSEVM=0](#). Separately, [-f script.awk](#) runs memoize the compiled program to [~/.awkrs/scripts.rkyv](#) as an rkyv shard (rkyv outer envelope, bincode inner blob — the same layout as [zshrs](#) and [stryke](#)) with zero-copy mmap reads, invalidated on source or binary mtime change and disabled by [AWKRS_CACHE=0](#); the README's survey of BWK awk, gawk, mawk, goawk, fawk, and zawk concludes [awkrs](#) is the first awk to pair a bytecode VM with a persistent on-disk bytecode cache (fawk has the JIT but recompiles each run; gawk's pm-gawk persists variables, not compiled bytecode). Record processing parallelizes via rayon with deterministic output reordering when a static parallel-safety check passes ([-j/--threads](#), default 1; regular files are memory-mapped, stdin chunks up to [--read-ahead](#) lines). Its [examples/](#) directory carries dozens of standalone awk programs — a recursive-descent calculator, a Brainfuck interpreter, Dijkstra, Aho-Corasick, an LZW codec — each byte-for-byte verified against gawk in CI, so they double as gawk-extension regression tests. Reported benchmarks against BSD awk, gawk 5.4.0, and mawk 1.3.4 (hyperfine, 1 M lines, Apple silicon) are provisional as printed. In the fleet [awkrs](#) absorbs the AWK text-processing role and shares the [fusevm](#) numeric-op set with the language siblings.

```

awkrs -f script.awk data.txt # cached: skips recompile
awkrs -j 8 '{ print $1 }' big.txt # parallel, order preserved
awkrs -k '{ print $2 }' data.csv # gawk-style CSV mode
AWKRS_FUSEVM=0 awkrs -f numeric.awk # force the interpreter

```

Covered in full by its own book: [awkrs — AWK in Rust \(reference manual\)](#).

vimlrs — Vimscript outside the editor

[vimlrs](#) (binary [viml](#)) runs [.vim](#) scripts as ordinary standalone programs. Its README describes it as the first compiled standalone VimL interpreter — Vimscript has only ever run embedded inside Vim or Neovim — and a faithful port of Neovim's C [eval/*](#) tree, with the C source treated as the spec rather than reinvented. The ported subtree under [src/ported/](#) mirrors that C in 54 Rust files ([eval](#), [typval](#), [list](#), [dict](#), [blob](#),

`charset`, `mbyte`, `ex_eval`, `mark`, and the rest), and `scripts/gen_port_stubs.sh` emits one cited stub per not-yet-ported C function so the remaining surface is enumerated and the drift gate covers it. It is the fourth language hosted on `fusevm`: it lexes and parses VimL through the `eval1...eval7` grammar to an AST, lowers that to `fusevm` bytecode, and lets the shared engine's three-tier Cranelift JIT run it, carrying no VM or JIT of its own. The value layer — `typval`, `list`, insertion-ordered `dict`, `blob`, with the typed `tv_dict_*` accessors, coercions, `typval_compare`, and `string():echo` rendering — is ported and marked working.

The JIT story the README documents is that provably-Number locals become `Op::GetSlot/SetSlot` and a function's numeric `while` loop rotates so the condition is the backedge and trace-JIT-compiles to native code, at both function and script (top-level) scope; float accumulators, short-circuit compound conditions, native `%`, native bitwise builtins, and numeric ternaries all stay trace-eligible, and `VIMLR5_JIT_STATS=1 viml script.vim` reports the traces it compiled while `VIMLR5_NO_JIT=1` forces the interpreter baseline — the README's figure is ~15–100× on a 20M-iteration loop (provisional as stated). The builtin surface is partial: roughly 113 functions from `funcs.c` (string/list/dict, char-indexed string ops, `json_encode/json_decode`, `system`, `sha256`, `rand/srand` bit-exact against Neovim's xoshiro128** generator, date/time, transcendental math), with the full table pending. A large share of the work is in the regex dialect — Vim magic, ASCII-only `\a/\w/\d` character classes per `:help /\a`, option-derived atoms, POSIX bracket classes, and `\c/\C` case-fold rules — and every `examples/*.vim` is written as a self-test run in CI against real vim/nvim, so the regex and builtin behavior is pinned to the editors it imitates. It ships the standalone `viml` binary with `-e/-c/file/--repl`, an rkyv script cache at `~/.cache/vimlrs`, an AOT `--build`, a `--disasm` disassembler, an `--lsp` server, and a `--dap` debugger, plus a vim9script foundation (`:def` with typed parameters, automatic line continuation). Its fleet role is the Vimscript execution engine, paired with the vim-editor work elsewhere in the stack.

```
viml -e 'echo range(5)->map({_, v -> v * v})' " [0, 1, 4, 9, 16]
viml script.vim                             " run a .vim standalone
VIMLR5_JIT_STATS=1 viml hotloop.vim          " report traces compiled
viml --lsp                                   " diagnostics, hover, ...
```

Covered in full by its own book: **vimlrs — Vimscript in Rust (reference manual)**.

elisprs — Emacs Lisp on the shared VM

`elisprs` (binary `elisp`) runs Emacs Lisp `.el` files as standalone programs from the command line, with a REPL and no Emacs process required. Its README frames it as the fifth language hosted on `fusevm`. The reason it exists as a distinct effort is that Emacs Lisp is a Lisp-2 — every symbol carries a separate value cell and function cell — and supports both lexical and dynamic scoping; those facts are the personality of the language, so `elisprs` owns the value model and semantics and leans on `fusevm` only for execution. The source splits along that line: `reader.rs` is an elisp-correct S-expression reader (`1+/1-`, `#'foo`, `?c`, `:kw`, backquote, dotted pairs); `host.rs` holds the `ElispHost` object heap — interned symbols, real dotted cons cells, vectors, hash tables, and closures — the Lisp-2 obarray, the dynamic binding stack, and the `fusevm` extension handler that runs elisp ops, with every heap object reaching the VM as a `Value::Obj(u32)` handle; `compiler.rs` lowers each macro-expanded top-level form to a `fusevm::Chunk` with lambda bodies as sub-chunks; and `builtins.rs` plus `prelude.rs` carry the standard library, part native and part written in elisp on top of the primitives. An earlier bootstrap on the `rust_lisp` crate was replaced by this own value model, and `rust_lisp` is no longer a dependency.

The README enumerates 21 special forms (`quote`, `lambda`, `if`, `cond`, `let/let*`, `defun`, `defmacro`, `defvar`, `condition-case`, `unwind-protect`, ...) and roughly 90 subrs across arithmetic, lists, the `c*r` combinators, vectors, hash tables, predicates, strings, regexp, markers, and text properties, plus `pcase` (including backquote patterns), generalized `setf`, and both lexical (`lexical-binding: t`) and dynamic binding. It goes far enough into the editor layer to carry a registry of named live buffers with point, mark, and narrowing, first-class markers that auto-adjust across inserts and deletes, and string and buffer text-property intervals — enough, the README claims, that `tabulated-list-print` produces byte-identical propertyized output to GNU Emacs. It AOT-compiles to standalone native binaries (`--aot-exe`, through `fusevm::aot::compile_object`), caches lowered bytecode in an rkyv shard at `~/.elisprs`, and ships `--lsp` and `--dap` servers. It is explicit that this

is a useful elisp core, not the ~1000-subr GNU Emacs surface — overlays, a real interval tree, and redisplay are named out of scope rather than faked. In the fleet it is the elisp execution engine that also underpins the embedded-elisp editor work in **zmax**.

```
elisp -e '(mapcar (lambda (x) (* x x)) (list 1 2 3 4))'
; => (1 4 9 16)
elisp examples/demo.el           ; run a self-testing ERT file
elisp --aot-exe prog.el -o a.out  ; standalone native binary
elisp --lsp                       ; completion / hover / diagnostics
```

Covered in full by its own book: **elisprs — Emacs Lisp on fusevm (reference manual)**.

powerliners — the statusline without the Python tax

powerliners is a Rust port of **powerline-status**, the Python-driven statusline and prompt renderer used in tmux, zsh, bash, vim, and the Python REPL. Its README's rationale is startup cost: the Python implementation pays a ~50–150 ms interpreter-startup tax on every render, which compounds because tmux refreshes the statusline per interval and per window and zsh redraws its prompt after keystroke-triggered **precmd** hooks. **powerliners** is a single static binary with sub-millisecond render, and it is drop-in compatible with the existing **powerline/config** JSON theme and segment files, so themes stay unchanged. It ships a five-binary suite mirroring the upstream layout — **powerline** (the native client, forwarding **argv + cwd + env** to the daemon and falling back to a **powerline-render** exec if the daemon is unreachable), **powerline-config**, **powerline-lint**, **powerline-render**, and **powerline-daemon** — and the daemon speaks the identical **argc\0arg\0...\0cwd\0KEY=VAL\0...\0\0** wire format with an **E0F\0\0** shutdown sentinel, so the pip-installed Python C client talks to a running Rust daemon unchanged over the same socket path (**/tmp/powerline-ipc- $\$$ UID** on macOS, an abstract socket on Linux).

The render path is a function-level port of the Python pipeline: config-cascade load and **mergedicts**, colorscheme alias chasing and cterm resolution, **gen_segment_getter**, **process_segment**, the **do_render/render_segments** loop with hard/soft divider insertion and outer padding, and **TmuxRenderer #[fg=...,bg=...]** markup emission. It wires 54 segment adapters (55 **ADAPTERS** keys with the bare **exec** alias) and adds net-new ones — a p10k-style single-fork **git_status**, plus Docker/Kubernetes/AWS/GCP/GitHub-CI context segments — and, most relevant to this book, four that read the fleet's own caches: **powerliners.zshrs.rkyv_cache**, **powerliners.stryke.rkyv_cache**, **powerliners.awkrs.rkyv_cache**, and **powerliners.fusevm.jit_cache**, each a pure filesystem probe over a documented resolution chain, plus version segments for **zshrs/stryke/awkrs**. A separate extension (**src/extensions/watch.rs**) turns the normally pull-only prompt into a push: the warm daemon watches each client's **cwd**, **.git/HEAD**, **.git/index**, and active branch ref through the **notify** crate (kqueue/FSEvents on macOS, inotify on Linux) and writes a single wake byte to a per-client FIFO that zsh's ZLE reads via **zle -F**, redrawing the prompt in place the instant you check out a branch in another pane. The README reports 134/137 upstream **.py** files at its DONE tier (97.8% — the remaining three are class-only shells with no function bodies left to port), 2436 passing library tests, and 462 parity tests run against the live upstream Python (all provisional as printed). That makes **powerliners** the fleet's prompt-and-statusline surface, and the one tool here whose output visibly reflects the state of the rest of the stack.

```
brew install powerliners           # installs the 5-binary suite
powerline-daemon -q                # double-fork, detach, bind the socket
POWERLINE_CONFIG_PATHS=~/config/powerline \
  powerline-daemon --foreground --socket /tmp/powerliners
powerline-render tmux right -p ~/config/powerline # no daemon
```

Covered in full by its own book: **powerliners — powerline port (companion)**.

ztmux — tmux, reimplemented in Rust

ztmux is a from-source port of tmux to Rust — the whole program: server, client, grid/screen model, input parser, layouts, the command language, formats, and the terminal back end. Its README is emphatic that it is not a wrapper around the **tmux** binary and not control mode (**tmux -CC**); it is tmux itself, reimplemented. The port stands on the upstream tmux C sources, vendored read-only and SHA-pinned under **vendor/tmux/**, with **Cargo.toml** declaring its own **[workspace]** that excludes the reference so Cargo never walks into it, and every ported function carries a back-link comment to its C origin (e.g. **// vendor/tmux/grid.c:320 grid_create()**). It links a C **libevent** for the event loop and an ncurses terminfo database exactly as tmux does, and its **build.rs** drives lalrpop for the command grammar and the libevent linking. It opens its own socket namespace (**ztmux-<uid>**) so it never collides with a running tmux.

Correctness is measured rather than asserted: a parity suite runs identical inputs through the real **tmux** and **ztmux** and diffs them byte-for-byte, reported at 1107/1107 cases passing against the vendored tmux (provisional as stated), and it has earned that by root-causing real divergences — a **{l:...}** server crash traced to a dropped pointer increment in **format_unescape**, even-horizontal layout rounding, **{pane_current_command}** on macOS. An anti-drift gate (**tests/ported_fn_names_match_c.rs**) fails the build if a free **fn** is added to **src/** whose name has no counterpart in the tmux C source, with pre-existing wrappers frozen in an allowlist to burn down. The server pins **PROTOCOL_VERSION = 8** (verified in **src/ported/tmux_protocol_h.rs**) to match the separate **ztmux-core** client engine, so a GUI host can drive it over the same wire protocol without upstream tmux's release cadence being able to break the contract. On top of the port it ships 119 extension modules under **src/extensions/** — JSON-emitting inspectors over the live server (**ps**, **pstree**, **git**, **net**, ...), live ratatui TUIs (**dashboard**, **switcher**, **watch**), and opt-in zellij-style modes (framed/inset panes, a pane stack, a tab bar, floating panes, modal keybindings, and **resurrect** session persistence) — all exempt from the anti-drift gate precisely because they are not tmux, and all gated behind **set -g** toggles so **@ztmux-ratatui off** restores the byte-for-byte plain-tmux draw path. In the fleet **ztmux** is the multiplexer the shell lives inside; unlike the five language tools it does not touch **fusevm**.

```
cargo run --release -- new-session      # start server + session
bash parity/run_parity.sh --summary    # ztmux vs tmux, every case
ztmux ps --json                       # original inspector, JSON
ztmux dashboard                       # full-screen server monitor
```

Covered in full by its own book: **ztmux — tmux port (companion)**.

How to read the rest of this compendium

Those seven are the tools with dedicated volumes; everything after this chapter is a tool that has no book elsewhere and therefore gets its full chapter here. When a later chapter references a shell primitive, a parallel builtin, an AWK cache detail, or a statusline segment, the authoritative treatment lives in the volume named above — this index is deliberately thin on internals so those books stay the single source. Two structural facts carry across the fleet and are worth holding while you read on: five of these tools converge on one bytecode VM and JIT (**fusevm**), which is why their performance stories rhyme; and each porting effort treats the upstream source as the specification, with the original engineering being the Rust architecture, the binary suite, the drop-in compatibility, and the measured parity — not the invention of the language or format being ported.

lssofrs — a Rust reimplementation of lsof

`lssofrs` lists the open files of every process on the system: regular files, directories, character and block devices, FIFOs, pipes, kqueues, and — the part most people actually reach for it for — sockets. It is a Rust rewrite of `lsofng`, which is itself a modernized fork of Vic Abell's original `lsof`(8). The lineage matters, because the design of the tool is not an invention: the option grammar, the columnar output, the `-i` network selector, the field-output mode, the `+L` link-count filter, and the terse `-t` form are all lifted from `lsof` and reproduced faithfully. The original engineering here is the port — the FFI against the Darwin kernel, the `/proc` walker on Linux, the `sysctl` path on FreeBSD, the parallel gathering, and a dense set of interactive TUI modes layered on top of the classic single-shot behavior. The binary is published on crates.io as `lssofrs` (`Cargo.toml:2`), version 4.9.1 (`Cargo.toml:3`), under MIT, and ships two identical binaries — `lssofrs` and the shorter `lsf` — both built from `src/main.rs` (`Cargo.toml:17-23`, `default-run = "lssofrs"`). The dependency set is small and durable: `clap` for parsing, `serde/serde_json` for interchange, `ratatui` + `crossterm` for the TUI, `nix` and `libc` for the syscall surface, `users` for name resolution, `regex` for command matching, `chrono` for timestamps, `rayon` for the parallel gather, `toml` + `dirs` for config (`Cargo.toml:25-38`). There is no build script, no network fetch at build time, and nothing that needs a C toolchain beyond what `libc` already links.

What it ports, and how honestly

The compatibility surface is deliberate. `src/cli.rs` does not just define a clap parser and stop; it carries a hand-written `normalize_lsof_args` pass (`cli.rs:368`) that runs *before* clap sees anything. `lsof`'s option grammar is not getopt-clean — it has a `+` prefix family (`+L`, `+d`, `+D`, `+c`, `+r`), options with attached arguments (`-o9`, `-sTCP:LISTEN`, `+L1`), and a long tail of platform-only flags that scripts pass but which mean nothing on a given host. The normalizer splits `--clusters` getopt-style through `expand_dash_cluster` (`cli.rs:423`), expands `+` tokens through `expand_plus_token` (`cli.rs:517`), maps each `lsof` option to its clap equivalent (a short flag, a `--long`, or nothing at all), and passes everything after `--` through untouched (`cli.rs:388-415`). The program name in `argv[0]` is copied verbatim so `lsf` and `lssofrs` both parse identically.

Two categories are worth calling out because they are where a naive rewrite would break real scripts. First, the accept-and-ignore set: single-char options like `-b` `-O` `-C` `-M` `-X` `-E` `-L` are consumed inside a cluster and dropped (`cli.rs:452`), `-z` `-Z` `-S` `-f` drop any attached argument and end the cluster (`cli.rs:456`), and `-A` `-k` `-m` `-D` `-e` are treated as taking a required argument, so the following `argv` token is swallowed if none is attached (`cli.rs:460-463`). A script that invokes `lsof` with, say, `-b` (avoid kernel blocking) or `-O` (bypass overhead) runs under `lssofrs` without error even though those flags have no meaning in this implementation. The `+` side mirrors this: `+w` `+f` `+E` `+M` `+X` `+m` are silent toggles and `+e` consumes a separate argument (`cli.rs:564-569`).

Second, the semantically overloaded flags are disambiguated exactly the way `lsof` does. `-s` with no argument means “always show size,” `-s` with a colon spec (`-sTCP:LISTEN`) means “filter sockets by state,” and a `-s` followed by a non-colon rest is still treated as size-always with the cluster continuing (`cli.rs:479-491`). `-o` optionally takes a leading digit count for offset precision (`-o9` = nine digits), which the normalizer peels off character by character and re-emits as `--offset-always` plus `--offset-digits` (`cli.rs:465-476`). `+L` alone lists the NLINK column; `+L1` additionally selects only files whose link count is below 1 — i.e. unlinked-but-open files, the classic deleted-file-holding-a-disk-block case (`cli.rs:521-531`). A handful of `lsof`-only flags are mapped to real behavior rather than ignored: `-T` becomes `--tcp-info`, `-x` becomes `--cross-over`

(follow symlinks/mounts while walking `+d/+D`), and `-K` becomes `--list-tasks` (`cli.rs:494-505`). `+c` sets the command-column width with `+c0` meaning unlimited, and `+r` sets repeat-until-empty with an optional attached interval (`cli.rs:546-561`).

The point of all this is drop-in substitution. You can alias `lsyf` to `lsf` and expect existing muscle memory and existing scripts to keep working, including the flags that do nothing here. The whole bridge is one file and one function pair, which keeps the compatibility contract auditable in one place rather than smeared across the parser.

Architecture

The crate is split cleanly between platform data-gathering, filtering, and output, with a large TUI subsystem bolted on — 29 modules under `src/`. `src/main.rs` is the dispatcher: it parses args, prints help or version and returns early, resolves the effective theme and refresh interval (CLI flags override the saved `~/.lsyfrs.conf` preferences, `main.rs:64-79`), then branches into exactly one mode. The dispatch order is explicit and matters, because several flags can coexist on the command line; the first match wins. It runs: unified TUI, watch, follow, top, monitor, leak-detect, the socket-stall classifier, live-summary (`--summary` with `-r`), generic repeat, and only then the single-shot modes — stale, ports, pipe-chain, csv, net-map, tree, summary, json, terse, field-output, and finally the default columnar printer (`main.rs:82-191`).

Platform enumeration is `#[cfg]`-gated behind three modules that all produce the same `Vec<types::Process>`:

- `src/darwin.rs` — macOS via `libproc` FFI. It declares `proc_listpids` and `proc_pidinfo` as raw `extern "C"` and calls them against `repr(C)` structs matched to the kernel headers, first to size the PID buffer, then to fill it. No shelling out, no parsing of another tool's output. Because macOS exposes socket send/rcv buffer *limits* alongside occupancy, the Darwin path fills in the `send_buf_size/rcv_buf_size` fields that the stall classifier later needs.
- `src/linux.rs` — Linux via the `/proc` filesystem. Per PID it reads `cwd`, `root`, and `exe` symlinks into the `cwd/rtd/txt` pseudo-descriptors Isof users expect, then walks `/proc/<pid>/fd` for the numbered descriptors. Sockets are correlated through a prebuilt socket map keyed by inode. Queue occupancies come from `/proc/net/tcp`, which does not expose the buffer limits, so `send_buf_size/rcv_buf_size` stay `None` on Linux.
- `src/freebsd.rs` — FreeBSD via `sysctl` plus `procfs`.

`gather_processes()` in `main.rs:193-211` selects the right backend at compile time and prints an “unsupported platform” line to `stderr` on anything else. Gathering is parallelized with `rayon` across PIDs, and username resolution is memoized per `Process` through a `OnceCell` so the `getpwuid` lookup runs at most once regardless of how many files that process holds (`types.rs:416`, `types.rs:466-472`). `Process` even hand-implements `Clone` so that a populated username cache is carried into the clone rather than recomputed (`types.rs:419-440`).

The core data model lives in `src/types.rs`. `FileType` enumerates the full Isof type vocabulary — `REG`, `DIR`, `CHR`, `BLK`, `FIFO`, `sock`, `LINK`, `PIPE`, `KQUE`, `unix`, `IPv4`, `IPv6`, `sysm`, `PSEM`, `PSHM`, `ATALK`, `FSEV`, plus an `Unknown(String)` fallback for raw type codes (`types.rs:41-104`). `TcpState` maps the eleven kernel TCP states to Isof's spelling (`ESTABLISHED`, `TIME_WAIT`, `FIN_WAIT_1`, and so on) with an `Unknown(i32)` catch-all, and `from_raw` decodes the numeric kernel codes 0–10 (`types.rs:174-209`). `FdName` is the descriptor identity — the special forms `Cwd`, `Rtd`, `Txt`, `Mem`, `Err` render as `cwd/rtd/txt/mem/err`, numbered descriptors carry an access suffix (`3r`, `3w`, `3u`) through `with_access`, and `Other(String)` handles anything else (`types.rs:243-288`). `OpenFile` carries everything a column can display — fd, access mode, lock char, type, device, size, offset, inode, nlink, name, an optional `name_append` (this is where `(deleted)` lands), plus platform extras `rdev`, `file_flags`, and `file_struct_addr` (`types.rs:291-327`). Its `SocketInfo` is richer than the columns show: local and foreign endpoints, protocol, TCP state, `recv_queue/send_queue` occupancy, `recv_buf_size/send_buf_size` limits, and raw `socket_options/socket_state` words (`types.rs:218-240`) — the queue and buffer fields exist specifically to feed the backpressure classifier. The selection-flag constants (`SEL_CMD`, `SEL_FD`, `SEL_NET`, `SEL_PID`, `SEL_UID`, and the composite `SEL_PROC/SEL_FILE/SEL_`

NW unions) are the C implementation's bitmask scheme reproduced verbatim, and three tests pin each union to its documented bit-OR ([types.rs:9-37](#), [types.rs:519-531](#)).

Filtering is centralized in [src/filter.rs](#). `Filter::from_args` parses the comma-separated, `^`-negatable PID/user/command/fd selectors into separate include and exclude vectors, compiles `/regex/` command specs into `regex::Regex`, and precomputes prefix strings for file and directory matching so the hot path avoids re-allocating. `-s` protocol:state specs are parsed into `StateFilter` structs with an uppercased protocol and a state list where empty means “any state for this protocol.” The `-a` flag flips the whole thing from OR to AND semantics, matching Isof’s selection algebra. `gather_and_filter` in [main.rs:281-297](#) applies the process predicate, prunes each surviving process’s file list, drops processes left with no files (unless `-t` terse is set, which keeps bare PIDs), and finally sorts by PID for stable output.

Output splits across [src/output.rs](#) (columnar and field formats, ANSI theming), [src/json.rs](#) (serde-derived JSON), and [src/csv_out.rs](#) (RFC 4180 CSV). The remaining modules are the specialized modes — [summary](#), [tree](#), [stale](#), [ports](#), [pipe_chain](#), [net_map](#), [watch](#), [follow](#), [leak](#), [delta](#), [monitor](#), [stall](#), [top](#) — each a self-contained view over the same gathered `Vec<Process>`. The live TUI framework is [src/tui_app.rs](#) (a `TuiMode` trait giving every live mode the same keybindings, alternate-screen handling, and atomic frame rendering) with [src/tui_tabs.rs](#) implementing the unified tabbed dashboard on [ratatui](#) and [crossterm](#).

The CLI surface

The classic invocations are unchanged from Isof:

```
lsf                # every open file on the system
lsf -p 1234         # files held by PID 1234
lsf -c nginx        # command starting with "nginx"
lsf -c '/nginx|apache/' # command matched by regex
lsf -u root,www-data # files for either user
lsf -u ^root        # every user except root
lsf -a -p 1234 -i    # AND: PID 1234's network files
lsf -d 0-10         # only descriptors 0 through 10
lsf /var/log/syslog # who has this path open
lsf -t -c nginx     # bare PIDs, for kill $(lsf -t ...)
```

The network selector follows Isof’s `-i [4|6][proto][@host][:port]` grammar. `NetworkFilter` in [types.rs:476-492](#) holds an optional protocol, address family, address, host, precomputed `parsed_host` (the host string parsed once as an `IpAddr` for O(1) endpoint comparison rather than re-parsing per file), and a port range:

```
lsf -i              # all internet connections
lsf -i 4            # IPv4 only
lsf -i 6            # IPv6 only
lsf -i TCP          # TCP only
lsf -i :443         # port 443
lsf -i TCP:443      # TCP on port 443
lsf -i 4TCP:22      # IPv4 TCP on port 22 (one token)
lsf -s TCP:LISTEN   # TCP sockets in LISTEN state
```

Output formats cover scripting and interchange. `-t` is terse (PIDs only); `-F` is Isof field output where each requested field is emitted on its own line prefixed by a type letter — `p` pid, `c` command, `g` pgid, `R` ppid, `u` uid, `L` login name, `f` fd, `a` access, `t` type, `D` device (hex), `s` size, `o` offset (as `0t<n>`), `i` inode, `n` name, `P` protocol, `T` TCP info (emitted as `TST=<state>`) ([output.rs:426-499](#)), with `p f n` as the default field set when the argument is empty ([output.rs:417-418](#)). `-0` swaps the field terminator from newline to NUL for safe parsing of paths with embedded whitespace. `-J/--json` emits a JSON array via `serde`; `--csv` emits RFC 4180 CSV. When stdout is not a TTY, columnar output drops its colors and cyberpunk headers and prints plain, script-safe text ([main.rs:57-62](#)); the `--color` flag forces `always/never/auto`. A version query is verbatim Isof-shaped:

```
$ lsf -v
lsufrs 4.9.1
```

Terse mode is the pipeline workhorse — one PID per line, deduplicated by the sort-and-retain in [gather_and_filter](#):

```
$ lsf -t -c zsh
13855
13860
13925
13927
```

Beyond the lsof-compatible core, [lsufrs](#) adds a set of derived views that are all single-shot when piped and interactive when attached to a terminal:

```
lsf --tree           # process tree + per-node FD breakdown
lsf --summary        # aggregate FD stats, bars, top procs
lsf --ports          # listening TCP/UDP ports (cross-platform)
lsf --net-map        # connections grouped by remote host
lsf --pipe-chain     # pipe / unix-socket IPC pairs
lsf --stale          # FDs pointing at deleted files (+L1)
lsf --top 10         # live top-N processes by FD count
lsf --watch /tmp/x.sock # +OPEN / -CLOSE events for one path
lsf --follow 1234    # one process's FDs in real time
lsf --leak-detect=10,5 # flag FD counts rising 5 times running
lsf --stall          # per-FD socket backpressure classifier
lsf --monitor        # full-screen top(1)-style refresh
lsf --delta -r 2     # repeat every 2s, color new/gone FDs
lsf --tui            # unified tabbed dashboard for the above
```

Every one of these accepts the same selection filters, so [lsf --tree -u root](#), [lsf --stale -u www-data --json](#), or [lsf --ports -u root](#) all compose. The leak detector defaults to a 5-second interval and a 3-consecutive-increase threshold, clamps the threshold to a minimum of 2, and parses an optional [interval,threshold](#) spec ([cli.rs:346-357](#)). The [-V](#) flag reproduces lsof's "report search items that matched nothing" behavior: after filtering, [unfound_specs](#) walks the positive [-p/-c/-u/file](#) specs and prints a [no PID found/no command found/no user found/no file found](#) line to stderr for each that matched no process, skipping exclusions (^) and command regexes ([main.rs:216-279](#)). Directory arguments count as matched when any held file lives under them, so [-V /tmp](#) is satisfied by [/tmp/anything](#).

--ports

[--ports](#) is the [ss -tlnp](#) analogue and its selection rule is exact ([ports.rs:22-54](#)): a file qualifies only if its [SocketInfo](#) protocol is TCP or UDP; TCP additionally must be in [LISTEN](#) state; UDP qualifies when it has a bound local port greater than zero. SCTP and other protocols are excluded, established/[TIME_WAIT/CLOSE_WAIT](#) TCP sockets are excluded, and protocol matching is case-insensitive. Rows are grouped into a [BTreeMap<u16, ...>](#) keyed by port, which sorts the output ascending by port number. A run on a developer laptop:

=== Listening Ports ===

PROTO	LOCAL ADDR	PORT	PID	USER	COMMAND
TCP	0.0.0.0	6986	13890	user	WavesLocalServer
TCP	::	6986	13890	user	WavesLocalServer
UDP	0.0.0.0	7579	13917	user	NIHostIntegration
UDP	::	55135	13752	user	replicatord
TCP	0.0.0.0	56230	13676	user	rapporstd
UDP	203.0.113.3	59961	86624	user	vpn Helper

(Login name and remote addresses above are sanitized for print; the tool shows the real values.) The column

widths are computed from the data with sane minimums, alternating rows get a subtle background, and a `N listening port(s) found` footer closes the block. `--net-map` uses the mirror-image rule: it keeps only IPv4/IPv6 files with a *foreign* endpoint, skips wildcard/unconnected sockets, and groups by remote host into a `HashMap`, accumulating per-host protocol sets, port sets, and the distinct processes talking to that host (`net_map.rs:26-90`).

--summary

`--summary` aggregates the whole scan into a type histogram and a top-N-by-FD table. It is one of the modes that goes live when combined with `-r` (`main.rs:125-128` routes `--summary -r` to `run_summary_live`). A single-shot run:

```
=== lsofirs summary ===
```

```
Processes: 431          Open files: 4,032
```

```
-- File type breakdown --
```

REG	1,739	43.1%	
CHR	1,326	32.9%	
DIR	422	10.5%	
PIPE	196	4.9%	
unix	186	4.6%	
IPv4	55	1.4%	
FIFO	45	1.1%	

```
-- Top 10 processes by FD count --
```

PID	COMMAND	USER	FDs
13649	UserEventAgent	user	247
86359	vpnd	user	246
13711	corespotlightd	user	136

--stall

The socket backpressure classifier is the newest live mode and has no lsof antecedent (`src/stall.rs`). It samples every socket FD's kernel send/rcv buffer occupancy across successive scans (`--stall` is meant to be paired with `-r` to set the interval) and issues a directional verdict per `(pid, fd)`: `TX-STALLED` when the send buffer stays persistently near capacity and is not draining — the local process keeps writing but the peer or the network is not reading; `RX-STARVED` when the receive buffer stays persistently high and grows — bytes have arrived but the process is not `read()`-ing them fast enough; and `HEALTHY` otherwise. `StallDetector` keeps a bounded per-FD history (`HISTORY_SIZE = 16`), requires at least three consecutive samples before it will commit to a verdict (`MIN_SAMPLES = 3`), and treats an occupancy at or above 80% of the buffer limit as “near capacity” (`HIGH_RATIO = 0.80`) (`stall.rs:27-34`). Classification adapts to platform: on macOS all four inputs are available so it uses a fill-ratio rule (`tx_severity/tx_severity` compare queue against buffer size); on Linux, where `/proc/net/tcp` gives occupancy but not the limit, it falls back to a trend rule — persistently non-zero and non-draining for TX, persistently non-zero and strictly growing for RX (`stall.rs:265-324`). When both directions are backed up, the higher-severity side wins (`stall.rs:328-347`). PID reuse is handled: if the command name on a `(pid, fd)` changes, the history is cleared and the verdict resets. Only currently-seen stalled sockets are printed, so a quiet system reports `No stalled sockets detected`.

The TUI layer

The interactive modes are where `lsOfrs` departs most from anything in the `lsOf` tradition. `--tui` opens a full-screen tabbed dashboard — TOP, SUMMARY, PORTS, TREE, NET-MAP, PIPES, STALE — navigable by clicking, `Tab`/arrow, or number keys 1–7. There are 31 built-in color themes, confirmed by both the enum-length assertion in `theme.rs` (`ThemeName::ALL.len() == 31`, `theme.rs:860`) and the documented list in `lsOfrs.default.conf` (NeonSprawl through Zaibatsu). A theme chooser (`c`) previews them live with color swatches; a theme editor (`C`) builds custom six-color palettes and writes them to `~/.lsOfrs.conf`. Config persistence is TOML via `src/config.rs`, and the `Prefs` struct stores more than the visible knobs suggest: the selected theme, refresh rate, border toggle, the last active tab, a named map of custom themes, the active custom theme, a list of pinned PIDs, and the `sort_frozen`, `compact_view`, and `hover_tooltips` toggles (`config.rs:24-56`). Everything is `#[serde(default)]`-guarded so an old or partial config never fails to load — a missing field falls back to its default rather than erroring, which is the forward-compat behavior you want for a file the tool rewrites itself. `load()` reads `~/.lsOfrs.conf` and silently returns defaults on any parse error (`config.rs:88-99`); `save()` is compiled out under `#[cfg(test)]` so the test suite never touches the user's real config (`config.rs:102-114`). All live modes share a common keymap through the `TuiMode` trait — digit keys set the refresh interval, `p` pauses collection, `/` opens a regex filter popup, `y` copies the selected row, `e` exports the tab, `q/Esc/Ctrl-C` quit. Non-TTY output never enters the TUI; it prints one snapshot and exits, which keeps every mode pipeline-safe.

Repeat, delta, and the live loop

The plain repeat path (`-r N`, no dedicated mode) lives in `run_repeat` (`main.rs:299-365`). It optionally wires a `delta::DeltaTracker`: each iteration calls `begin_iteration`, gathers and filters, records every current `(pid, fd, name)`, and after printing counts what went away with `count_gone`. When `--delta` is set the columnar printer receives a per-row classifier closure that returns `New/Gone/Unchanged`, so appearing FDs are colored and vanished ones are listed after the table, followed by a one-line change summary and a `=====` separator between frames. JSON, terse, and summary outputs are honored inside the loop too, so `lsf --summary -r 2` and `lsf -J -r 2` both stream. Leak detection (`run_leak_detect`) and the stall classifier (`run_stall`) follow the same shape — construct a stateful detector once, then `update/report` on each interval — which keeps all three live analyzers structurally identical and easy to reason about (`main.rs:367-389`).

Integration into the stack

`lsOfrs` installs as two binaries on `PATH` (`lsOfrs` and `lsf`), a zsh completion, and two man pages. The completion at `completions/_lsOfrs` is a real `#compdef lsOfrs lsf` function covering both command names, built on `_arguments -C -s -S` with dedicated state handlers — `->filedes` for `-d`, `->addresses` for `-i`, `->states` for `-s`, `->fields` for `-F` — and it reuses the shell's own `_pids`, `_users`, and `_pgids` completers with comma-sequencing (`_sequence -s ,`). It drops into any `fpath` site-functions directory. The man pages split by depth — `lsOfrs.1` is the short reference (dated `2026-07-02`, versioned `lsOfrs 4.9.1`) and `lsOfrsall.1` documents every flag, mode, theme, and architectural note. `lsOfrs.1` defines a real exit-status contract (`0` success, `1` on error or no match), a `SEE ALSO` block pointing at `lsOf(8)`, `ss(8)`, `fstat(1)`, `netstat(1)`, `pstree(1)`, `top(1)`, and `proc(5)`, and a full OPTIONS section that mirrors the `-h` help screen. On a machine that already has `lsOf`, aliasing `lsOf=lsf` is the intended path: the accept-and-ignore flag handling exists specifically so that existing tooling — anything that shells out to `lsOf` with platform-specific options — continues to run unmodified.

Parity and its limits

The parser reproduces the `lsOf` grammar; the data model reproduces `lsOf`'s type and state vocabularies; the output formats (`-F`, `-t`, columnar, `-o`, `-s`, `+L`) match `lsOf`'s spelling, down to details like the `0t<offset>`

encoding and the `TST=<state>` field prefix. Where `lsof.rs` diverges is in the flags it accepts-and-ignores rather than implements — the kernel-tuning and mode flags like `-b`, `-O`, `-D`, `-K` (the last mapped to `--list-tasks` rather than dropped) are honored syntactically, but the kernel-blocking, overhead-bypass, and device-cache behaviors behind `-b/-O/-D` are absent, not replicated. That is the correct tradeoff for drop-in script compatibility, but it means the behaviors are syntactic no-ops. Platform coverage is macOS, Linux, and FreeBSD; anything else prints an unsupported-platform notice to stderr and returns nothing (`main.rs:206-210`). The socket buffer-limit inputs the stall classifier prefers are macOS-only; on Linux it degrades to the trend heuristic rather than fabricating a ratio.

Status

Version 4.9.1 per `Cargo.toml:3`, MIT-licensed, edition 2024, published to crates.io with a Homebrew tap and a docs site. The test surface is large: grepping `#[test]` returns 1,029 functions inline across `src/` and 710 more across the integration files in `tests/`, plus 21 shell-based documentation and formatting checks (the `tests/*.sh` set that gates man-page synopsis structure, final newlines, README badges, and workflow hygiene). Those counts come from `grep`, not a `cargo test` run, so treat them as inventory rather than a green-suite claim. The tests are behavioral, not decorative — `cli.rs` alone carries well over a hundred parse-combination cases pinning flag interactions, and `stall.rs`, `ports.rs`, and `types.rs` each verify their classification rules against hand-built fixtures. The release profile is tuned for size and speed (`opt-level = 3`, `lto = true`, `codegen-units = 1`, `strip = true`, `Cargo.toml:43-47`). The README publishes hyperfine benchmarks claiming a 5–21× advantage over `ls` 4.91 and `lsfd` on macOS; those numbers are the project's own measurements on one machine (roughly 470 processes and 5,000 open files — consistent with the 431/4,032 snapshot captured above), and they were not reproduced here, so they should be read as the author's reported figures rather than independently verified results.

nmaprs — a Rust port of nmap's scan engine and CLI dialect

What it is and what it ports

`nmaprs` is a network scanner written in Rust that reimplements the command-line surface and the core scanning techniques of **Nmap**, the network-mapping tool that has been the reference port scanner since 1997. The upstream is named plainly in the project's own `Cargo.toml` description — “High-performance parallel network scanner with nmap-compatible CLI surface” — and throughout the README, which points anyone needing “authoritative Nmap NSE/OS DB behavior” back to Nmap itself. This honesty is the correct framing for the tool: the scan *techniques* (SYN half-open, NULL/FIN/Xmas, ACK/Window/Maimon, UDP, SCTP INIT/COOKIE-ECHO, idle scan, IP-protocol scan, FTP bounce) are Nmap's inventions, the file formats it consumes (`nmap-service-probes`, `nmap-os-db`, `nmap-protocols`) are Nmap's data files, and the flag vocabulary (`-sS`, `-sU`, `-Pn`, `-oX`, `-T4`, and the long-option set) is Nmap's. What `nmaprs` contributes is the engineering of the port: a parallel Rust implementation of those techniques, a `clap`-driven CLI that accepts Nmap's glued-token syntax, and a worker-pool scan engine that beats Nmap on wall-clock for the common small-scan cases while consuming the same databases.

The tool is explicit about where the port stops. The full **NSE Lua runtime is not embedded** — `--script` and `-sC` provide a small set of built-in Rust “scripts” (banner grab, a `default` set) rather than the thousands of Lua scripts Nmap ships. OS detection (`-O`) is partial: IPv4 has a raw probe engine that scores against `nmap-os-db`, but IPv6 remains a TTL-heuristic guess. Version detection (`-sV`) parses `nmap-service-probes` and runs the probes but skips Perl-only regex constructs that Rust's `regex` crate cannot express. These boundaries are documented in a capability matrix in the README rather than glossed over, which is the right posture for a port that is genuinely useful without claiming byte-for-byte fidelity.

The crate is at version **0.1.9** (from `Cargo.toml`), licensed MIT, published to crates.io with a Homebrew tap for installation, and attributed to the organization MenkeTechnologies. The man pages carry a “May 2026” date and stamp `nmaprs 0.1.9`, so the documentation is version-synchronized with the crate.

The binary suite

`Cargo.toml` declares one library (`src/lib.rs`) and two binaries. The primary binary is `nmaprs` (`src/main.rs`, seven lines — its entire `main` calls `nmaprs::run_from_cli_env()`); the second is `nms` (`src/bin/nms.rs`), a short alias whose body is the same call. Both binaries share identical CLI parsing and behavior — `nms` exists purely to save keystrokes at the prompt, the same convention MenkeTechnologies uses across the fleet. A `scan` Criterion benchmark harness is registered as well.

`run_from_cli_env` in `lib.rs` is the shared entry: it initializes `tracing_subscriber` from the `RUST_LOG` env filter, defaulting to `warn` when none is set, then parses argv, builds a multi-threaded `tokio` runtime with `enable_all()`, and blocks on the async `run(args)` before exiting with its return code. The `warn` default keeps normal runs quiet — informational chatter lives behind `info!/debug!` and only surfaces when the operator raises the log level.

The release profile is tuned for a shipped CLI tool: `lto = true`, `codegen-units = 1`, `panic = "abort"`, and `strip = true`. The benchmark profile inherits from release so microbenchmark numbers reflect the same optimization level as the installed binary.

The dependency set is conventional and durable rather than fashionable. Async runtime is `tokio` with the `full` feature; raw packet construction uses `pnet` and `pnet_sys` (0.35); TLS for `-sV` SSL probes is `rustls` 0.23 with the `ring` backend, `tls12`, and `webpki-roots`; DNS resolution for `--dns-servers` is `hickory-resolver` 0.25; proxy support is `tokio-socks`; concurrent shared maps are `dashmap` 6.1; CLI parsing is `clap` 4.5 with `derive`, `env`, and `string` features; error handling splits between `anyhow` at the boundaries and `thiserror` 2.0 for typed errors. CRC32 for SCTP checksums comes from `crc32c` and `crc32fast`. There is one piece of C++ FFI: `c/expr_match.cpp`, compiled by `build.rs` via the `cc` crate at `opt_level(3)` as `nmap_expr_match`, providing the OS-fingerprint expression matcher. The dev-dependencies (`assert_cmd`, `predicates`, `criterion`, `tempfile`) drive the integration and benchmark suites.

Source layout

The source is roughly 24,000 lines across 35 files under `src/`. The largest and most load-bearing modules map directly to scan techniques and pipeline stages:

- `src/config.rs` (2,314 lines) — the `ScanPlan` and configuration model, the single largest module.
- `src/os_scan.rs` (1,860) — the raw IPv4 OS-fingerprint probe engine (SEQ×6, ECN, T2–T7, U1, IE×2 probes).
- `src/syn.rs` (1,760) — raw half-open TCP: SYN, NULL, FIN, Xmas, ACK, Window, Maimon, plus the sharded recv/send pipeline.
- `src/lib.rs` (1,552) — the top-level orchestration, target expansion (`expand_specs_ordered`), and host batching (`host_batches`).
- `src/vscan.rs` (1,499) — version scanning against `nmap-service-probes`.
- `src/discovery.rs` (1,437) — host discovery (the `-P*` family).
- `src/sctp.rs` (1,182) and `src/ip_proto.rs` (1,137) — SCTP and IP-protocol scans.
- `src/scan.rs` (1,171) — the TCP-connect worker pool and scan-loop core.

Smaller modules cover the surrounding machinery: `argv_expand.rs` (glued-token normalization before clap), `scanflags.rs` (custom TCP flag sets), `idle.rs` (zombie idle scan), `ftp_bounce.rs`, `icmp_listen.rs` and `icmp_ping.rs` (raw ICMP), `ipv6_l4.rs`, `os_db.rs` / `os_fp_db.rs` / `fp_match.rs` / `os_detect.rs` (OS DB parsing and matching), `output.rs` and `skiddie.rs` (output formats), `resume.rs`, `trace.rs`, `nse.rs`, `ports.rs`, `target.rs`, `net_util.rs`, and `help_tp.rs` (the styled help screen). All are re-exported as public modules from `lib.rs`, so the crate doubles as a library — the `docs.rs` documentation link in `Cargo.toml` reflects that the internal API is part of the shipped surface, not just the binary.

Argv expansion: the glued-token front end

Before `clap` ever runs, `argv_expand::expand_nmap_style_argv` rewrites Nmap's compressed short-option grammar into the separated long-option form that clap's derive parser expects. This is the seam that lets `nmaprs` accept a real Nmap command line verbatim. The rewriter walks argv token by token and applies a fixed set of rules: `-sL/-sn/-s0` map to their `--sL/--sn/--s0` long forms; `-sI` consumes a following zombie argument; `-iL/-iR` become `--iL/--iR`; three-character `-oN/-oX/-oS/-oG/-oA/-oM/-oH` become `--oN...`; a two-letter `-sX` splits into `--scan-type X` unless it is the overloaded `-sV` ($\rightarrow$ `--version-scan`) or `-sC` ($\rightarrow$ `--script-default`); `-Pn` becomes `--no-ping`; the ping family `-PS/-PA/-PU/-PY` splits off any glued port list (`-PS80,443` $\rightarrow$ `--ping-S 80,443`); `-PO` becomes `--ping-ip-proto` with Nmap's default `1,2,4` protocol list when given bare; `-PE/-PP/-PM` map to `--ping-E/-P/-M`; `-T0...-T5` become `--timing N`; and stacked `-v/-vv/-d/-dd` collapse to `--verbosity=N--debug=N` by counting the repeated letter. The ordering of the match arms matters and is commented as such: `-s0` is handled before the generic `-sX` arm so it is not mistaken for a scan-type char, and `-PO` is handled before the generic `-P*` arm so it is not read as `--ping-0`.

The net effect is visible when tracing a command through the front end:

```
$ nmaprs -sS -Pn -PS80,443 -T4 -oX out.xml 192.0.2.0/30
# after expand_nmap_style_argv, clap receives:
```

```
# --scan-type 5 --no-ping --ping-5 80,443
# --timing 4 --oX out.xml 192.0.2.0/30
```

`Args::parse_from_env` performs this expansion, then re-derives the binary basename from `argv[0]` (so `nms` shows `nms` in help), builds the clap command with that name, and intercepts clap's help/version errors to route them into `help_tp::print_help` / `print_version` rather than clap's default renderer. That is why the styled help screen appears instead of clap's plain output. A handful of Nmap "hidden" aliases are folded in after parsing through `effective_*` accessors: `--vv` adds 2 to verbosity, `--ff` adds 2 to debug, `--rH` is an alias for `--randomize-hosts`, and `--fuzzy` is an alias for `--osscan-guess`.

The ScanPlan and timing model

`config.rs` turns the parsed `Args` into a `ScanPlan`, the single value that every later stage consumes. It is a large struct — resolved `ports`, `concurrency`, `connect_timeout`, the primary `scan_kind` plus an `extra_scan_kinds` vector (so `-sS -sU` runs SYN and UDP together), an optional `tcp_scan_flags` byte from `--scanflags`, all the output paths, evasion options, proxy chain, and the timing-derived rate and retry knobs. `ScanKind` is the enum of techniques: `TcpConnect`, `TcpSyn`, `TcpNull`, `TcpFin`, `TcpXmas`, `TcpAck`, `TcpWindow`, `TcpMaimon`, `Udp`, `IpProto`, `SctpInit`, `SctpCookieEcho`, and `Idle`. Each variant carries a `flag_name()` for error messages and a `tcp_port_raw_kind()` that maps the seven raw TCP variants onto `syn::TcpPortScanKind`.

The timing template is the clearest place to see the port's concrete choices. `from_args` maps `-T0...-T5` (default `-T3`) to a `(base_concurrency, base_timeout_ms)` pair:

```
-T0 → 32 workers, 10000 ms connect timeout
-T1 → 64 workers, 5000 ms
-T2 → 128 workers, 2000 ms
-T3 → 256 workers, 1500 ms (default)
-T4 → 512 workers, 750 ms
-T5 → 1024 workers, 300 ms
```

`--max-parallelism` overrides the base concurrency (capped at 65,535), `--min-parallelism` raises it, and `--min-rtt-timeout` / `--max-rtt-timeout` / `--initial-rtt-timeout` adjust the connect timeout with `parse_duration`. `effective_probe_concurrency()` layers one more rule on top: when `--min-rate` is set and `--max-parallelism` was **not** given explicitly, it raises the concurrency floor toward `min(min_rate, 65_535)`, so a Nmap-style minimum throughput request is not silently capped by the default template. `from_args` also validates the invariants Nmap would — `--max-rate ≥ --min-rate`, `--max-hostgroup ≥ --min-hostgroup`, `--privileged` and `--unprivileged` mutually exclusive, `-sn` and `-sL` not combined — and rejects violations with a terse `bail!` rather than a wrong scan.

The scan pipeline

The README documents a seven-stage pipeline, and the module map confirms it:

1. **Argv expansion** (`argv_expand.rs`) normalizes Nmap's glued tokens as described above.
2. **Plan** (`config.rs`, `scanflags.rs`) builds a `ScanPlan`, including an optional custom TCP flag byte when `--scanflags` is present.
3. **Targets** (`target.rs`, `lib.rs`) expand IPv4/IPv6 literals, CIDR blocks, Nmap-style IPv4 octet ranges, DNS names, `-il` files, and `-iR` random-host counts. Expansion runs in parallel with stable ordering (`expand_specs_ordered`), so multiple `-il` lines and CLI targets resolve concurrently but print in input order.
4. **Discovery** (`discovery.rs`, `icmp_ping.rs`) runs before the port scan unless `-Pn` is set. The default fires raw ICMP echo concurrently with raw TCP SYN to ports 443 and 80; `-PR` ARP ping auto-runs for local-subnet IPv4 hosts. The full `-P*` family is supported: `-PS` (SYN), `-PA` (ACK), `-PU` (UDP, default port 40125), `-PY` (SCTP INIT, default port 80), `-PO` (IP-protocol ping, default protocols 1,2,4), and `-PP/-PM`

- (ICMP timestamp and address-mask, IPv4 Unix only). For `-PY` and `-P0` the IPv4 and IPv6 results are merged independently on partial failure, so a v6-only raw failure does not discard v4 liveness.
5. **Scan engine** (`scan.rs`, `syn.rs`, `sctp.rs`, `ip_proto.rs`, `ftp_bounce.rs`) executes the selected techniques, with optional `--min-hostgroup/--max-hostgroup` batching. `DEFAULT_HOSTGROUP_MAX` is 1024.
 6. **Post-scan** (`ping.rs`, `trace.rs`, `resume.rs`, `nse.rs`, `os_detect.rs`, `vscan.rs`, `skiddie.rs`) handles traceroute, OS guessing, version detection, checkpointing, and script-kiddie output.
 7. **Output** (`output.rs`) writes the requested report formats.

The performance story rests on stage 5. For the default TCP-connect scan, `tcp_connect_scan` in `scan.rs` chooses between two execution paths at runtime with `let use_blocking = plan.proxies.is_empty() && n >= 64`. When there are at least 64 probes and no proxy, it uses a **worker pool of OS threads**: a `TcpConnectCtx` (shared work vector, an `AtomicUsize next_idx`, timeout, retries, pacer, host deadlines) is wrapped in an `Arc`, results go into a `ResultSlots` array of `UnsafeCell<MaybeUninit<PortLine>>`, and `tokio::task::spawn_blocking` runs a `thread::scope` that spawns `conc` threads. Each thread loops `fetch_add(1, Relaxed)` on the shared index, breaks when it exceeds the work length, runs a blocking `std::net::TcpStream::connect_timeout`, and writes its result into the slot at its own index — exactly once per slot, so no lock is needed. The `unsafe impl Send/Sync for ResultSlots` is justified by that invariant, documented in the source: each slot is written by exactly one worker and read only after all workers join. The comment gives the rationale in syscall terms: the blocking probe is `socket + connect + close` (3 syscalls), where tokio's async path additionally sets the socket non-blocking with `ioctl(FIONBIO)` and registers it with the reactor (4+ syscalls). For small work sets (under 64 probes) or when `--proxies` are configured — where connect-through-proxy needs the SOCKS/HTTP-CONNECT state machine — the engine falls back to async tokio tasks draining the same atomic index. An optional `progress AtomicUsize` is incremented after each probe for `--stats-every` reporting.

The raw scan paths use a different structure. `syn.rs` runs a dedicated recv thread per address family plus main-thread sends, registering the reply key in a `DashMap` before each send to avoid a recv race, and shards work across up to `MAX_SYN_PARALLEL_SHARDS = 16` independent pipelines per family (each with its own recv thread and 64 KiB `RX_BUF`). `split_into_syn_chunks` divides the ordered work into balanced contiguous chunks so each shard keeps port ordering locally. Mixed IPv4+IPv6 target sets run both families concurrently under `tokio::join`, and `--max-retries` runs additional full rounds on a fresh raw socket for probes still unresolved, dropping scan-delay and rate pacing on later rounds to match TCP-connect retry behavior. The same sharded pattern is reused by SCTP (`sctp.rs`, CRC32c checksums, IPv4 Layer3 datagrams plus IPv6 raw protocol 132) and by the IP-protocol scan (`ip_proto.rs`). Rate control is centralized: `ProbeRatePacer` installs a shared `Mutex<Instant>` spacing gate only when `--max-rate` is set — a `--min-rate` floor alone adds no mutex overhead — and one pacer is shared across concurrent IPv4 and IPv6 senders so the ceiling is global.

The main loop

`run` in `lib.rs` is the async driver. It short-circuits the informational modes first — `--script-help` and `--script-updatedb` print an acknowledgement and exit 0 (there is no bundled NSE database), and `--iflist` enumerates interfaces via `if_addrs` and exits. It then builds the `Arc<ScanPlan>`, loads the OS fingerprint DB in a `spawn_blocking` task only if `-O` was requested, and calls `trace_nmap_compat`, which logs (at `debug` level, to the log only) when packet-level or resolver options are set that are accepted for parity but not fully wired. Absent any target, `-iL`, `-iR`, or `-sL`, it bails. `--sL` (list scan) expands targets and prints them without scanning. Otherwise it collects hosts, applies `--exclude/--excludefile` via `apply_exclude`, deduplicates when `--unique` is set, shuffles when `--randomize-hosts/--rH` is set, and — unless `-sn` or `-Pn` — runs `discovery::hosts_after_discovery` to drop hosts that did not answer. The `-sn` ping-only path is a self-contained branch: it opens the output set, writes headers with any `--stylesheet/--webxml` handling, pings hosts with `ping::ping_hosts`, optionally attaches an OS guess line (from the fingerprint DB's `format_os_guess` or the TTL heuristic `os_detect::guess_from_ttl`), writes per-host records to every open format, emits a footer with up/down/total counts, and optionally runs traceroute. Traceroute itself is

bounded parallel: `trace.rs` caps concurrency at `MAX_TRACEROUTE_PARALLEL = 32` and preserves scan output order.

The CLI surface

`src/cli.rs` (653 lines) defines the `Args` struct with `clap` derive. The `about` line reads “NMAPRS // GRID SCANNER — rust-native parallel port probe.” The flag inventory mirrors Nmap closely. Scan-type selectors include `-sT` (connect, default), `-sS` (SYN), `-sU` (UDP), `-sN/-sF/-sX` (NULL/FIN/Xmas), `-sA/-sW/-sM` (ACK/Window/Maimon), `--sO` (IP protocol), `--sI` with a zombie (idle), `-sY/-sZ` (SCTP), `-b` (FTP bounce), and `-sn` (ping scan only). Host-discovery probes are the `-P*` set exposed through long options like `--ping-S`, `--ping-A`, `--ping-U`, `--ping-Y`, `--ping-E`, `--ping-P`, `--ping-M`, and `--ping-ip-PROTO`. Port selection uses `-p` (with `-p` shorthand for all 65536), `-F` (fast), `-r` (sequential, no randomize), `--top-ports`, `--port-ratio`, and `--exclude-ports`.

Timing and pacing controls are Nmap-shaped: `--timing/-T` (0–5 templates), `--min-parallelism`, `-M/--max-parallelism`, `--min-rtt-timeout`, `--max-rtt-timeout`, `--initial-rtt-timeout`, `--max-retries`, `--host-timeout`, `--scan-delay`, `--max-scan-delay`, `--min-rate`, `--max-rate`, `--min-hostgroup`, and `--max-hostgroup`. The evasion set is wired into raw packet construction: `-f/--mtu` (fragmentation), `-D` (decoys), `-S` (source spoof), `-e` (interface), `-g/--source-port`, `--data/--data-string/--data-length` (custom payload), `--ttl`, `--badsum`, `--ip-options`, and `--spoof-mac` (applied to ARP discovery frames). Proxying is `--proxies` with a `--proxy` visible alias; custom DNS is `--dns-servers`, with `--system-dns` to force the OS resolver. Beyond the flags in `nmap --help`, the struct also carries the extended long-option set from Nmap’s `nmap.cc` — `--resolve-all`, `--defeat-rst-ratelimit`, `--defeat-icmp-ratelimit`, `--discovery-ignore-rst`, `--max-os-tries`, `--nsock-engine`, `--release-memory`, `--versiondb`, `--servicedb`, `--stats-every`, and the hidden `--vv/--ff` aliases — several of which are accepted for parity and logged rather than acted on. The `_nmaprs` zsh completion file mirrors this surface for both `nmaprs` and `nms`, and starts with `#compdef nmaprs nms` for installation onto `fpath`.

A representative set of invocations, all grounded in the README’s example section:

```
# TCP connect – top ports from the embedded frequency table
nmaprs scanme.example.com
nms scanme.example.com           # identical via the short binary

# Ping scan only, no port scan
nmaprs -sn scanme.example.com

# UDP, top 100 ports
nmaprs -sU --top-ports 100 target

# Targets from a file, resume checkpoint, normal output
nmaprs -iL hosts.txt --resume state.json -oN out.txt

# IPv6, explicit ports, a /126 block
nmaprs -6 -p 80,443 2001:db8::/126
```

Raw scans need privilege. `nmaprs` auto-detects root/sudo via `geteuid()` and silently falls back from SYN to TCP connect when unprivileged; `--privileged` forces raw mode, `--unprivileged` forces connect mode and rejects raw-only techniques (`--sO`, `--sI`, SCTP) with an error rather than a wrong answer.

Output formats and parity

Output (`output.rs`, `skiddie.rs`) covers Nmap’s report formats. `-oN` is normal text, `-oG` is grepable, `-oX` is XML, `-oA` writes all three, and `-oS` is the “script-kiddie” leet-text variant that mirrors the `-oN` lines while stdout stays normal. The XML uses an Nmap-compatible `<nmaprun>` root with `<scaninfo>`, `<host>`, `<status>`,

<address>, <hostnames>, <ports>, <extraports>, <service>, and <runstats> elements, so existing XML-consuming tooling has a chance of parsing it. Two formats are partial: `-oM` writes the same grepable-style machine lines as `-oG`, and `-oH` creates a placeholder file (full hex capture is not implemented). Stylesheet handling (`--stylesheet`, `--webxml`, `--no-stylesheet`) emits the XML preamble and `xml-stylesheet` processing instruction when `-oX` is active.

A grepable line from a connect scan looks like Nmap's:

```
# nmaprs -oG
Host: 192.0.2.10 ( ) Status: Up
Host: 192.0.2.10 ( ) Ports: 22/open/tcp//ssh///, 80/open/tcp//http///
  Ignored State: closed (998)
```

The port-state model follows Nmap's semantics per technique, which is where the port earns its keep. TCP connect and SYN map SYN/ACK to `open` and RST to `closed`. Maimon (`-sM`) sends FIN+ACK but classifies replies like SYN (RST→`closed`, SYN/ACK→`open`). ACK scan (`-sA`) maps RST to `unfiltered` and no-reply to `filtered`, the firewall-detection interpretation. Window scan (`-sW`) reads the TCP window on the RST: non-zero window is `open`, zero window is `closed`, following BSD-derived stacks. UDP treats a reply as `open`, an ICMP port-unreachable as `closed`, and other unreachable codes as `filtered`; on Unix a single `poll(2)` listener thread drains ICMP errors for both families and `closed` wins over `filtered`. SCTP maps INIT-ACK/COOKIE-ACK to `open` and ABORT to `closed`; the IP-protocol scan maps ICMP protocol-unreachable (IPv4 type 3 code 2, or ICMPv6 parameter-problem type 4 code 1) to `closed` and timeout to `filtered`; the idle scan reads the zombie's IP-ID delta, treating a delta ≥ 2 as `open`. These are not cosmetic mappings — reproducing them correctly is the substance of matching Nmap's behavior, and the README's capability matrix documents each one precisely.

OS and version detection

`-sV` version detection (`vscan.rs`) loads `nmap-service-probes` and runs both TCP and UDP `Probe` blocks, honoring `ports/sslports` filtering (with rustls TLS for SSL ports), the `rarity` vs `--version-intensity` threshold, and `match/softmatch` lines with the full template field set — `p/` product, `v/` version, `i/` info, `o/` OS, `d/` device type, and `cpe:/` URIs. Each rule is a compiled `regex::bytes::Regex`; soft matches defer to later probes until a hard match is found, matching Nmap's staged behavior. Perl-only regex features that Rust's `regex` cannot express are skipped rather than mis-parsed, and per-open-port probing runs in parallel.

`-O` OS detection (`os_scan.rs`, plus the C++ `expr_match.cpp` FFI) is the more ambitious partial. On IPv4 with privileges and a `nmap-os-db` under `--datadir`, it sends Nmap's second-generation probes across phases: `NUM_SEQ_SAMPLES = 6` SEQ probes spaced by `OS_SEQ_PROBE_DELAY_MS = 100`, then ECN, T2–T7, U1, and two IE probes. From those it builds a subject fingerprint — SEQ/OPS/WIN/ECN/T1–T7/U1/IE tests, including `GCD/ISR` sequence-rate values, `seq_relation/ack_relation` classifications (Z/I/BI/RI/RD-style IPID, timestamp classes, CI from T5–T7, II from IE echo replies, shared-IPID detection) — and scores every reference entry with `MatchPoints` weighting via the `expr_match` C++ matcher. When the raw engine or the DB is unavailable, `-O` degrades to ICMP TTL heuristics with example DB titles. IPv6 OS detection remains TTL-oriented until a v6 probe path exists. `-A` bundles `-O` + `-sV` + default `-sC` scripts + `--traceroute`, the same aggregate flag Nmap defines, minus the NSE runtime.

Databases and integration

Three data files sit under `data/` (or a `--datadir` DIR). `data/top_ports.txt` holds 1,000 TCP port numbers in frequency order (embedded via `include_str!` and loaded once through a `OnceLock`); `top_ports(n)` takes the first `n`, `default_tcp_ports()` takes 1,000, and `fast_tcp_ports()` takes 100 for `-F`. `data/nmap_ip_protocols_fast.txt` is the IP-protocol fast list derived from Nmap's `nmap-protocols`, used for `-sO -F` to match Nmap's `[P:0-]` selection. The two large upstream databases — `nmap-service-probes` and `nmap-os-db` — are deliberately **not committed**; `scripts/fetch_nmap_data.sh` fetches them (with

`scripts/fetch_top_ports.sh` regenerating the port table), and without them `-sV` logs a warning and `-O` falls back to TTL-only heuristics. Keeping the multi-megabyte Nmap DBs out of the repo while consuming them at runtime is the pragmatic choice for a port. Port parsing enforces the real bounds — `-p -` expands to all 65536 values, `65535` is the maximum accepted, and `65536` is a parse error.

Shell integration follows the fleet convention. `completions/_nmaprs` is a zsh completion covering both `nmaprs` and `nms`; it is dropped into a directory on `fpath` before `compinit`. Two man pages ship: `man/man1/nmaprs.1` (short reference) and `man/man1/nmaprsall.1` (the full parity matrix and every technique, modeled explicitly on `zshall(1)` so the whole manual is one greppable page). The help screen (`help_tp.rs`) is styled after the `temprs` CYBERPUNK template with cyan/magenta/yellow color bands, and suppresses all escapes on a non-TTY or when `NO_COLOR` is set, so piped output stays clean.

Performance and status

The README reports hyperfine comparisons against nmap 7.99 on macOS arm64, all against 127.0.0.1 with `-n -Pn --min-itt-timeout 50ms --max-retries 0` and output to `/dev/null`. The headline range is **1.5–5.1× faster** across port counts, with the largest gains (3–5×) on small scans where Nmap’s startup cost (Lua/NSE init, libpcap, database loads) dominates, narrowing to 1.5–1.6× on full 65536-port sweeps where actual probe work dominates. A Criterion microbenchmark of a three-closed-port localhost connect scan reports roughly 74–82 μs. These are the author’s own measurements on one machine and configuration; treat them as directional rather than as guaranteed cross-platform figures. (*Provisional: benchmark numbers are as reported in the README and were not re-run for this chapter.*) The architectural basis for the connect-scan number is the worker pool described above — blocking `connect_timeout` cutting per-probe syscalls, and the lock-free `UnsafeCell` result array avoiding contention on write-back.

Test coverage is substantial. The `tests/` directory holds 29 files mixing Rust integration tests and shell-based doc/format guards; `tests/cli.rs` alone carries 204 test functions, with `cli_extra_flags.rs` (36) and several `contract_*` files (8, 8, 7) covering the port database, scanflags, OS DB parsing, resume edges, and target expansion. Counting inline `#[test]/#[tokio::test]` attributes across source and tests together yields roughly **1,468** test functions. (*Provisional: counts are from `grep` over the test attributes and may include helper or ignored tests; not all were executed here.*) The shell guard tests enforce repo hygiene — final newlines, man-page synopsis sections, README badges and HTTPS links, docs HTML structure, no inline handlers, no deprecated tags — alongside the Rust contract tests.

The honest summary is the one the README itself offers: `nmaprs` is a fast, parallel Rust implementation of Nmap’s scan techniques and CLI dialect, faithful on the connect/SYN/UDP/SCTP/IP-protocol/idle/bounce paths and their per-technique port-state semantics, partial on OS detection and version scanning, and explicitly without the NSE Lua runtime. For everyday scanning inside the shell it stands on its own; for authoritative NSE and OS-database behavior, the upstream Nmap remains the reference.

temprs — a stack-structured temporary-file manager for the shell

The name suggests a temperature probe. It is not. `temprs` (pronounced as “temp-R-S,” and shipped as the two-character command `tp`) is a manager for **temporary files**, organized as a stack. It gives the shell a persistent, named, index-addressable scratch space that survives between commands and between separate shell invocations, and it wraps that scratch space in a set of operations borrowed from two different traditions: the list/array primitives of a programming language (`push`, `pop`, `shift`, `unshift`, `swap`, `dup`, `reverse`, `sort`) and the file-inspection primitives of Unix (`head`, `tail`, `wc`, `grep`, `diff`, `cat`). The result is a single small binary that behaves like a clipboard stack for arbitrary command output, with every entry backed by a real file on disk that any other program can read.

It is worth being precise about what `temprs` ports and what it invents, because the house rule is to name the upstream honestly. `temprs` is **not** a Rust reimplement of a single named Unix utility — there is no `temp(1)` it descends from. What it does port, in the honest sense, is the *behavior* of several coreutils, scoped to its own stack. Two of those coreutils it reproduces in process — `--head`, `--tail`, `--grep`, `--cat`, and the `wc`-flavored line count are Rust code operating on the tempfile’s contents — and two it delegates to the system binary rather than reimplement: `--diff` spawns the platform `diff -u`, and `--edit` spawns `$EDITOR`. That distinction matters for a port-honest reading. Where the earlier framing said “reimplements the useful part of `grep`,” read it precisely: `--grep` is a literal substring scan (`str::contains`), not a regex engine, and `--diff` is not a reimplemented diff at all — it is a subprocess call that inherits the exit code of whatever `diff` is on `PATH`. The engineering that matters here is the *container* — the atomic, lock-protected, null-delimited master record and the stack algebra layered on top of it — not the invention of a diff.

What it is, concretely

Every time you pipe data into `tp`, it writes that data to a fresh file inside a managed directory and records the file at the top of a stack. The directory defaults to `$TMPDIR/temprs` (falling back through `std::env::temp_dir()`), and can be overridden wholesale by exporting `TEMPRS_DIR`. Each stored file is named `tempfile<milliseconds>` — the millisecond timestamp of creation, from `util_time_ms()`, is the on-disk identity — and the ordering, along with any human-assigned tag names, lives in a single sidecar file called `temprs-stack` in the same directory. That sidecar is the master record, and most of the tool’s engineering is spent keeping it honest under concurrent access.

The mental model is a stack of clipboards. The newest thing you pushed is index `-1` (or the highest positive index); the oldest is index `1`. You can read any entry back to stdout, pipe fresh data over an existing entry, append to one, open one in `$EDITOR`, or address it by a name you gave it instead of by number. Because the backing store is real files in a real directory, the stack persists across terminal sessions — close the shell, open a new one, and `tp -l` still lists everything you left there until you clear or expire it.

```
# capture some command output onto the stack
$ curl -s https://example.com/api/users | tp -w users
# ...later, in a different shell:
$ tp -l
/tmp/temprs/tempfile1749145203871
$ tp -o users | jq '.[].email'
```

There is a self-healing property baked into startup that the earlier chapter passed over. When `TempApp::new()`

reads the master record back in, it does not trust it blindly — it zips the stored paths against their names, filters with `.filter(|(p, _)| p.exists())`, and immediately rewrites the pruned record with `util_paths_and_names_to_file`. So if you or another program deletes a `tempfile*` out from under the stack, the phantom entry is dropped on the very next `tp` invocation rather than lingering as a dangling index. The master record converges toward the real directory contents every time the tool runs.

Architecture: crates, modules, binaries

The build produces two binaries — `temprs` and `tp` — from the same `src/main.rs`, which does nothing but construct a `TempApp` and call `run()`. The two names are byte-for-byte equivalent programs; `tp` exists purely because a temporary-file tool that you reach for dozens of times an hour needs a two-keystroke name. This is declared in `Cargo.toml` as two `[[bin]]` targets pointing at the same path, alongside a `[lib]` target (`src/lib.rs`) that exposes the internals for the test suite.

The dependency list is deliberately short and foundational, which matters for a tool meant to keep building for years:

Crate	Version	Role
<code>clap</code>	4.6	argument parsing (all 38 flags)
<code>fs2</code>	0.4	<code>flock</code> -style advisory locking (<code>lock_exclusive</code>)
<code>log / env_logger / simple_logger</code>	0.4 / 0.11 / 5.2	logging; runtime uses <code>simple_logger</code> at <code>Level::Info</code>
<code>criterion</code> (dev)	0.8	the <code>benches/benchmarks.rs</code> harness

The release profile is tuned for a small, fast static-ish binary: `lto = true, panic = "abort", strip = true, codegen-units = 1, opt-level = 3`. The crate is on the 2024 edition.

Internally the code splits into three layers. `src/model/state/mod.rs` defines `TempState`, the plain data record. It carries fifteen fields: the path of the file being written (`new_temp_file`), the master record path, the managed directory, and — the two parallel vectors that are the heart of the design — `temp_file_stack: Vec<PathBuf>` and `temp_file_names: Vec<Option<String>>`. Around those sit the transient per-invocation slots the CLI fills in: `arg_file`, `insert_idx`, `input_temp_file`, `output_temp_file`, `append_temp_file`, `name`, an `output_buffer` and a `holding_buffer` that shuttle piped bytes between input and output, plus the `silent` and `verbose` flags. Position `i` in both vectors describes one stack entry: its file and its optional tag. Keeping names in a parallel `Vec<Option<String>>` rather than inside the path is what lets a rename or a reorder move the tag with the file without touching the file itself. `TempState` is a pure struct with getters, mutable getters, and a `write_master()` that funnels the two vectors to the atomic writer — several of its accessors are `#[cfg(test)]`-only, existing solely so the test suite can construct and inspect state without a real lock file.

`src/model/app.rs` is the controller — `TempApp` — holding a `TempState` and, importantly, an owned `_lock_file: File`. The lock is acquired in `TempApp::new()` and held for the entire lifetime of the process; the leading underscore signals that the field exists only for its RAII side effect (the exclusive lock releases when the file handle drops at process exit). `run()` is a three-phase pipeline: `parse_opts()` to interpret the CLI, `input()` to read from stdin or the argument file, and `output()` to emit results. `input()` branches on `stdin().is_terminal()` so that `CMD | tp` reads the pipe while a bare `tp` at a TTY waits for typed input; `output()` mirrors that with `stdout().is_terminal()`, choosing the boxed cyberpunk renderer for a terminal and a plain byte-faithful `print!` for a pipe.

`src/model/opts.rs` holds the `clap` command definition and — this is the bulk of the source at roughly 47,000 lines — an enormous generated body of flag-recognition tests. `src/util/utils.rs` (~2,200 lines) is the free-function toolkit: path-to-string conversion, the millisecond clock, the atomic master-record writer, index transformation, the terminate-on-error helper, and the whole family of `cyber_*` terminal-output routines. `src/util/consts.rs` centralizes every string constant, from the directory name to each error message to all 38 CLI flag identifiers, so that nothing is stringly-typed at a call site.

The master record: null-delimited, atomic, locked

The single most load-bearing piece of engineering in `temprs` is how it persists the stack. Three problems have to be solved at once: filenames can contain any byte except NUL and `/`; the file can be written by several shells at the same moment; and a crash mid-write must never leave a half-written record.

The format solves the first problem with NUL delimiters. Within a record, the file path and its optional tag name are separated by a single `\0` (`MASTER_FIELD_DELIM`); records are separated by `\0\0` (`MASTER_RECORD_DELIM`). Because NUL is the one byte that cannot appear in a Unix path, this framing is unambiguous even for filenames containing newlines, tabs, or spaces. The reader, `util_file_to_paths_and_names`, splits on the double-NUL, skips any record that is empty after trimming, and for each surviving record uses `split_once(MASTER_FIELD_DELIM)` — a present-but-empty field becomes `None`, an absent delimiter means a bare untagged path. The writer in `util_paths_and_names_to_file` builds the buffer by zipping the path vector against the name vector, emitting `path` alone when the tag is `None` and `path\0name` when it is `Some`:

```
// src/util/utils.rs - record assembly, then atomic swap
let records: Vec<String> = paths.iter().zip(names.iter())
    .map(|(p, n)| match n {
        Some(name) => format!(
            "{}{}{}", util_path_to_string(p),
            MASTER_FIELD_DELIM, name),
        None => util_path_to_string(p),
    })
    .collect();
// ... join with "\0\0", append a trailing "\0\0", then:
let tmp = out.with_extension("tmp");
util_overwrite_file(&tmp, &buf);
if rename(&tmp, out).is_err() {
    util_terminate_error(ERR_MASTER_WRITE);
}
```

The atomicity comes from the last three lines: the new record is written to `temprs-stack.tmp` and then `renamed` over `temprs-stack`. On a POSIX filesystem `rename` within a directory is atomic, so a reader either sees the complete old record or the complete new one, never a torn write. An empty stack writes an empty buffer rather than a stray delimiter, so a cleared stack round-trips to zero records. The concurrency problem is handled a level up, in `TempApp::new()`, which creates `temprs-stack.lock` and calls `lock_exclusive()` on it via `fs2`; because the whole program runs while holding that lock, two shells racing to mutate the stack serialize cleanly rather than clobbering each other. Corruption tolerance is the fourth guarantee: records that fail to parse — empty or with an empty path field — are skipped on read (with a `warn!` to the log) and quietly dropped on the next write, so a damaged master file self-heals rather than wedging the tool.

The CLI surface

`temprs` exposes 38 operations, catalogued in `consts.rs` and wired to `clap` in `opts.rs`. Almost every index-taking flag sets `allow_hyphen_values(true)`, which is what lets `-1`, `-2` and friends arrive as *values* rather than being mistaken for short flags — negative indexing works precisely because of that per-argument setting. The operations fall into groups.

I/O and pipeline. `-i INDEX` writes stdin into an existing entry; `-o INDEX` reads an entry to stdout; `-A INDEX` appends stdin to an entry; `-v/--verbose` (a counting flag) raises the log level to `Debug` and echoes what was written; `-q/--quiet` sets a silent flag. Port-honest note: that silent flag is stored on `TempState` and asserted extensively in the state tests, but the runtime output path does not currently consult it — `-q` parses and records intent without yet suppressing anything at runtime. With no flags, `CMD | tp` pushes a new entry and `tp FILE` pushes a file's contents; if both a pipe and a positional `FILE` are present, the pipe wins (the positional argument's help text states this explicitly).

Stack algebra. `-p/--pop` removes from the top, `-s/--shift` removes from the bottom, `-u/--unshift` pushes to the bottom, `-a/--add INDEX` inserts at a position, `-r/--remove INDEX` deletes a specific entry, `-M/--mv FROM TO` moves, `-S/--swap A B` exchanges, `-x/--dup INDEX` duplicates onto the top, `--rev` reverses the whole stack, and `--sort name|size|mtime` reorders it (bare `--sort`, via `default_missing_value("name")`, sorts by name). The reordering operations all funnel through one helper, `apply_permutation`, which rebuilds both the path vector and the name vector from an index list so that tags always follow their files:

```
pub(crate) fn apply_permutation<T: Default>(<
    vec: &mut Vec<T>, indices: &[usize]) {
    let mut taken = std::mem::take(vec);
    vec.extend(indices.iter()
        .map(|&i| std::mem::take(&mut taken[i])));
}
```

`sort_stack` computes the permutation once — comparing `file_name()`, cached file sizes, or cached mtimes depending on the key — then applies it to stack and names in lockstep. `move_tempfile` removes the source index and clamps the insertion point with `to_idx.min(len)` so an out-of-range destination lands at the end rather than panicking. `apply_permutation` is the one purely-functional helper on `TempApp`, and it is pinned by six unit tests in `app.rs` covering identity, reverse, element duplication (a repeated index leaves `T::default()` behind because of the `mem::take`), subset truncation, empty drain, and a two-element swap.

Inspection. `-l` lists entries, `-n` lists them numbered, `-L/-N` do the same with contents inlined, `-k/--count` prints the stack size, `-I/--info INDEX` prints per-entry metadata, `-d/--dir` prints the managed directory, `-m/--master` prints the master-record path, and `--path INDEX` prints the backing file path so scripts can hand it to another program (`cat "${tp --path 1}"`). The coreutils-analogue operations live here too. `--head INDEX N` and `--tail INDEX N` read the whole file into a `String` and then `.lines().take(n)` or slice the last `n` — not a streaming implementation, but correct for the scratch-sized files the tool is meant for. `--wc INDEX` prints a *line count only* (`content.lines().count()`), not the word/byte columns of real `wc`. `--size INDEX` prints raw bytes from `fs::metadata`. `-g/--grep PATTERN` does a literal substring match, prints each matching entry with per-line numbers, and exits 0 on any match and 1 otherwise so it composes in `&&` chains. `-D/--diff A B` shells out to `diff -u` and propagates its exit code (0 identical, 1 differ, 2 error). `-C/--cat` concatenates several entries in any given order (`num_args(1..)`).

Editing and lifecycle. `-e/--edit INDEX` opens an entry in `$EDITOR` (falling back to `vi`) and exits with the editor's code, `--replace INDEX PATTERN REPLACEMENT` does an in-place find-and-replace and prints the match count, `-c/--clear` purges everything, and `--expire HOURS` purges entries older than a fractional number of hours (`--expire 0.5` for thirty minutes) and prints how many it removed. Two of these carry hardening that reads like scar tissue from real bugs, documented in the source comments. `--expire` parses its argument as `f64` and explicitly rejects negative, NaN, and infinite values before the `as u64` cast — the comment records that a pre-fix `--expire=-1` saturated to a zero cutoff and silently wiped the entire stack. `--replace` rejects an empty pattern up front, because `str::replace("", repl)` splices the replacement at every byte boundary and would balloon the file to `0(len * repl.len())`.

The `-c/--clear` guard is the sharpest of the three. Clearing removes the parent directory of the master record with `remove_dir_all`, and a naive version combined with `TEMPRS_DIR=$HOME` would delete the home directory. The safe contract requires the directory's basename to contain the substring `temprs` (case-insensitively) before it will remove anything; anything else is refused outright:

```

let safe_name = parent.file_name()
  .and_then(|s| s.to_str())
  .map(|s| s.to_ascii_lowercase().contains("temprs"))
  .unwrap_or(false);
if !safe_name {
  util_terminate_error(ERR_NO_FILE);
  unreachable!()
}

```

Dual indexing by position or name

Every operation that takes an `INDEX` also takes a name. Entries can be tagged at creation with `-w/--name NAME` or renamed later with `-R/--rename OLD NEW`, and thereafter the tag is interchangeable with the number. The resolution rule is in `resolve_idx`: the argument is first parsed as a signed integer, and only if that fails is it looked up as a tag.

```

fn resolve_idx(&self, f: &str) -> Option<usize> {
  match f.parse::<i32>() {
    Ok(idx) => Some(util_transform_idx(
      idx, self.state.temp_file_stack().len())),
    Err(_) => self.state.temp_file_names().iter()
      .position(|n| n.as_deref() == Some(f)),
  }
}

```

`util_transform_idx` folds negative indices into the valid range — `-1` is the top of the stack, `-stack_size` is the bottom — and terminates with `ERR_INVALID_IDX` for anything out of bounds. Index `0` is always invalid by construction, since the stack is one-based: a positive `idx` maps to `idx - 1` and a zero would map to `-1`, which the guard rejects. Tag names must be unique across the stack and must not contain NUL bytes, which would break the master-record framing; both constraints are enforced with dedicated error constants (`ERR_INVALID_NAME`, `ERR_NAME_NUL`) at both `--name` set time and `--rename` time. Because a parse of `"1"` succeeds as an integer, a purely numeric tag name would be shadowed by index resolution — the ordering makes numbers win, which is the sensible default for a positional stack.

The terminal-output layer

One area the earlier chapter did not describe is how `temprs` decides what its output looks like. The `cyber_*` family in `utils.rs` renders one way to a terminal and another way to a pipe, keyed off a cached TTY check: `is_tty()` memoizes `io::stdout().is_terminal()` in a `thread_local! Cell<Option<bool>>` so the syscall is paid at most once per run. When stdout is a terminal, listings gain ANSI color and box glyphs — `cyber_print_content` frames a payload in a `┌ ... ┐` box, `cyber_hr` draws a cyan `███` rule, indexed listings print `[NN]` markers in yellow with `@tag` names in a second color. When stdout is a pipe, every one of those routines collapses to plain, byte-faithful text: `cyber_path` prints just the path, `cyber_hr` prints eighty dashes, `cyber_print_content` degrades to a bare `print!`. That split is what makes `tp -o 1 | jq` safe — the machine-consuming path never sees an escape code — while an interactive `tp -n` gets the decorated view.

The same aesthetic reaches the help output. `opts.rs` defines a `CYBERPUNK_TEMPLATE` help layout, a multi-line ASCII-art `BANNER` shown via clap's `before_help`, and an `AFTER` footer, all stitched from `concat!` with `env!("CARGO_PKG_VERSION")` interpolated so the version stripe in the banner never drifts from `Cargo.toml`. The version, author, and about strings all come from `env!("CARGO_PKG_*)` macros, so the CLI metadata is single-sourced from the manifest.

Here is the difference in practice — piped output stays clean and scriptable:

```

$ printf 'alpha\nbeta\ngamma\n' | tp -w greek
$ tp -k
1
$ tp --wc greek
3
$ tp --path greek
/tmp/temprs/tempfile1749145203871
$ tp -g beta
1:/tmp/temprs/tempfile1749145203871 @greek
2:beta
$ tp --expire 24
0

```

Integration into the shell and the stack

`temprs` ships a single zsh completion file, `completions/_temprs`, whose `#compdef tp temprs` line registers it for both binaries. The completion is not static: its helper `_tp_indices` reads the live `temprs-stack` master file out of `${TEMPRS_DIR:-${TMPDIR:-/tmp}/temprs}` and offers the current stack indices, file basenames, and `@`-prefixed tag names as completion candidates, so tab-completion reflects the actual contents of your stack at the moment you press Tab. It even solves a real encoding problem inline: zsh variables cannot hold NUL bytes, so the completion pipes the master record through `tr '\0' '\x1e'`, turning the `\0\0` record delimiter into `\x1e\x1e` and the `\0` field delimiter into `\x1e`, then splits on those. Tagged entries get two candidates — one keyed by index, one keyed by name — so either addressing form completes. This is the integration point that makes the tool feel native in an interactive shell rather than like a foreign command you have to remember indices for.

Two man pages are installed. `man/man1/temprs.1` is the conventional short reference — synopsis, indexing rules, options. `man/man1/temprsall.1` is a comprehensive single-document reference explicitly modeled on `zshall(1)`, folding every section into one page for readers who want to grep the whole manual at once. That choice is a deliberate nod to the zsh documentation tradition that the surrounding stack is built in.

In practice `temprs` slots between commands as a stack-aware pipe buffer:

```

$ ps aux | tp -w procs          # push, tagged
$ tp -g python procs && echo hit # grep the tagged entry
$ dmesg | tp                    # push another
$ tp -D procs -1                # diff the two by name/index
$ tp --expire 1                 # drop anything older than 1h
$ tp -c                         # wipe the stack

```

Status: version, tests, CI

The crate is at **version 2.9.12** (`Cargo.toml` and both man-page `.TH` lines agree). It is MIT-licensed and published on crates.io with a Homebrew tap install path (`brew install temprs`) and the `cargo install temprs` route.

The test surface is large and lives mostly beside the code. A static grep of `#[test]` attributes across `src/` yields **5,059** unit tests — **4,449** of them are the generated flag-recognition tests in `opts.rs`, with the remainder spread across `consts.rs` (87), `utils.rs` (174), `state/tests.rs` (181), `apply_permutation_tests.rs` (162), and the six `apply_permutation` tests in `app.rs`. The `tests/` directory adds **585** more `#[test]` functions across integration files (`integration.rs`, `edge_cases.rs`, `expire_invalid_hours.rs`, the `contract_*` invariant suites, `master_record_tag_roundtrip.rs`, `tag_alignment_after_mutation.rs`) plus **21** shell-based gate scripts that check documentation and hygiene — final newlines, README badges and section structure, man-page synopsis sections, executable bits and shebangs on the test scripts themselves, and the absence of deprecated HTML tags, inline handlers, `http://` links, and placeholder hrefs in the docs

site. These counts are from a static grep of the source, not from a `cargo test` run in this session, so treat the totals as provisional; the authoritative numbers come from the `test result: lines of cargo test --locked`. A `criterion` bench (`benches/benchmarks.rs`) microbenchmarks the hot free functions — `transform_idx`, `path_to_string`, `time_ms`, and the 100-record master-record read/write paths (`file_to_paths_100`, `paths_to_file_100`, `overwrite_file_100_lines`, `append_file`, `file_contents_to_string_100_lines`, `lines_to_file_100`).

The CI workflow (`.github/workflows/ci.yml`) runs a long chain of gates on every push and pull request to `main`: Check, Test (on an OS matrix, `--no-fail-fast`), Format (`cargo fmt --check`), Clippy (`-D warnings`), Doc, Release Build (target matrix), plus the Docs, Polish, Semantic, Newline + README, and Structure gate jobs that run the shell scripts. A separate `release.yml` cross-compiles per-target binaries (including musl via `cross`), packages them, publishes a GitHub Release, and auto-updates the Homebrew tap formula with freshly computed SHA-256 sums.

What `tempris` costs, plainly: it keeps state on disk that outlives your shell, so a forgotten stack accumulates files until `--clear` or `--expire` reclaims them; it holds a process-lifetime exclusive lock on its master record, so overlapping invocations serialize rather than run truly in parallel; and `--head/--tail/--grep/--replace` read whole files into memory rather than streaming, which is fine for scratch data and wrong for gigabyte logs. Those are deliberate trades in favor of the guarantee the tool actually sells — that the stack never corrupts and never tears, no matter how many shells reach for it at once.

iftoprs — iftop rebuilt on ratatui and pcap

What it is and what it ports

`iftoprs` is a Rust reimplementation of `iftop`, the per-connection bandwidth monitor that has shipped on Unix systems since the early 2000s. The upstream tool — originally written in C by Paul Warren and Chris Lightfoot, driven by `libpcap` and a `curses` front end — answers one question: *which hosts are eating my link right now, and at what rate*. It sniffs packets off an interface, aggregates them into bidirectional flows, and paints a live table of source/destination pairs with rolling 2-second, 10-second and 40-second averages plus a header bar summarizing total throughput. `iftoprs` keeps that model intact and rebuilds the machinery underneath it in Rust: `pcap` for capture, `ratatui` + `crossterm` for the terminal UI, `tokio` channels to move packets off the capture thread, and native OS syscalls for the one thing classic `iftop` never did — attributing each flow to the process that owns it.

The format is not the invention here. Flow tables, sliding-window rate columns, and the `src ↔ dst` layout are `iftop`'s. The engineering is the port: a single statically-linked binary that carries its own process-attribution backend (no shelling out to `lsof`), a capture pipeline that auto-restarts on interface flaps, a 40-slot ring-buffer accounting model, an NDJSON headless mode for pipelines, and a `ratatui` render layer with mouse support, sparklines, hover tooltips and a config that persists to TOML. The manifest describes it plainly as an “iftop clone in Rust with ratatui TUI” (`Cargo.toml:5`). One of the 31 bundled color themes is literally named `Iftopcolor` (`src/config/theme.rs:68`, `iftoprs.default.conf`), a deliberate parity palette for people who want the original look — the enum's own doc comment notes the palette set was “ported from storageshower” (`src/config/theme.rs:4`), another MenkeTechnologies tool, so the theme machinery is shared rather than reinvented per binary.

The crate is at version **2.22.9** (`Cargo.toml:3`), Rust **edition 2024**, and pins the **stable** toolchain with `rustfmt` and `clippy` via `rust-toolchain.toml`. The README states a minimum of Rust 1.85. macOS and Linux are the supported platforms; both need `libpcap` present, and raw capture needs root. The dependency list is deliberately small and durable: `anyhow`, `clap` + `clap_complete`, `crossterm`, `dirs`, `dns-lookup`, `pcap`, `ratatui` 0.30, `regex`, `serde/serde_json`, `tokio` (full features), `toml`, `chrono`, and — only on macOS — `libproc` for the kernel socket table (`Cargo.toml`, `[target.'cfg(target_os = "macos")'.dependencies]`). The release profile builds with `opt-level = 3`, `lto = true` and `strip = true` for a lean single binary; local development uses a plain `cargo build`.

Architecture: five modules behind a thin binary

`src/lib.rs` exposes the crate as five submodules — `capture`, `config`, `data`, `ui`, `util` — and `src/main.rs` is the binary that wires them together. The whole tree is roughly 23,000 lines of Rust across 20 files (via `wc -l` on `src/**/*.rs`; 23,040 total), and the bulk of it lives in three files: `src/ui/app.rs` (3,805 lines), `src/capture/parser.rs` (3,074 lines) and `src/ui/render.rs` (2,610 lines). The rest is distributed across `resolver.rs` (1,733), `flow.rs` (1,685), `theme.rs` (1,263), `main.rs` (1,136), `tracker.rs` (1,040), `prefs.rs` (966), `history.rs` (859), `format.rs` (673) and `procinfo.rs` (425), with the CLI definition in `cli.rs` (2,344).

capture — libpcap in, ParsedPacket out

`src/capture/sniffer.rs` owns the packet source. `start_capture` validates the requested device up front (so a typo'd interface fails immediately), then spawns a dedicated OS thread named `packet_capture` that opens the device with `snaplen(256)`, a 100 ms read timeout, and optional promiscuous mode, applies a BPF filter string if one was given, and pushes every decoded packet into a `tokio::sync::mpsc::UnboundedSender`. The snaplen of 256 bytes is deliberate: iftoprs only needs L2–L4 headers to build a flow key, never payload, so it caps the copy.

The capture loop is resilience-first. If the device fails to open or the read loop hits a non-timeout pcap error, it treats it as transient, sleeps with exponential backoff starting at 250 ms and capped at `MAX_BACKOFF = 30s`, and retries — resetting the backoff on any successful reopen. `TimeoutExpired` is not an error; it just continues. The only clean exit is the receiver being dropped, i.e. the app shutting down:

```
match cap.next_packet() {
  Ok(packet) => {
    let parsed = match datalink {
      pcap::Linktype::ETHERNET =>
        parser::parse_ethernet(packet.data, local_net),
      pcap::Linktype(0) =>
        parser::parse_loopback(packet.data, local_net),
      pcap::Linktype(113) =>
        parser::parse_sll(packet.data, local_net),
      pcap::Linktype(101) =>
        parser::parse_raw(packet.data, local_net),
      _ => parser::parse_raw(packet.data, local_net),
    };
    if let Some(p) = parsed
      && tx.send(PacketEvent { parsed: p }).is_err() {
      return LoopExit::ReceiverDropped;
    }
  }
  Err(pcap::Error::TimeoutExpired) => continue,
  Err(_) => return LoopExit::TransientError,
}
```

The datalink dispatch is real link-layer handling, not an Ethernet-only shortcut. It decodes Ethernet (`DLT_EN10MB`), BSD loopback (`DLT_NULL = Linktype(0)`), Linux cooked capture (`DLT_LINUX_SLL = Linktype(113)`, what you get on the `any` pseudo-device), and raw IP (`DLT_RAW = Linktype(101)`), with raw IP as the fallback for unknown link types.

`src/capture/parser.rs` turns bytes into a `ParsedPacket { key, direction, len }`. `parse_ethernet` reads the ethertype, dispatches `0x0800` to IPv4 and `0x86DD` to IPv6, and handles `0x8100` 802.1Q VLAN tags by skipping the 4-byte tag and re-reading the inner ethertype. IPv4 parsing reads IHL to find the L4 offset, takes `total_len` from the header as the byte count, maps the protocol number through `Protocol::from_ip_next_header` (`src/data/flow.rs:44`: `6` → TCP, `17` → UDP, `1/58` → ICMP, anything else → `Other(n)`), and extracts source/destination ports for TCP and UDP. The `Protocol` enum's `Display` renders unknown protocols as `Proto(n)` — so an ESP tunnel shows as `Proto(50)`, SCTP as `Proto(132)`, with those exact cases pinned by tests (`flow.rs:124`, `flow.rs:129`).

Direction is where the local-network filter earns its place. `determine_direction` compares src and dst against the local CIDR: if the source is inside the local net and the destination is not, the packet is `Sent`; the reverse is `Received`; if both or neither are local it defaults to `Sent`. The flow key is then run through `FlowKey::normalize` (`src/data/flow.rs:66`), which canonicalizes the (addr, port) pair ordering using a `u128` comparison so that `A → B` and `B → A` collapse to the same key, flipping the recorded direction when it swaps. The comparison lifts both IPv4 and IPv6 addresses into `u128` (`u32::from(v4)` widened, or the raw v6 bytes) and compares the `(addr, port)` tuples, so the ordering is total and allocation-free. That is how one logical connection stays one row regardless of which side a given packet traveled.

data — the accounting model

`src/data/history.rs` is the rate engine. Each flow gets a `FlowHistory` holding two `VecDeque<u64>` ring buffers — one for bytes sent, one for received — capped at `HISTORY_SLOTS = 40` one-second slots (`history.rs:5`), plus cumulative `total_sent/total_recv`, `peak_sent/peak_recv`, `last_seen`, and optional process name/PID. `add_sent/add_recv` fold bytes into the current (back) slot, bump the cumulative counter, and stamp `last_seen = Instant::now()`. `rotate()` is called once a second: it folds the just-completed slot into the peak trackers, pushes a fresh zero slot, and evicts the front once either deque exceeds 40 entries (`history.rs:92`) — a fixed-size sliding window that never grows unbounded regardless of connection lifetime.

The 2s/10s/40s figures come from `window_total`, which sums the trailing N slots:

```
fn window_total(slots: &VecDeque<u64>, n: usize) -> f64 {
    let len = slots.len();
    if len == 0 { return 0.0; }
    let take = n.min(len);
    slots.iter().rev().take(take).sum::<u64>() as f64
}
```

So `avg_sent_2s()` is the summed byte count over the last two one-second slots, `avg_sent_10s()` over ten, `avg_sent_40s()` over forty. Those windowed sums are what feed the display columns and the JSON output; `readable_size` then multiplies by 8 when rendering in bits (the default) or leaves the byte figure alone under `--bytes`.

`src/data/tracker.rs` is the shared state: a `FlowTracker` wrapping `Arc<Mutex<FlowTrackerInner>>` with a `HashMap<FlowKey, FlowHistory>`, global cumulative and peak counters, plus `current_sent/current_recv` accumulators for the in-progress second, and a `last_rotation` timestamp. `record` (`tracker.rs:104`) locks, looks up (or default-inserts) the flow, folds in the bytes, and updates both the per-flow history and the global counters. `maybe_rotate` (`tracker.rs:131`) fires only when at least one second has elapsed: it promotes `current_sent/current_recv` into the global peak if they exceed the running maximum, zeroes the accumulators, rotates every flow's history, and — critically — prunes. Flows whose `last_seen` is older than 60 seconds are `retain`-dropped (`tracker.rs:156`), so an idle connection ages out of the table instead of accumulating forever. `snapshot()` (`tracker.rs:161`) walks the map once, building a `Vec<FlowSnapshot>` — per-flow windowed rates, cumulative totals, process info, and a combined sent+recv per-slot history vector (`s + r` per slot) for the sparkline — plus a `TotalStats` header aggregate computed in a single pass over the same snapshots. The mutex is poison-tolerant everywhere — `lock().unwrap_or_else(|e| e.into_inner())` — so a panic in one thread can't wedge the tracker.

util — attribution, resolution, formatting

`src/util/procinfo.rs` is the part classic `iftop` doesn't have, and it does the attribution without a subprocess. The table it builds is a `HashMap<(u16, u8), (u32, String)>` keyed by `(local_port, protocol)` mapping to `(pid, name)` (`procinfo.rs:14`), stored in a `static PROC_CACHE: OnceLock<Arc<Mutex<ProcTable>>>`. On macOS, `refresh_proc_table_macos` enumerates every PID via `pids_by_type(ProcFilter::All)`, sizes each process's fd buffer from `pidinfo::<BSDInfo>().pbi_nfiles`, walks its fds with `listpidinfo::<ListFDs>`, and for each socket fd reads the local port and protocol straight out of the kernel via `pidfdinfo::<SocketFDInfo>` — selecting the correct arm of the `soi_proto` union by `soi_kind` (TCP vs. datagram), converting the network-order `insi_lport` with a local `ntohs`, and skipping anything that isn't TCP(6)/UDP(17) or has a zero local port (`procinfo.rs:47–124`). The process name is resolved lazily via `name(pid)` only once a socket is confirmed, falling back to `pid:<n>` if the name lookup fails. The `Cargo.toml` comment notes this compiled-in path replaces a runtime `lsof`. On Linux, `refresh_proc_table_linux` parses `/proc/net/tcp`, `/proc/net/tcp6`, `/proc/net/udp` and `/proc/net/udp6` for the inode → port map, then walks `/proc/[pid]/fd/` for inode → PID and reads `/proc/[pid]/comm` for the name (`procinfo.rs:127`). The table is refreshed on a 2-second cadence by the dedicated `proc-lookup` thread; `lookup_process(src, src_port, dst, dst_port, proto)` resolves a flow by trying the source port

first, then the destination port, against the port table (`src/util/mod.rs`), and returns `None` for ICMP or any non-TCP/UDP protocol (pinned by `procinfo.rs:270`).

`src/util/resolver.rs` handles async, cached reverse DNS via `dns-lookup`, wrapping a `HashMap<IpAddr, CacheEntry>` behind an `Arc<Mutex<>>` with a TTL-based `evict_stale` pass (`resolver.rs:120`). It also eagerly parses `/etc/services` on startup into a globally-cached `HashMap<(u16, &'static str), &'static str>` (`resolver.rs:44`, `resolver.rs:150`) for port→service-name mapping, normalizing the protocol string before lookup. `src/util/format.rs` provides `readable_size` (iftop-style `b/kb/Mb/Gb/Tb` in bit mode with a $\times 8$ conversion, `B/KB/MB/GB/TB` in byte mode), `readable_total` for cumulative counters (which always display in bytes regardless of bit/byte mode — `format.rs:26`), and `sparkline`, which maps a slice of values onto the block glyphs , rendering a literal space for a zero-valued slot rather than the lowest block (`format.rs:62`) so idle gaps read as gaps.

ui and config

`src/ui/app.rs` holds `AppState` — sort column, active view tab (`Flows / Processes`), CLI overrides, filter and selection state, and the several popup sub-states (theme chooser, theme editor, interface chooser) — and `src/ui/render.rs` draws it with `ratatui`. Two small enums drive the flow-row layout: `LineDisplay` cycles `Twoline` → `Oneline` → `SentOnly` → `RecvOnly` (`app.rs:63`), and `BarStyle` cycles `Gradient` → `Solid` → `Thin` → `Ascii` with a `name()` for the status line (`app.rs:88`); `Gradient` is the default. `src/config/theme.rs` defines the `ThemeName` enum with 31 variants (verified: `ThemeName::ALL.len() == 31`, `theme.rs:335`) and their RGB triples, plus a `CustomThemeColors` struct of six palette slots for user-authored themes. `src/config/prefs.rs` is the persistence layer: a TOML file at `~/.iftoprs.conf`, created on first run if missing (`load_prefs` writes the default when the file is absent — `prefs.rs:123`), auto-saved on every in-TUI toggle. The persisted fields include `theme`, `dns_resolution`, `port_resolution`, `show_ports`, `showBars`, `use_bytes`, `show_processes`, `show_cumulative`, `bar_style`, `pinned` (a `Vec<PinnedFlow>`), `show_border`, `show_header`, `hover_tooltips`, `refresh_rate`, an `alert_threshold` in bytes/sec, an optional `interface`, and the `custom_themes` map plus `active_custom_theme` name (`prefs.rs:10–65`). Every field carries a `#[serde(default)]` so an older config missing a key still loads — a forward-compat guard against schema drift. `src/config/cli.rs` is the clap `Args` definition plus the custom help renderer and completion generation.

The CLI surface

The binary front-loads its non-TUI modes in `main()` before ever touching the terminal (`src/main.rs:32–66`): `--help` prints a custom help screen, `--version` prints `iftoprs <version>`, `--completions <shell>` emits a completion script, `--list-colors` previews all 31 themes with swatches, `--list-interfaces` lists devices and exits, and `--json` runs the headless streaming mode. Everything else falls through to the TUI. The capture, display, output and system flags are:

```

# capture
-i, --interface NAME    interface to monitor
-f, --filter EXPR       BPF filter, e.g. "tcp port 80"
-F, --net-filter CIDR   IPv4 local network, e.g. "10.0.0.0/8"
-p, --promiscuous       promiscuous mode

# display
-n, --no-dns            disable reverse DNS
-N, --no-port-names     disable port→service names
-b, --no-bars           disable bar graphs
-B, --bytes             show bytes instead of bits
-P, --hide-ports        hide ports next to hosts
-Z, --no-processes      hide the process column

# output / system
--json                 stream NDJSON, no TUI
-c, --config FILE       config path (default ~/.iftoprs.conf)
-l, --list-interfaces   list interfaces and exit
--list-colors          preview all 31 themes
--completions SHELL    zsh | bash | fish | elvish | powershell

```

The five completion shells are real, but the source of the enum is worth stating precisely: `cli.rs` imports `clap_complete::Shell` (`cli.rs:5`) and wires it directly as the `--completions` argument type, rather than defining its own — so `Zsh`, `Bash`, `Fish`, `Elvish` and `PowerShell` are `clap_complete`'s built-in variants, round-tripped through the parser and pinned by the tests at `cli.rs:450–474` and `tests/contract_args_independence_round4.rs.generate_completions` (`cli.rs:175`) simply calls `clap_complete::generate` against the built `Args::command()`. A prebuilt zsh completion also ships checked-in at `completions/_iftoprs`, starting with the required `#compdef iftoprs` header.

Two clap details are set explicitly: `disable_help_flag` and `disable_version_flag` (`cli.rs:68`) are both on, because `-h/--help` and `-V/--version` are handled by hand as plain `bool` fields so the custom cyberpunk help screen (`print_cyberpunk_help`, `cli.rs:8`) renders instead of clap's default. The `-F` net filter is parsed by `parse_net_filter` (`cli.rs:181`), which splits on `/`, parses the address and prefix, and returns `None` (with a stderr warning) on anything that isn't valid CIDR — accepting both IPv4 and IPv6 forms, verified across a spread of cases including `2001:db8::/32`, `::1/128`, `0.0.0.0/0` and the malformed `10.0.0.0/24/8`.

Typical invocations (raw capture needs root):

```

sudo iftoprs -i en0          # monitor one interface
sudo iftoprs -f "tcp port 443" # HTTPS only, via BPF
sudo iftoprs -F 10.0.0.0/8 -B # scope local net, bytes
sudo iftoprs -Z             # hide the process column
iftoprs --completions zsh > _iftoprs
iftoprs --list-colors       # theme swatches, then exit

```

JSON mode for pipelines

`--json` (`run_json_mode` in `src/main.rs:762`) stands up the same capture and proc-lookup threads but skips the TUI entirely, emitting one JSON object per line (NDJSON) at the configured refresh interval (`prefs.refresh_rate` seconds). Each line is a `JsonSnapshot` with an RFC-3339 timestamp, `flow_count`, cumulative totals, peaks, and a `flows` array. Per-flow rate fields are multiplied by 8 into bits unless `--bytes` is set, `src/dst` are rendered as `host:port` (resolved through the DNS cache), and `process_name/pid` are omitted when unknown (`[serde(skip_serializing_if = "Option::is_none")]`, `main.rs:827`):

```

sudo iftoprs --json | \
jq -c '.flows[] | {dst, sent_2s, process_name}'

```

```
{
  "timestamp": "2026-07-13T10:22:04-04:00", "flow_count": 2,
  "total_sent": 184320, "total_recv": 998400, "peak_sent": 24576.0,
  "peak_recv": 131072.0, "flows": [
    {
      "src": "host.example.com:52344",
      "dst": "cdn.example.com:443", "src_port": 52344, "dst_port": 443,
      "protocol": "TCP", "sent_2s": 98304.0, "recv_2s": 786432.0,
      "sent_10s": 410000.0, "recv_10s": 3100000.0, "sent_40s": 1200000.0,
      "recv_40s": 9800000.0, "total_sent": 184320, "total_recv": 998400,
      "process_name": "curl", "pid": 40122
    }
  ]
}
```

That makes the same accounting core usable as a data source for logging, dashboards, or ad-hoc `jq` slicing, not just an interactive view.

The interactive loop

The TUI runs at roughly 30 fps: a 33 ms tick (`run_app`, `main.rs:174`) drains the packet channel into the tracker with `try_recv`, calls `maybe_rotate`, refreshes the display snapshot only at the user's configured refresh rate, redraws with `ratatui`, and polls `crossterm` for input within the remaining tick budget. The snapshot refresh is decoupled from the render tick — the terminal repaints every ~33 ms, but `update_snapshot` only pulls a fresh flow table every `refresh_rate` seconds, so a 10-second refresh still gives a smooth, responsive UI while the numbers hold steady between updates.

Mouse capture is enabled (`main.rs:124`), and `handle_mouse` (`main.rs:652`) routes clicks: left-click selects a flow row, right-click on a row raises a tooltip with TX/RX rates, totals, process and a 40-second sparkline, right-click on a header segment raises a persistent segment tooltip, middle-click pins a flow, scroll navigates, and any mouse-move dismisses the flow tooltip. The row-to-index math accounts for the extra header row in the process view (`main.rs:699`).

Keyboard controls mirror iftop where they overlap and add a layer beyond it. The core set: `1/2/3` sort by the 2s/10s/40s columns, `</>` sort by source/destination name, `r` reverses the sort, `o` freezes the current order (so rows stop reshuffling under you), `t` cycles the two-line/one-line/sent-only/recv-only display, `b` cycles the bar style, `n/N` toggle DNS and service-name resolution, `B` toggles bits/bytes, `p` toggles ports, `U` toggles cumulative-total columns, `Z` toggles the process column, `x` toggles the border, `g` toggles the header bar, `T` toggles hover tooltips, `f` cycles the refresh rate, and `P` pauses. Navigation is vi-style: `j/k` (or arrows) move the selection, `G/End` and `Home` jump to the extremes, and `Ctrl-D/Ctrl-U` page down/up — all dispatched per active view tab. Beyond iftop it adds: `Tab` switches between the flow view and a per-process aggregated view, `Enter` drills into a process's flows (in the process tab), `/` opens a live filter, `0` clears it, `c` opens the theme chooser, `C` opens the theme editor, `i` opens the interface chooser, `F` pins a flow to the top, `y` copies the selected flow, `e` exports all flows to `~/.iftoprs.export.txt`, and `h/?` toggle the help HUD. Every display toggle calls `save_prefs()` immediately, so the config file tracks the live UI without an explicit “save.”

Modal sub-states

Four of those keys open modal input states that intercept keys before the main dispatch (`main.rs:225–427`). The **filter** (`/`) is a full mini-readline: character insert, `Backspace`, cursor `Left/Right/Home/End`, and the emacs bindings `Ctrl-W` (delete word), `Ctrl-A/Ctrl-E` (line start/end), `Ctrl-K` (kill to end) and `Ctrl-U` (clear), with a live preview that updates `screen_filter` on every keystroke; `Enter` commits, `Esc` restores the previous filter. The **theme chooser** (`c`) navigates with `j/k` or the arrows (and scroll wheel), applies each theme live as you move, and persists on `Enter`. The **theme editor** (`C`) is a six-slot RGB palette editor: `j/k` move between slots, `h/l` nudge the selected color index by 1 and `H/L` by 10 (wrapping), applying the palette live; `s/Enter` enters a naming sub-mode that writes the result into `custom_themes` under a user-supplied name (capped at 20 chars). The **interface chooser** (`i`) lists devices from `list_interfaces` and, on `Enter`, records the selection to config with a “restart to apply” status note — capture is bound at launch, so the interface change is persisted rather than hot-swapped.

Threshold alerts

The `alert_threshold` preference (bytes/sec, default 0 = off, config-file only) drives `check_alerts` (`app.rs:1107`). On each snapshot it sums each flow's 2-second sent+recv rate; any flow at or above the threshold that wasn't already alerting produces a new alert. When new alerts appear it sets `alert_state.flash = Some(Instant::now())` (a timed border flash handled by the renderer), writes an  ALERT status line naming the source and rate, and emits a terminal bell (`print!("{}", '\x07')`, `app.rs:1135`). The alerting set is tracked across ticks so a sustained flow flashes once rather than continuously.

The process view and export

`Tab` switches to a per-process aggregate built by `aggregate_processes` (`app.rs:1612`): flows are grouped by `process_name` (unattributed flows collapse into a single `(unknown)` bucket), their 2s/10s/40s rates and cumulative totals summed, flow counts tallied, and the result sorted descending by 2-second combined rate. `Enter` on a process drills back into the flow view filtered to that process; `Esc` clears the drill-down. `e` writes a formatted plaintext report to `~/.iftoprs.export.txt` — a timestamped header, a fixed-width `SOURCE <=> DESTINATION / PROTO / TOTAL / 2s / 10s / 40s` table with labels truncated to 40 columns, and TX/RX summary lines carrying cumulative, peak, and the three windowed rates (`app.rs:1442`). `y` copies the selected flow's line to the clipboard.

Where it sits in the stack

`iftoprs` is one of the MenkeTechnologies `*rs` Rust reimplementations of Unix networking and system tools; the README cross-links it with `lsotrs`, `zshrs` and `strykelang`. It installs from crates.io (`cargo install iftoprs`) or the Homebrew tap (`brew install MenkeTechnologies/menketech/iftoprs`). Two man pages ship: `man/man1/iftoprs.1` (short reference, with `NAME/SYNOPSIS/DESCRIPTION/OPTIONS/KEYBINDINGS/EXAMPLES/ENVIRONMENT/FILES/EXIT STATUS` sections) and `man/man1/iftoprsall.1` (the full reference). Note that `iftoprsall` is a second *man* page, not a second binary; the crate builds one executable, `iftoprs`. A checked-in `iftoprs.default.conf` documents every TOML key and enumerates all 31 theme names as reference for hand-editing the config.

Parity and status

Against upstream iftop, the shared surface is the flow table, the three rolling windows, the src ↔ dst bidirectional accounting, BPF filtering, promiscuous mode, bits-vs-bytes display, and the sort/display keybinds. Beyond iftop it adds native per-flow process attribution with no external tools, a per-process aggregate view with drill-down, 31 themes plus a live RGB theme editor and user-saved custom palettes, mouse and hover tooltips, sparklines, four bar styles, four line-display modes, TOML config that auto-persists, threshold alerts (border flash plus terminal bell), an NDJSON headless mode, and an auto-restarting capture pipeline with real handling for Ethernet, loopback, Linux cooked and raw-IP link types.

Two accounting details are worth stating precisely rather than glossing as “identical to iftop.” First, the 2s/10s/40s columns hold the *summed* byte count over the trailing N one-second slots (`window_total`), rendered through `readable_size` with a ×8 bit conversion — the value is a windowed total scaled by the display, not a slot-averaged bytes/sec. Second, direction assignment leans on the local CIDR; when neither endpoint is inside the configured local net (or no net filter is active and none is auto-detected via `auto_detect_local_net`, which derives the prefix from the interface netmask and defaults to `/24`), packets default to `Sent`, which affects the TX/RX split on transit or fully-remote traffic.

CI runs on every push and PR across Ubuntu and macOS runners (libpcap-dev installed via apt on Linux) and gates on `cargo fmt --check`, `cargo clippy -D warnings`, `cargo doc` with `RUSTDOCFLAGS=-D warnings`, `cargo build/cargo test`, and a set of shell scripts under `tests/` that enforce docs/man/README/newline hygiene (final-newline checks, man-page synopsis presence, README

badge/section/https-link checks, workflow no-tabs, and a battery of docs HTML lints). Integration tests execute the built binary through `CARGO_BIN_EXE_iftoprs` rather than `cargo run`, so CLI output is read from the real process. The source carries a large embedded test suite: a `grep` for `#[test]` returns 2,275 annotations (1,730 inline in `src/`, 545 under `tests/`) — a provisional count of annotations, not a guarantee of passing runs, since it was measured by pattern match rather than a `cargo test` execution in this session.

Design commentary

Two decisions define the shape of the port. The first is the 40-slot ring buffer with 60-second idle eviction: it fixes the memory ceiling at “number of active flows × 40 u64 slots × 2 directions” and refuses to let a long-lived session’s history grow, while the eviction pass keeps the visible table bounded to genuinely recent traffic. The second is compiling the process-attribution backend into the binary rather than shelling out. Classic `iftop` has no process column at all; tools that add one typically parse `lsof` or `ss` output at runtime. `iftoprs` reads the kernel socket table directly — `libproc` syscalls on macOS, `/proc` on Linux — on a 2-second background cadence, keyed by `(local_port, protocol)`, so attribution is a hash lookup against a pre-warmed table rather than a subprocess spawn per refresh. The tradeoff is honest: attribution only resolves TCP and UDP flows, only where the local port is visible in the socket table, and it inherits the privilege limits of inspecting other processes’ fds — which is why some flows show no process even under root. Within those bounds it is a single self-contained binary that does what `iftop` did and attributes it too.

htoprs — the interactive process viewer, ported function-for-function into Rust

`htoprs` is a faithful Rust port of `htop`, the cross-platform, ncurses-based interactive process viewer that most people reach for when `top` is not enough. The tool it descends from is not new — `htop` has existed since 2004 and is standard equipment on virtually every Linux and BSD box. Nothing about the *concept* of a scrollable, colored, tree-capable process table is invented here. What is engineered here is the port itself: `htop`'s C source treated as an executable specification and translated into Rust module by module, function by function, with each translated `fn` carrying a citation back to the `<File>.c:<line>` it came from. The value proposition is not “a better process viewer” — it is “the same process viewer, byte-for-byte, rebuilt in a memory-safe, vendorable, cross-architecture Rust codebase with a mechanically-enforced parity discipline.” The upstream is `htop 3.5.1`, pinned in-repo as the `vendor/htop` submodule, and that pin is the spec of record.

The package is `htoprs` version **0.5.7** (Cargo.toml), edition 2021, licensed **MIT** — original Rust code that reimplements `htop`'s behaviour rather than deriving from its source. It builds one library (`htoprs`) and one binary (`htoprs`). Attribution is MenkeTechnologies.

What a “faithful port” means mechanically

The distinguishing engineering decision in this codebase is that faithfulness is not an aspiration in the README — it is a build gate. The source is split into two trees with different rules:

- `src/ported/<file>.rs` — one Rust module per `htop` C file. Every free `fn` here is supposed to be a translation of a specific C function, and each carries a `/// Port of` doc comment naming its origin. There are **1,930** such `/// Port of` citations across the ported tree (grep of `src/ported`), spread over **141** Rust modules.
- `src/extensions/<name>.rs` — `htoprs`-original code that is *not* a translation of anything in `htop`, and is therefore explicitly exempt from the port rules.

The gate lives in `build.rs` and runs on every `cargo build`, `cargo test`, and `cargo check` that touches `src/ported/`. It extracts every free `fn` name under `src/ported/` and checks it against a snapshot of `htop`'s actual C function names at `tests/data/htop_c_fn_names.txt` — a flat index of `<File>.c:<name>` pairs, **4,132** entries as checked in. A function whose name has no counterpart in that index *fails the build*, printing each violation as a `cargo:warning` line and then a `panic!` with instructions (inline the body, rename to the real C function, or — last resort — justify it in the allowlist). Rust-original helpers are rejected outright; genuine architectural helpers that have no C analog must be listed explicitly in `tests/data/fake_fn_allowlist.txt`, which at present holds **zero** entries — nothing in the ported tree needed the escape hatch.

The gate is more careful than a regex sweep would be. `collect_free_fns` in `build.rs` is a byte-level brace-counting state machine that tracks nesting depth so it can distinguish module-level free functions (depth 0) from methods inside `impl/trait` blocks (which it skips), and it enters and leaves `mod tests { / mod test {` blocks so unit-test helpers do not trip the check. Crucially it ignores braces that appear inside line comments, block comments (with nesting), string literals, raw strings (`r#"..."#` with balanced hash counts), and char literals — without that, a test that merely contains `"{"` in a string would corrupt the depth counter and misfire the gate after any harmless reordering. It strips `pub`, `pub(crate)`, `pub(super)`, `unsafe`, `async`,

and `extern "C"` prefixes before reading the name, so the gate sees the same identifier regardless of how the port declares visibility or ABI. If the ported root or the C-name snapshot is missing entirely, the build script returns silently rather than failing an unrelated workspace.

The C-name snapshot is regenerated by `tests/data/extract_c_fn_names.sh` after pulling a new upstream htop (it honors `$HTOP_C_SOURCE`, default `~/forkedRepos/htop`), so the gate tracks the real upstream rather than a hand-maintained list that drifts. Coverage is reported by `scripts/gen_port_report.py`, which walks both the C source and the Rust port and writes `docs/port_report.html` with per-file and overall numbers derived from source at run time — nothing hardcoded. Critically, a `todo!()` or `unimplemented!()` body is counted as **stubbed**, never **ported**, so scaffolding cannot inflate the coverage figure. The stub scaffold generator (`scripts/gen_port_stubs.py`) lays out the full port surface — one module per C file, one `todo!()` stub per C function named to satisfy the gate — and never overwrites an already-ported module.

Current port state

Per the README's "Current state" section and the generated `docs/report.html`, the port covers **1,069 of 1,093 C functions (97.8%)** across **130 of the 131 C files**, with **1 stub remaining**, and the TUI runs as a daily driver on macOS. (A raw grep for `todo!()/unimplemented!()` under `src/ported` returns 57 tokens; most sit in the intentionally-unported allocation families and the feature-gated BSD/PCP backends, which the report classifies as out-of-scope rather than as in-scope stubs — that interpretation is provisional, the authoritative breakdown is the generated report.) The ported surface is end-to-end: the process model and table build (`Process`, `ProcessTable`, `Table`, `Row`, `Machine`), the container and utility layer (`Vector`, `Hashtable`, `RichString`, `XUtils`), the full meter set, the UI panels and main loop, key dispatch, the `CRT` terminal layer, and the per-OS machine/process-table backends.

Both man pages still lag the code. `man/man1/htopr.1` carries a **PORT STATUS** note claiming "the port is at an early stage ... the binary does not yet present an interactive interface," and `man/man1/htoprall.1` similarly says only the `XUtils.c` layer is ported; those texts predate the 97.8% figure and should be treated as stale relative to the README's current-state numbers and the live `docs/port_report.html`.

Some C is intentionally *not* ported because it has no faithful safe-Rust analog. The `xMalloc` allocation family, the raw `Object**` dynamic-array memory machinery in `Vector.c` (`Vector_new/_insert/_add/_resizeIfNecessary`), the open-addressing bucket table in `Hashtable.c` (`Hashtable_new/_put/_get/_foreach`), the varargs formatters, and `String_freeArray` all manage allocation, bounds, probing, and lifetimes by hand — work that Rust's `Vec` and `HashMap` already own. Porting those byte-for-byte would mean reimplementing an allocator in `unsafe`, which is precisely the class of bug the port exists to eliminate. `combSort` in `Vector.c` is commented-out dead code and is not ported. These omissions are honest and documented, not coverage gaps papered over.

The dependency surface

The runtime crate graph is deliberately small and foundational, chosen to keep the build vendorable and durable. From `Cargo.toml`: `crossterm` 0.29.0 (the terminal layer), `libc` 0.2.186 (the real `getopt_long`, `sysctl`, and platform syscall surface), `nix` 0.31.3 (typed syscall wrappers), `unicode-width` 0.2.2 (column-width math for the styled string buffer, matching htop's `wcwidth` handling), and `serde` 1.0.228 with `serde_json` 1.0 for persisting preferences. `regex` 1 backs the saved-filters extension. `ratatui` 0.30.0 is pulled with `default-features = false` and only the `std` feature so it drags in *no* terminal backend — the themed help overlay, chooser, and editor render into a `ratatui::Buffer` while htoprs still drives the actual terminal through its own `crossterm`, avoiding a second, conflicting `crossterm` in the tree.

The Linux target adds three pure-Rust, no-FFI helper crates behind `cfg(target_os = "linux")`: `libmedium` 0.13.4 for hwmon sensors, `neli` 0.6 (a generic-netlink client) for the delay-accounting path that queries the kernel TASKSTATS family (`linux/LibNL.c`), and `zbus` 5 (a pure-Rust D-Bus client) for the `SystemdMeter` port (`linux/SystemdMeter.c updateViaLib`). Both `neli` and `zbus` speak their respective

wire protocols directly over a socket rather than linking a C library — a deliberate choice to keep the build free of C dependencies and vendorable long-term. `zbus`'s blocking module mirrors `htop`'s synchronous sd-bus property reads.

Module map

The `src/ported/` tree is a near-mirror of `htop`'s file layout. The generic model and UI substrate includes `process.rs`, `processtable.rs`, `table.rs`, `row.rs`, `machine.rs`, `object.rs` (`htop`'s `vtable` OOP rendered as a Rust `Object` trait with a class-chain `Object_isA`), `richstring.rs`, `vector.rs`, `hashtable.rs`, and `xutils.rs`, alongside the settings and support modules `settings.rs`, `history.rs`, `scheduling.rs`, `affinity.rs`, `userstable.rs`, `listitem.rs`, `optionitem.rs`, and `htop.rs` (the ported `main`). The `ProcessField` enum in `process.rs` transcribes `htop`'s `RowField.h` reserved-field discriminants verbatim (`PERCENT_CPU = 47`, `PERCENT_MEM = 48`, and so on), because those numeric IDs are load-bearing for column identity and must match.

The UI layer is `panel.rs`, `screenmanager.rs`, `mainpanel.rs`, `functionbar.rs`, `header.rs`, `crt.rs`, plus the setup and dialog screens: `categoriespanel.rs`, `columnspanel.rs`, `availablecolumnspanel.rs`, `colorspanel.rs`, `displayoptionspanel.rs`, `headeroptionspanel.rs`, `meterspanel.rs`, `availablemeterspanel.rs`, `screenspanel.rs`, `screentabspanel.rs`, `signalspanel.rs`, and `affinitypanel.rs`. Interaction dispatch is `action.rs`, `incset.rs` (incremental search/filter), and `lineeditor.rs`. Auxiliary screens include `envscreen.rs`, `openfilesscreen.rs`, `processlocksscreen.rs`, `commandscreen.rs`, `tracescreen.rs`, `backtracescreen.rs`, `dynamicsscreen.rs`, and `infoscreen.rs`.

The meter set is fully represented: `cpumeter.rs`, `memorymeter.rs`, `swapmeter.rs`, `memoryswapmeter.rs`, `loadaveragemeter.rs`, `batterymeter.rs`, `networkiometer.rs`, `diskiometer.rs`, `gpumeter.rs`, `zfsarcmeter.rs`, `zfscompressedarcmeter.rs`, `tasksmeter.rs`, `uptimemeter.rs`, `datetimemeter.rs`, `hostnamemeter.rs`, `sysarchmeter.rs`, `filedescriptormeter.rs`, and the `dynamicmeter.rs` / `dynamiccolumn.rs` machinery for platform-defined meters. `meter.rs` holds the shared value-formatting core, including `Meter_humanUnit` (kibibytes → human-readable `K...Q` with an `inf` cap).

Platform backends live in per-OS subdirectories: `darwin/`, `linux/`, `freebsd/`, `openbsd/`, `netbsd/`, `dragonflybsd/`, `solaris/`, plus `generic/`, `pcp/`, and `unsupported/`. The Darwin backend is `darwinmachine.rs`, `darwinprocess.rs`, `darwinprocesstable.rs`, `platform.rs`, and `platformhelpers.rs`. The Linux backend is the largest — sixteen modules including `linuxmachine.rs`, `linuxprocess.rs`, `linuxprocesstable.rs`, `platform.rs`, `cgrouputils.rs`, `libnl.rs`, `libsensors.rs`, `gpu.rs`, `ioprioritypanel.rs`, and the platform meters `systemdmeter.rs`, `pressureallmeter.rs`, `selinuxmeter.rs`, `zrammeter.rs`, `openrcmeter.rs`, and `hugepagemeter.rs`. The `generic/` directory holds portable fallbacks and the FFI-gated leaves — `demangle.rs`, `unwindptrace.rs`, `openzfs_sysctl.rs`, `fdstat_sysctl.rs`, `gettime.rs`, and `uname.rs`. On Linux the port reaches the kernel through the pure-Rust `neli/zbus/libmedium` crates named above rather than through C libraries.

Terminal backend

`htop` renders through `ncurses`. `htopr`s does not — the terminal layer is `crossterm` 0.29, pure-Rust, cross-architecture, no C dependency. This is a *behavioral* port rather than a line-for-line one: the constraint is that output must be byte-for-byte identical to `htop`, not that the draw code mirror `ncurses` call-for-call. To meet that constraint the color substrate underneath the draw code is ported faithfully — `CRT.c`'s `ColorElements` enum and every `CRT_colorSchemes` entry are transcribed verbatim so that `htopr`'s colors match `htop`'s exactly, glyph for glyph and attribute for attribute. The optional feature-gated `ncurses`-adjacent build variants `htop` offers via `./configure` are mirrored as Cargo features: `pcp` (Performance Co-Pilot backend), `demangle` (C++/Rust symbol demangling for the backtrace screen), `unwind` (libunwind-ptrace backtraces), `capabilities` (libcap capability dropping), and `hwloc` (topology-aware affinity picker). All are off by default; each declares its foreign

FFI surface by hand in `extern` blocks and is verified by primary-source reading plus the port gate rather than by linking on every platform (several of those libraries do not link on macOS). The `hwloc` feature carries an explicit, documented substrate gap: `AffinityPanel_new`'s topology-tree build reads a `host->topology` field the ported `Machine` omits (`machine.rs:46`), so until that field is wired the `topoRoot/allCpuset` values stay their null/None defaults — the Cargo.toml feature comment states this outright rather than hiding it.

The clippy-allow ledger

`src/lib.rs` carries sixteen crate-wide `#![allow(clippy::...)]` attributes, each with a doc paragraph tying it to a faithful-port reason, because CI runs `cargo clippy --all-targets -- -D warnings`. These are not blanket lint suppression — every one names the C shape it preserves. `needless_range_loop` and `explicit_counter_loop` keep the dual-index `for (i, j)` walks that mirror the C pointer arithmetic in `strncpy` and `RichString` fills. `manual_div_ceil` keeps `(cpus + 1) / 2`, the C half-split formula. `neg_cmp_op_on_partial_ord` keeps `!(rate >= 0.0)`, a deliberate NaN guard from `Row.c` that `rate < 0.0` would silently break. `manual_c_str_literals` keeps `b"kern.osrelease\0"` as the literal constant passed to `sysctlbyname`, since `c"..."` would hide the NUL the C source spells out. `field_reassign_with_default` keeps `htop's *_init` routines setting fields one at a time on a freshly-`calloc`'d struct rather than collapsing into a Rust struct literal. The full list is the clearest single artifact of the port's discipline: where idiomatic Rust and the C spec disagree, the spec wins and the lint is documented, not the other way around.

CLI surface

The command-line contract mirrors `htop's getopt_long` table exactly, driven through the real `libc::getopt_long` symbol so parsing behavior matches. `parseArguments` in `src/ported/commandline.rs` marshals `argv` into a null-terminated C `char**`, builds the same `option` array `htop` builds (with `has_arg` 0/1/2 for none/required/optional), and calls `libc::getopt_long` in the same loop. The option spec is:

<code>-h, --help</code>	Print the help screen and exit
<code>-V, --version</code>	Print version info and exit
<code>-d, --delay=DELAY</code>	Update delay, tenths of a second (1..100)
<code>-n, --max-iterations=NUMBER</code>	Exit after NUMBER refresh iterations
<code>-s, --sort-key=COLUMN</code>	Sort by COLUMN (--sort-key help lists)
<code>-u, --user[=USERNAME UID]</code>	Only this user's processes (self if none)
<code>-C, --no-color, --no-colour</code>	Monochrome mode
<code>-M, --no-mouse</code>	Disable mouse control
<code>-U, --no-unicode</code>	ASCII graph meters (not Unicode braille)
<code>-t, --tree</code>	Tree view
<code>-p, --pid=PID,PID...</code>	Show only the given PIDs
<code>-F, --filter=FILTER</code>	Fixed-string filter (" " multi-term)
<code>-H, --highlight-changes[=DELAY]</code>	Highlight new/old processes
<code>--no-meters</code>	Hide graph meters
<code>--no-function-bar</code>	Hide the function bar (--no-functionbar)
<code>--readonly</code>	Disable process/system-changing features

Two flags are intentional divergences: `htoprs -h` and `htoprs -V` route to branded help/version screens before delegating to the shared ported startup path. `src/main.rs` is a thin wrapper that scans `argv` for `-h/--help` and `-V/--version` and short-circuits — `-V` calls the ported `printVersionFlag` (which prints `htoprs 0.5.7`), `-h` calls `extensions::help::print_help` (the figlet-bannered, sectioned screen in the MenkeTechnologies house style). Everything else calls `CommandLine_run`, the ported entry that does the `getopt` parse, assembles the runtime object graph, and drives `ScreenManager_run` — the same path `htop's main` follows. The plain ported `printHelpFlag` is retained verbatim as the spec the styled screen tracks; it emits the version line, the (C) MenkeTechnologies 2026. copyright, the MIT licence notice, and the same aligned option list `htop` prints. There are two `CommandLine_run` definitions in `commandline.rs`, selected by

`cfg`, so the non-interactive/test path and the live-TUI path share one signature.

The zsh completion (`completions/_htoprs`, `#compdef htoprs`) keeps its option set in lockstep with that `getopt` table, and it completes sort columns *dynamically* — the `sort-keys` state calls `_call_program sort-keys $words[1] --sort-key help` and parses the tool's own live column list rather than hardcoding one:

```
case $state in
  sort-keys)
    tmp=( ${f}"${_call_program sort-keys \
      $words[1] --sort-key help}" )
    tmp=( ${tmp/#[:space:]}## )
    tmp=( ${tmp//:/\\:} )
    _describe -t sort-keys 'column (key)' tmp 66 ret=0
  ;;
esac
```

The extensions layer

On top of the port sits `src/extensions/` — **21 modules** (22 files including `mod.rs`) that are `htoprs`-original and gate-exempt; this has grown past the “20 modules” the README still cites, because the `forecast` module below was added. This is where `htoprs` adds capability `htop` does not have, carefully bound to keys `htop` leaves free so that no upstream binding is shadowed. The theming system (`extensions::theme`, `colors`, `prefs`, `overlay`) provides 31 built-in 6-color palettes plus custom-theme plumbing, remapping the live `htop` UI in 256-color at the single `Ncurses::to_color` choke point and persisting the choice to `~/.config/htoprs/prefs.json`. Overlay keys: `z` opens the theme chooser, `~` the theme editor, `h`/`?`/`F1` a themed help overlay, `g` toggles the header, `B` toggles the border, and `b` cycles the bar fill-glyph style (Classic, Gradient, Solid, Thin, Ascii) through the ported `BarMeterMode_draw` fill loop.

The live monitoring suite is driven by a single thread-local `PanelState` in `extensions::panels`, fed the real ported process table each sample tick. That struct holds the persistent engines (`ring: ProcRing` per-PID history, `alerts: AlertEngine`, `filters: FilterStore`, `cpu_hist`, and an optional `baseline` snapshot), the latest sampled frame (`table`, the `firing` PID set, `selected_pid`, `cpu_peak`, `tick`), the forecast fields, and the UI state (a `Modal` enum with ten variants and a `SparkMode` with three). It takes first refusal on keystrokes — yielding to the theme overlay if that owns the frame — and dispatches on raw key bytes chosen from keys `htop` leaves unbound:

```
f  Fuzzy process finder – Enter jumps the cursor to the match
r  Regex/substring filter over comm/cmdline/user, saved named
   store (~/.config/htoprs/filters.json)
d  Snapshot: first press = baseline, next = diff
   (+started -exited ~changed); w writes the snapshot JSON
o  Export the current table to JSON + CSV under ~/.config/htoprs/
A  Threshold alerts – the rule set plus every firing PID
E  Forecast – per-PID memory-exhaustion ETA (see below)
G  Braille CPU history graph (system total plus selected PID)
y  Aggregate/pivot – CPU+memory totals grouped by
   user/command/parent (Tab cycles)
:  Command palette – fuzzy-search every action by name, run it
v  Per-PID CPU sparkline: off → right-edge column → inline graph
```

The `:` palette is backed by a `CMD5` registry that pairs each action's searchable name with the key byte it injects into the run loop, so a palette pick flows through the *whole* dispatch pipeline — the palette can reach both extension actions and `htop`'s own keys (`k` kill, `t` tree, `/` search, `S` setup) with one mechanism. Two of these reach the rows themselves rather than a modal, injected at the per-row draw site in `Panel_draw` (the same extension-hook pattern the theme border uses, so no new ported `fn` is introduced): a firing-alert PID's row is recolored, and the `v` sparkline is overdrawn per row. The inline `v` graph mode (`SparkMode::Double`) makes

the panel variable-height — each process grows a full-width braille CPU graph whose line count scales with its CPU usage, so `PanelDraw/PanelOnKey` project every screen-Y, page step, and scroll clamp through per-row heights (`item_height = 1 + graph_lines(cpu)` for a process, `1` otherwise). Non-process panels keep `rowHeight = 1` and stay byte-identical to ported behavior.

Forecast: the memory-exhaustion horizon

The newest extension, `extensions::forecast`, answers a question htop cannot: not what memory a process holds *now*, but *when* that becomes a problem. Bound to `E`, it opens a modal listing every rising PID sorted by soonest ETA to exhaustion. Each refresh it fits an ordinary least-squares trend on that PID's bounded memory ring and projects the wall-clock time until its resident set crosses its own real ceiling.

The math is split so it is testable without an OS underneath it. `Trend::fit` is a pure hand-rolled closed-form slope (three sums, no clock, no I/O) over the sample indices of a PID's KiB series — positive slope means growing, and it returns `None` for fewer than two samples. `Ceiling::detect` reads the enforced limit *once* and caches it, because the enforced limit does not move during a session, and `select_ceiling` is a separate pure function that picks it in precedence order: a Linux cgroup-v2 `memory.max` if present, else the process address-space `RLIMIT_AS`, else total machine RAM. The comment is explicit that a stats crate was declined for the fit — the closed form is three sums and the durability rule prefers no new dependency. `PanelState` measures `sample_secs` live from the gap between ticks so the ETA tracks the real refresh rate (and pauses when updates pause), keeps every rising PID's `Projection` sorted by ETA, and maintains a `forecast_soon` set of PIDs whose ETA is at or under a `forecast_horizon` threshold — that set drives an optional inline row tint, toggled from within the modal, while the projections keep computing whether or not the tint is on.

Man pages, completion, and documentation

htoprs ships two man pages. `htoprs.1` is a short quick-reference of the CLI contract. `htoprsall.1` is a comprehensive all-in-one reference modeled on `zshall(1)` — eighteen numbered sections concatenated into one greppable page covering invocation, every command-line option, the process-list columns, the header meters and their Bar/Text/Graph/LED modes, tree view, search/filter/tagging, sorting, every interactive keybinding, the function bar, the setup screen, color schemes, mouse support, signals, configuration files, environment, exit status, worked examples, and porting notes. It cross-references the rest of the fleet in SEE ALSO (`iftoprsall`, `lsofrsall`, `nmaprsall`, `zshrsall`). Both pages, as noted above, still carry the stale early-stage porting caveat and should be resynced to the 97.8% figure.

Verification, parity, and crash handling

There are **1,411** `#[test]` functions across `src/` and `tests/` (grep count) — hand-crafted unit tests pinning the C edge-case behavior each ported function must reproduce, rather than generic coverage padding. Beyond unit tests, a dedicated **parity suite** at `tests/parity/` (registered as the single `parity` cargo test target, since Cargo does not auto-discover files under that subdirectory) compares htoprs against the genuine htop in two ways. `cli_parity.rs` runs the reference C `htop` binary and `htoprs` on the same arguments and diffs output byte-for-byte, modulo the deliberate rebrand handled by `canon()` (`htoprs` → `htop`, differing version banner). `xutils_parity.rs` goes further: it compiles a tiny C harness (`tests/parity/cref/htop_cref.c`) *against htop's real XUtils.c* — the genuine 3.5.1 implementation, not a reimplementation — and diffs the Rust port's output against that compiled C oracle, function by pure function. The harness gates itself: `ref_available()` returns the reference path only if it exists and is the **3.5** series (`REF_HTOP_SERIES`), and the xutils path skips when the htop C source or a C compiler is absent, so CI without them stays green while a dev box with them runs the real diff. Point the binary comparison at a specific reference with `HTOP_REF=/path/to/htop`, and the source comparison with `HTOP_C_SOURCE=/path/to/htop`.

```

cargo build          # runs the port-purity gate
cargo test           # ported unit tests
cargo test --test parity # diff htoprs vs reference htop 3.5.x
cargo test --test parity cli # only the CLI cases
cargo test --test parity -- --ignored # documented gaps
python3 scripts/gen_port_report.py # regen the report

```

Because a TUI runs on the alternate screen with the tty in raw mode, a panic printed to the terminal would be painted over and lost the instant the screen is restored. htoprs therefore persists crashes to a logfile at `~/.cache/htoprs/crash.log` (honoring `$XDG_CACHE_HOME`, overridable with `$CRASH_LOG`), via `extensions::crashlog`. Two paths funnel into that one file: `log_panic`, called from a process-wide panic hook, and the ported SIGSEGV/SIGBUS handler's stderr report. Each entry records time, htoprs version, pid, thread, source location, panic message, and a full backtrace captured unconditionally — no `RUST_BACKTRACE=1` required — and entries are appended so the history survives across crashes. On a crash the restored terminal prints `htoprs: crash logged to <path>`.

Place in the stack

htoprs slots into the MenkeTechnologies CLI fleet as the process-monitoring front end, shipping a `#compdef htoprs` zsh completion whose option set is kept in lockstep with the getopt table in `commandline.rs`, plus the short and comprehensive man pages. It coexists with, rather than replaces, the C htop it ports — the parity suite depends on the original being installed as the oracle, and the xutils path links the original's `XUtils.c` directly. The honest framing is the one the README leads with: the C source is the spec, the port is faithful, and the engineering that matters is the architecture of the port, the two-tree gate that keeps it honest (a comment-and-string-aware brace state machine in `build.rs`, an empty allowlist, a 4,132-entry C-name index), the byte-for-byte parity harness that reaches down to individual C functions, the sixteen-entry clippy ledger that documents every place the spec overrides idiom, and the extension layer bolted on at keys htop leaves free — most recently the memory-exhaustion forecast. It is a re-engineering of a known tool, not a claim to have invented the process table.

storageshower — a real-time disk-usage TUI in Rust

What it is, and what it ports

`storageshower` is a terminal user interface for watching disk usage in real time. It enumerates every mounted filesystem, draws one colored usage bar per mount, refreshes on an interval, and lets you drill down into any mount to see which directories are consuming the space. It runs on macOS and Linux, is written entirely in Rust on top of `ratatui` and `crossterm`, and stores every preference you change back to a TOML config file so the next launch looks exactly like the last one you closed. The crate manifest describes it as a “Cyberpunk disk usage TUI,” and the visual language throughout — neon palettes, [`SYSTEM://DISK_MATRIX`] chrome, threshold “NOMINAL / WARNING / CRITICAL” badges — is deliberate, but underneath the styling it is a disciplined system-monitoring tool.

The lineage matters for honesty about what was invented here. `storageshower` is not a port of an external Unix utility. It is not `ncdu`, not `du`, not `df`. It is the Rust rewrite of the author’s own shell scripts, which still ship in the repository under `old_scripts/`: `storageShower.sh` carries a 2018 header and is a thin wrapper that dispatches to `storageshowerCyberpunk.sh`, a ~31 KB bash implementation. The original engineering problem — enumerate mounts, size directories, render bars, theme the whole thing — was solved first in bash. The work in this crate is the reimplementing of that concept as a compiled, threaded, tested Rust binary: the architecture, the background collectors, the drill-down scanner, the theme system, and the test suite are the original contribution, not the idea of a disk monitor. Where it overlaps conceptually with `ncdu` (recursive directory sizing with keyboard navigation) or `duf/df` (per-mount usage bars), it arrives at that overlap independently from the bash predecessor rather than by porting either tool’s code.

One thing to be precise about up front, because it is easy to assume a disk explorer has it: `storageshower` does not delete files. The `ncdu` workflow of pressing a key to remove the directory under the cursor has no analog here. The only `Delete` key handling in the codebase is inside filter-edit mode, where it removes a character from the filter string (`src/keys.rs:30`). The disk operations it does offer are open-in-file-manager, copy-path-to-clipboard, export, and bookmark — read and navigate, never destroy. That restraint is a design stance, not an oversight: the tool answers “where did my disk go” and hands you the path, then lets you act on it in whatever file manager or shell you already trust.

Architecture

The crate builds a single binary and no library consumers are expected in practice, though `lib.rs` exposes most modules publicly so the integration tests can reach them — `app`, `cli`, `columns`, `helpers`, `prefs`, `system`, `types`, and `ui` are `pub`, while `keys` and `mouse` are private modules whose tests live inline (`src/lib.rs:3`). The source is about 13,700 lines across a dozen modules:

- `ui.rs` (~3,860 lines) — the rendering layer: the disk list, drill-down view, help overlay, theme chooser, theme editor, tooltips, borders, and footer are all drawn here.
- `keys.rs` (~1,930 lines) — keyboard event handling for every mode, plus the bulk of the unit tests that live alongside the code.
- `cli.rs` (~1,550 lines) — the `clap` derive struct, the override logic, and the hand-formatted `--help`, `--version`, `--list-colors`, and `--export-theme` output.

- `system.rs` (~1,510 lines) — all platform data collection: mounts, directory sizing, I/O counters, SMART status, network-filesystem latency, system stats.
- `app.rs` (~1,320 lines) — the central `App` state machine that ties input, preferences, and rendering together.
- `mouse.rs` (~920 lines) — click, drag, scroll, and hover handling.
- `prefs.rs` (~690 lines) — load/save of the TOML config.
- `helpers.rs` (~680 lines) — formatting (`format_bytes`, `format_uptime`, `format_latency`, `format_rate`, `truncate_mount`), color math, layout helpers.
- `types.rs` (~695 lines) — the data model: `DiskEntry`, `DirEntry`, `SysStats`, `SortMode`, `BarStyle`, `ColorMode`, `UnitMode`, `ThemeColors`, `ViewMode`, `DrillSortMode`, `SmartHealth`.
- `columns.rs` (~230 lines) — column-width computation for the resizable layout.

The runtime is built around two ideas: a single-threaded render/input loop, and background worker threads that feed it through shared, mutex-guarded state.

The main loop lives in `run_app` (`src/main.rs:58`). It computes the effective refresh interval from `app.prefs.refresh_rate`, calls `app.refresh_data()` when that interval elapses, then in a scoped block reads the terminal size, subtracts the chrome height (border, header, and a two-row footer), and calls `ensure_visible` so the selection stays on screen — with a parallel `ensure_drill_visible` when the drill view is open. It re-sorts through `update_sorted`, draws one frame via `terminal.draw`, then polls `crossterm` for input with a 200 ms timeout. Key presses go to `app.handle_key`, mouse events to `app.handle_mouse` (with the current terminal width and height), `FocusLost` clears the hover state to `Default::default()`, and `Resize` is a no-op because the next draw already re-measures. This is a conventional immediate-mode TUI loop; nothing about the disk or system data collection happens on this thread.

The four non-interactive flags short-circuit before any of this: `main` (`src/main.rs:20`) checks `cli.help`, `cli.version`, `cli.list_colors`, and `cli.export_theme` in turn and returns after printing, so `--help`, `--version`, `--list-colors`, and `--export-theme` never initialize the terminal, never enable mouse capture, and never spawn the collector. Only after those checks does `main` build the initial snapshot with `collect_sys_stats` and `collect_disk_entries`, wrap it in an `Arc<Mutex<(SysStats, Vec<DiskEntry>)>>`, spawn the collector, enable `EnableMouseCapture` and `EnableFocusChange`, and enter the loop — and it disables both and calls `ratatui::restore()` on the way out regardless of how the loop returned.

The heavy data collection runs on a detached thread spawned by `spawn_bg_collector` (`src/system.rs:895`). It owns a `System` from `sysinfo`, and every three seconds it refreshes system stats, re-enumerates disks, computes I/O rates against the previous snapshot, and layers SMART health onto each entry, then writes the whole `(SysStats, Vec<DiskEntry>)` tuple into the shared mutex that the UI reads from:

```
pub fn spawn_bg_collector(
    shared: Arc<Mutex<(SysStats, Vec<DiskEntry>)>>,
) {
    std::thread::spawn(move || {
        let mut sys = System::new_all();
        let mut prev_io = read_io_counters();
        let mut prev_time = Instant::now();
        loop {
            std::thread::sleep(Duration::from_secs(3));
            sys.refresh_all();
            let stats = collect_sys_stats(&sys);
            let mut entries = collect_disk_entries();
            // ... I/O rates + SMART status applied here ...
            let mut lock = shared.lock().unwrap();
            *lock = (stats, entries);
        }
    });
}
```

The three-second cadence of the collector is independent of the user-facing refresh rate (1s / 2s / 5s / 10s,

cycled with the `f` key), which only governs how often the UI re-reads the shared snapshot. That decoupling is deliberate: expensive work like SMART queries and I/O counter reads happens at a fixed low frequency regardless of how snappy the display feels.

Disk enumeration

`collect_disk_entries` (`src/system.rs:427`) is platform-split three ways. On macOS it calls `getmntinfo(&mut mntbuf, MNT_NOWAIT)` directly through `libc`, walks the returned `statfs` array, and computes `total = f_blocks * f_bsize`, `avail = f_bavail * f_bsize`, `used = total - avail` with a saturating subtraction. On Linux it parses `/proc/mounts`. On any other Unix it falls back to `sysinfo::Disks`. Every path skips rows with an empty mount path — the doc comment notes CI runners occasionally emit unusable mount-table rows, and dropping them keeps the TUI from ever showing a blank mount. Each entry carries mount path, used/total bytes, percentage, disk kind, filesystem type, and optional latency, I/O rates, and SMART status. Two committed tests pin the contract: `collect_disk_entries_have_mount_points` asserts no entry has an empty mount, and `collect_disk_entries_pct_in_range` asserts every percentage lands in `0..=100` (`src/system.rs:1094`, `src/system.rs:1102`).

Drill-down scanning

Pressing `Enter` on a mount switches `ViewMode` to `DrillDown` and kicks off a background scan through `start_drill_scan` (`src/app.rs:234`). It sets `drill.scanning`, clears the previous entries and selection, resets the shared progress counters to zero, then spawns a thread running `scan_directory_with_progress` (`src/system.rs:232`), which reads the target directory, records the total entry count and a running processed count into two `Arc<Mutex<usize>>` handles so the UI can draw a progress bar, and for each child computes a size. Files use their own length; directories are sized recursively by `dir_size` (`src/system.rs:200`), which walks the tree with `read_dir`, accumulates with `saturating_add`, and — importantly — does not recurse through symlinks. A symlink counts as its own `symlink_metadata().len()`, not the tree it points at, which prevents cycle-driven runaway. A dedicated test, `scan_directory_symlink_cycle_completes_with_finite_sizes`, pins that behavior. Results come back sorted largest-first.

The completed scan is picked up on the render thread, not delivered by callback. Each `refresh_data` tick begins by checking `drill.scanning`; if set, it `take()`s the `Option<Vec<DirEntry>>` out of the shared `scan_result` slot, and when a value is present it swaps it into `drill.entries`, clears `scanning`, and re-sorts (`src/app.rs:194`). Navigation inside the drill view is breadcrumb-style: `drill.path` is a `Vec<String>` stack, `Enter` on a directory pushes the child path and rescans, `Backspace/Esc` pops the stack and rescans the parent — and popping past the last element returns to the disk list and clears the entries. Descent into a file, or into anything while a scan is still in flight, is ignored (`src/keys.rs:249`). Within a level, `s` sorts by size (toggling reverse if already size-sorted), `n` sorts by name, `r` flips the direction, `g/Home` and `G/End` jump to the ends, and `o` opens the current directory in the file manager.

Telemetry, I/O, SMART, and latency

Beyond raw usage, each `DiskEntry` can be annotated with live per-mount read/write throughput, drive health, and network latency. I/O rates come from `read_io_counters` (`src/system.rs:706` on macOS, with distinct Linux and fallback variants further down), which on macOS reads IOKit's `IOBlockStorageDriver` byte counters and on Linux reads sector counts from `/proc/diskstats`, mapping devices to mounts via `mount_to_device_map`, then diffing two snapshots over elapsed wall-clock time in `compute_io_rates`. SMART status is collected in `collect_smart_status` — macOS shells out to `diskutil info` and reads the `SMART Status` field, Linux reads `/sys/block/*/device/state` — and cached per base device; each variant returns `SmartHealth::{Verified, Failing, Unknown}` and the “other-OS” variant returns everything as `Unknown`. Network-filesystem latency (`measure_mount_latency`, `src/system.rs:177`) times a `read_dir` on the mount in a spawned thread with a 2-second timeout so a hung NFS/SMB/SSHFS mount cannot freeze the UI, and requires no root. `is_network_fs` (`src/system.rs:160`) recognizes `nfs`, `nfs4`, `cifs`,

`smbfs`, `afp`, `ncp`, `fuse.sshfs`, `fuse.rclone`, `fuse.s3fs`, `9p`, and `afs`, and is pinned by `is_network_fs_detects_known_types`.

Alerts, pause, and hover

`refresh_data` (`src/app.rs:193`) does more than copy the snapshot: it is also where threshold alerts fire. After swapping in the fresh disks it walks them, collects the set of mounts at or above the warn threshold, and diffs that set against the mounts it flagged on the previous tick. Any newly crossed mount produces a status line like `⚠ ALERT: /Volumes/data 92%`, flashes the alert banner by stamping `alert.flash` with the current `Instant`, and emits a terminal bell (`print!("\x07")`). Mounts that stay above the threshold do not re-alert; the alert set is rebuilt every tick so a mount that drops back below and later climbs again will alert a second time.

The `p` key sets `paused`, and a paused app returns from `refresh_data` before reading the shared snapshot — but only after the drill-scan pickup, so an in-flight drill still completes even while the disk list is frozen. Hover behaves on a timer: `hover_ready` (`src/app.rs:296`) reports a tooltip as visible only once the pointer has rested for 1000 ms, and auto-hides it after 4000 ms unless the hover came from a right-click, which pins it open. Tooltips can be toggled off entirely with `T`, which also drops a confirming status message.

The CLI surface

`storageshower` has no subcommands. It is a single invocation configured entirely by flags, parsed with `clap`'s derive API in `src/cli.rs`. The governing rule is that CLI flags override the config file, and almost every boolean flag has a `--no-` inverse so you can force a setting in either direction from the command line regardless of what the saved config says. Internally this is done with `clap`'s `overrides_with`: each visible flag and its hidden inverse override one another, so the last one on the command line wins and only one boolean survives to `apply_to` (`src/cli.rs:383`). The polarity of which half is hidden follows the default — `--no-bars`, `--no-border`, `--no-header`, `--no-used`, `--no-tooltips`, `--no-virtual`, and `--no-local` are the visible members whose positive counterparts are hidden, while `--reverse`, `--compact`, and `--full-mount` are the visible positives.

The flag families, all grounded in the `clap` struct and the README option tables:

- **Sorting:** `-s/--sort <name|pct|size>`, `-R/--reverse` (with hidden `--no-reverse`), `-l/--local-only` (with `--no-local`), `--virtual/--no-virtual`.
- **Display:** `-b/--bar-style <gradient|solid|thin|ascii>`, `--color <palette>` (30 builtins), `--theme <name>` for a custom theme, `-u/--units <human|gib|mib|bytes>`, `-k/--compact`, `-f/--full-mount`, and `--bars/--border/--header/--used/--tooltips` toggles.
- **Thresholds:** `-w/--warn <pct>` and `-C/--crit <pct>`.
- **Columns:** `--col-mount`, `--col-bar-end`, `--col-pct`, each a `u16` width with `0` meaning auto.
- **System:** `-r/--refresh <secs>`, `-c/--config <path>`, `-h/--help`, `-V/--version`.
- **Introspection:** `--list-colors` prints every builtin palette with a preview; `--export-theme` prints the current palette as TOML and combines with `--color` or `--theme` to export a specific one.

Note the layering: `-f` on the command line is `--full-mount`, while the interactive `f` key cycles the refresh rate, and `-r/--refresh` sets it from the shell — different surfaces, no collision. Representative invocations:

```
storageshower --color purple -b ascii # purple palette, ascii bars
storageshower -s pct -R                # sort by usage%, reversed
storageshower -l --no-virtual          # local physical disks only
storageshower -u gib -w 60 -C 85      # GiB units, custom limits
storageshower --list-colors           # preview builtin palettes
storageshower --export-theme --color blue # dump blue as TOML
```

Keys inside the TUI

Inside the interface the same knobs are reachable from the keyboard, and the handler in `src/keys.rs` is a mode stack: filter-edit, help, theme-chooser, theme-editor, and drill-down each capture input first, and only when none is active does the disk-list keymap run. On the disk list, `j/k` (and arrows) move the selection, `g/G` and `Home/End` jump to the ends, and `Ctrl+d/Ctrl+u` half-page while `Ctrl+g` returns to the top. Sorting is `n` (name), `u` (usage percent), and `s` (size); pressing the active sort key again flips direction, and `r` flips direction directly. Toggles are dense: `p` pauses data refresh, `l` local-only, `a` show-all, `v` bars, `d` used-vs-total, `g` header, `x` border, `m` compact, `w` full-mount, `T` tooltips. `b` cycles bar style through gradient → solid → thin → ascii, `i` cycles units through human → gib → mib → bytes, and `f` cycles the refresh rate 1 → 2 → 5 → 10 seconds.

Thresholds cycle through fixed steps rather than free entry: `t` walks the warn level 50 → 60 → 70 → 80 and wraps, and `z` walks the crit level 80 → 85 → 90 → 95 and wraps (`src/keys.rs:520`, `src/keys.rs:544`). The `/` key enters filter mode; `0` clears the filter without entering it. Filter mode is a full readline-style line editor over a `String` buffer with a cursor: `Ctrl+w` word-delete, `Ctrl+u` kill-to-start, `Ctrl+k` kill-to-end, `Ctrl+a/Home` and `Ctrl+e/End` for line motion, `Ctrl+b/Ctrl+f` and arrows for character motion, `Ctrl+h/Backspace` and `Delete` for erase, `Enter` to commit and `Esc` to restore the previous text (`src/keys.rs:14`).

The action keys operate on the selected mount. `o` opens it in the file manager (`open` on macOS, `xdg-open` on Linux, suppressed in `test_mode`), `y` copies the mount path to the clipboard, and `B` toggles a bookmark, pushing or removing the mount in `prefs.bookmarks` with a ★-prefixed status line. `e` writes a plain-text report of the whole disk matrix — host, timestamp, and a fixed-width `MOUNT / PCT / USED / TOTAL` table formatted through `format_bytes` — to `~/.storageshower.export.txt` (`src/keys.rs:621`). Mouse input is a first-class parallel path in `mouse.rs`: click to select, click again to drill, drag the column separators to resize, right-click for a pinned verbose tooltip, scroll to move the selection.

Themes and configuration

The theme system is the most developed non-monitoring feature. There are 30 builtin palettes defined as the `ColorMode` enum in `src/types.rs:31`, each with an internal name and a display name (`ColorMode::Default` renders as “Neon Sprawl,” `ColorMode::Green` as “Acid Rain,” and so on through “Zaibatsu”). A `ColorMode::ALL` slice, a `name()` map, and a `next()` method that indexes into `ALL` and wraps modulo its length drive in-app cycling, and tests assert the display names are all unique and that `next()` wraps cleanly through the full ring. On top of the builtins, users define custom themes as six-channel color sets (`ThemeColors: blue, green, purple, light_purple, royal, dark_purple`) either by hand in the config or through the live in-app editor bound to `C`:

```
[custom_themes.neonpink]
blue      = 199
green     = 46
purple    = 201
light_purple = 213
royal     = 196
dark_purple = 161

active_theme = "neonpink"
```

The theme editor is a six-slot picker over the 256-color palette. Opening it with `C` seeds all six slots from the current effective palette via `palette_for_prefs`, mapping each `Color::Indexed(n)` back to its index. `j/k` move between slots, `h/l` (and arrows) nudge the selected channel by ± 1 with `wrapping_add/wrapping_sub`, and `H/L` nudge by ± 10 — so a full sweep of the palette is a few keystrokes. `Enter` or `s` switches into naming mode, where the new theme is written into the custom-theme map under whatever name you type. The theme chooser bound to `c` is separate: it lists every builtin and custom theme, applies each as a live preview as you move through it with `j/k`, and restores the original on `Esc/q` — so browsing 30-plus palettes never commits an accidental change.

Every preference the app tracks — sort mode and direction, refresh rate, bar style, color mode, warn/crit thresholds, the visibility toggles, unit mode, column widths, the custom-theme map, the active theme, and the bookmark list — is serialized to `~/.storageshower.conf` in TOML via `serde` and restored on boot (`src/prefs.rs`). There is no explicit “save” step; the disk-list toggles call `self.save()` inline, so changing a setting persists it immediately. The first ten builtin palettes are also checked into `themes/` as ready-to-copy TOML files (`acid_rain.toml`, `data_stream.toml`, `ghost_wire.toml`, `ice_breaker.toml`, `neon_sprawl.toml`, and five more).

Formatting and units

All human-facing sizes go through `format_bytes` (`src/helpers.rs:3`), which is binary throughout: `Human` mode steps K/M/G/T at 1024, 1 048 576, 1 073 741 824, and 1 099 511 627 776 bytes with one decimal, while `GiB/MiB` force a single scale and `Bytes` prints the raw count with a `B` suffix. Rates use the same binary ladder (`format_rate`, B/s → K/s → M/s → G/s), latency reads as `<1ms`, `NNms`, or `N.Ns` past a second (`format_latency`), uptime collapses to `NdNhNm/NhNm/Nm` (`format_uptime`), and long mount paths are ellipsized with a trailing `...` by `truncate_mount`. Each of these has a cluster of unit tests immediately below it in the same module, which is where a large share of the crate’s inline test count comes from.

The RECLAIM_MAP overlay (in development)

The working tree carries an opt-in feature, `reclaim`, that is not part of the 0.28.8 release and is gated behind a Cargo feature so the default build pulls in zero extra dependencies. It adds a per-subtree estimate of how much space a directory could give back if compressed. The implementation lives in a `#[cfg(feature = "reclaim")] pub mod reclaim` inside `system.rs` (`src/system.rs:284`) and is honest about its method: it never compresses whole files. For each file it reads a bounded 64 KiB prefix (`SAMPLE_BYTES`), runs a single `lz4_flex` block over that sample to get a compression ratio, and projects `size - size/ratio` as the reclaimable bytes; a `shannon_entropy` helper is provided as a fast compressibility proxy. Directory totals fan out over `rayon` (`dir_reclaim` uses `par_iter().reduce`), symlinks are not followed to match the size scanner’s recursion policy, and unreadable files are treated as incompressible (ratio 1.0, zero reclaimable). When enabled, `scan_directory_reclaim` runs the ordinary sized scan and then `annotates` each `DirEntry`’s new `reclaimable` and `ratio` fields; the drill-down `c` key toggles the overlay, switches the drill sort to `DrillSortMode::Reclaim`, and rescans the current subtree so the estimate is populated. Two dependencies back it — `rayon 1.12` and `lz4_flex 0.13`, both marked `optional` in `Cargo.toml`. Treat this as active work: the feature is wired end to end behind the flag, but it is unreleased and the default (non-`reclaim`) build in the current working tree does not yet cover the new `DrillSortMode::Reclaim` variant in every match arm, so the shipped 0.28.8 remains the reference behavior.

Integration into the stack

`storageshower` installs as a standalone command, via the project’s Homebrew tap (`brew install storageshower`) or `cargo install storageshower`. It ships a `zsh` completion (`completions/_storageshower`, a `#compdef`-headed `_arguments`-style file) and two man pages: `storageshower.1`, the standard reference, and `storageshowerall.1`, an all-in-one page modeled on `zshall(1)` that concatenates invocation, keybinds, themes, drill-down, config, and alerts into a single greppable document. Within the wider environment it is a terminal-native monitoring surface — it reads mounts and directory sizes, opens paths in the file manager, and copies paths to the clipboard, so it slots in as the “where did my disk go” tool alongside the rest of the CLI suite rather than owning any part of the shell runtime.

Status

The version in the manifest and man pages is **0.28.8**, edition 2024 (`Cargo.toml`, `man/man1/storageshower.1` dated 2026-07-05), which implies a Rust $\geq$ 1.85 toolchain since edition 2024 stabilized there. Dependencies are deliberately foundational: `ratatui` 0.30, `crossterm` 0.29, `sysinfo` 0.38, `dirs` 6, `clap` 4, `serde` 1, `toml` 1.1, `libc` 0.2, with `rayon/lz4_flex` optional behind the unreleased `reclaim` feature. Dev-dependencies are `criterion` 0.8, `serde_json`, `tempfile`, and `toml`. The release profile is `opt-level = 3`, `lto = true`, `strip = true`.

The test surface is large and I verified the counts in this session: 1,336 integration test files under `tests/`, each a separate `*.rs` integration binary, carrying 3,101 `#[test]` functions between them (a `grep` count), plus 596 unit tests embedded in `src/` modules (also a `grep` count) — about 3,697 test functions in total, though I did not compile and run the full suite so the passing count is unverified. Criterion benchmarks live in `benches/bench.rs`; the README reports formatting operations in the tens of nanoseconds and full disk-list rendering for 200 mounts around 15.7 μ s on Apple Silicon, all **provisional** as I did not re-run the benchmarks. CI is two workflows: `ci.yml` runs library unit tests, integration tests, and doc tests (each `cargo test --locked` separately), plus `cargo fmt --all --check` and a `clippy` job, on an `ubuntu-latest` + `macos-latest` matrix; `release.yml` builds `--release --locked` across macOS and Linux targets, using `cross` for the cross-compiled aarch64 Linux build.

zcolorizer — the log stream, repainted in real time

What it is, and what it ports

`zcolorizer` is a streaming log colorizer. You pipe a log into it — from `tail -f`, from `journalctl -f`, from a file argument, from anything that produces text on a file descriptor — and it emits the same text back with ANSI escapes inserted so that dates, hostnames, IP addresses, error keywords, HTTP codes and dozens of other semantic fields each carry a color. The default look is a 256-color cyberpunk palette; everything about which text gets colored and what color it gets is user-editable through a TOML config with no recompile.

It is honest about its lineage. The `Cargo.toml` description reads “Real-time log file colorizer (ccze/pygments port)”, and the crate ships a full copy of the original C source under `vendor/ccze/` as a build-time reference. The upstream tool is `ccze` by cornet — a curses-based log colorizer from the early 2000s whose architecture is a set of format *plugins* (`mod_syslog.c`, `mod_httpd.c`, `mod_squid.c`, and so on), each of which matches whole-line PCRE patterns, paints the numbered capture groups with semantic colors, and hands the free-text remainder to a generic “word colorizer.” `zcolorizer` ports that architecture faithfully — the module set, the module-then-wordcolor precedence, the semantic token vocabulary — into Rust. The second influence, credited in the same breath, is the `pygments` “regex → token” idea: a lexer rule is a regex, and the thing it produces is a named token that a separate theme decides how to render.

The invention here is not the idea of colorizing logs; `ccze` did that. The engineering is the Rust rewrite: a three-layer decoupled architecture (rules / themes / engine), a streaming binary that is safe on binary and pre-colored input, a 47-module format library that extends `ccze`’s ~20 plugins into formats that did not exist in 2003 (systemd, JSON, Kubernetes, Postgres), a themeable color model that carries 32 palettes, a `--themes-json` surface built so a GUI theme-picker can drive the whole thing, and — the one piece with no `ccze` ancestor — a streaming statistical-novelty coloring mode that decides brightness from how often a line’s *shape* has been seen rather than from any rule. The vendored upstream C source — `ccze`, kept only as a parity reference — is explicitly excluded from the published crate (`exclude = ["/vendor", ...]` in `Cargo.toml`); the Rust is original work, MIT-licensed.

The three decoupled pieces

The design principle stated at the top of `src/lib.rs` is that three things are held apart so any one can change without disturbing the others:

- **Rules** (`src/rules.rs`) decide *which* span of text earns a semantic token — `date`, `error`, `host`, `ip`, and so on. A rule is a regex. Its **named capture groups are the tokens**: the pattern `(?P<date>\d{4}-\d\d-\d\d)\s+(?P<host>\S+)` paints the date field and the host field from a single expression, each in its own color. A rule with no named groups tags its whole match with one `token`. This is exactly the `pygments` model, and it is what lets rules live in the config as editable text.
- **Themes** (`src/theme.rs`) decide *what each token looks like*: a named map from token string to a `Style` (foreground color plus optional background and bold/dim/italic/underline/blink/reverse). Because color lives entirely in the theme, swapping the active theme recolors every log without touching a single rule.
- **Engine** (`src/engine.rs`) runs the compiled rules over each line and paints the spans they claim with the active theme.

`Colorizer::from_config` in `lib.rs` ties them together: it resolves the effective rule definitions from the config, compiles them, resolves the theme, and returns a ready `Colorizer`. A second constructor, `Colorizer::default_cyberpunk`, hands back the builtin generic rules plus the neon-sprawl theme with no config file at all, and it *expects* success on the grounds that the builtin rules and theme are always valid — the fast path a Rust host embedding the crate reaches for.

The matching engine and its ownership model

The core of `src/engine.rs` is `colorize_line`, and its central data structure is a byte-owner map. For a line of length `len`, it allocates `owner = vec![usize::MAX; len]` — one slot per byte, where `usize::MAX` means “unclaimed.” Rules run in listed order. Each rule’s matches call `claim(start, end, token)`, which walks `owner[start..end]` and sets any slot still equal to `usize::MAX` to that token’s interned id — but never overwrites a slot an earlier rule already claimed. The rendering pass then walks the owner map, groups maximal runs of equal owner, and emits one styled chunk per run; unclaimed runs get the theme’s `default` style.

This is the whole precedence story: **the first rule to claim a byte owns it**. A structured syslog-prefix rule listed before the generic bare-number rule will paint `192.168.0.1` as one `ip` span rather than letting the number rule chop out the octets — verified by the engine test `earlier_rule_wins_overlap`, which asserts the address renders as a single `38;5;135` chunk with exactly one reset. It is the same module-then-wordcolor ordering ccze used, expressed as span ownership rather than as a plugin dispatch chain.

The owner-building work is factored out of the render path into `build_owner`, a private method that runs every rule over a sanitized, non-empty line and returns `(Vec<usize>, Vec<Bstr>)` — the byte-to-token-index map plus the interned token names, all borrowing from `self.rules` under one lifetime. `colorize_line` calls `build_owner` and then `render`; the novelty path calls the same `build_owner` and then a sibling `render_intensity`. The two rendering functions are byte-for-byte the same run-grouping loop; they differ only in whether each chunk’s `Style` is first passed through `with_intensity`. Factoring the owner pass out this way is what keeps the novelty mode from being a parallel reimplement of the matcher — it reuses the exact span logic that the plain path is tested against.

Before any rule runs, every line passes through `sanitize`. Piped logs routinely carry pre-existing terminal escapes (a `tail` of an already-colored file) and raw binary bytes. If the colorizer ran a number rule over the digits inside an existing `\x1b[31m` sequence, it would inject its own SGR codes mid-escape and shred it into visible garbage. So `sanitize` strips full ANSI/VT escape sequences (CSI, OSC terminated by BEL or ST, and two-byte escapes) and every C0 control byte except tab, plus DEL and the C1 range. Tabs survive; the common case — a clean line — is returned borrowed, unchanged, with no allocation. The engine’s test suite pins all of this: OSC-8 hyperlink stripping, binary-byte dropping, a pure-control line collapsing to empty, and multibyte UTF-8 words surviving span slicing next to digit runs (span ends always land on regex match offsets, which are always char boundaries).

The color model

`src/color.rs` defines a `Color` as one of three representations, distinguished at parse time by an untagged serde enum: one of the 16 **named** ANSI colors (`"red"`, `"bright_cyan"`), a 256-color palette **index** written `{ index = 213 }`, or a 24-bit **truecolor** RGB triple written either as `"#ff00aa"` or `{ rgb = [255, 0, 170] }`. A `Style` bundles a foreground color, an optional background, and six boolean attributes. Its `prefix()` method emits the SGR opening sequence in a fixed canonical order — attributes (`1;2;3;4;5;7` for bold/dim/italic/underline/blink/reverse) then foreground then background — and `paint()` wraps text in that prefix plus a reset, returning the text verbatim when the style is plain so a colorless token emits no escape at all. Named backgrounds are foreground code + 10; indexed foregrounds use `38;5;N`, backgrounds `48;5;N`; truecolor uses `38;2;r;g;b`.

The intensity model

`Style::with_intensity(t)` is the color model's one novelty-aware method, and it is deliberately narrow: it takes a novelty score `t` in `[0, 1]` and adjusts *terminal attributes only*, never touching the foreground or background color. A token keeps its palette hue at every intensity; only its weight changes. The mapping is a clamped step function:

- `t >= 0.9` — first-seen or anomalous: set `bold` and `reverse`, clear `dim` (the line blazes in inverse video).
- `t >= 0.4` — rare: set `bold`, clear `dim`.
- `t < 0.15` — high-frequency noise: set `dim`, clear `bold` and `reverse`.
- otherwise (`0.15 <= t < 0.4`) — the style is returned unchanged.

Because the input is clamped to `[0, 1]` first, an out-of-range score degrades to one of the endpoints rather than misbehaving. Keeping color fixed and moving only weight is what makes the mode legible: a `journalctl` stream stays recognizably neon-sprawl-colored, but the eye is pulled to the bold-reverse lines and slides past the dimmed ones. This is separable from everything else — the theme, the rules, the modules all work identically; intensity is a post-filter applied to whatever `Style` a token already resolved to.

The novelty model

`src/novelty.rs` is the streaming statistics behind that intensity. `NoveltyModel` is an online frequency table: a `HashMap<u64, u32>` from a line-template hash to how many times that template has been seen, plus a running `total` and an optional decay factor. Its one hot method, `observe_and_score(hash)`, records a sighting and returns the template's novelty score in `[0, 1]`. The score is `1.0` on first sight, and thereafter `1 / (k + 1)` where `k` is the number of prior sightings — so a template's score falls monotonically `1.0, 0.5, 0.33, 0.25, ...` as it recurs. Feed that score straight into `with_intensity` and the behavior falls out for free: a first-seen line blazes, the first repeat is still bold, and once a template has been seen seven or more times its score has dropped below `0.15` and it dims into the background. The model's tests pin exactly this: `first_seen_scores_high_repeats_decay_toward_dim` asserts the score strictly decreases on every repeat and lands below `0.15` within a dozen sightings, and `distinct_templates_tracked_independently` asserts that hammering one template to noise does not bleed into an unrelated template's first-seen `1.0`.

The trick that makes this work on real logs is *templating*, and it lives in the engine as `template_hash`. Scoring the raw line would be useless — every line of a busy stream is textually unique because timestamps and IDs change — so before hashing, the engine masks out the spans whose *value* is a variable field. `VARIABLE_TOKENS` in `engine.rs` names the eight token classes treated as variable: `number`, `ip`, `address`, `pid`, `size`, `version`, `date`, and `time`. `template_hash` walks the same owner map the renderer uses; for a run owned by one of those variable classes it hashes a sentinel byte plus the *class name* instead of the literal bytes, and for everything else — literal text and non-variable tokens like `error` or `host` — it hashes the bytes verbatim. Two syslog lines that differ only in their timestamp, PID, and source IP therefore collapse to one template hash, while a line whose message text changes produces a new one. The class name is hashed rather than a single universal mask, so an `ip` span and a `number` span in the same position stay distinguishable — swapping a bare port number for an IP is a real structural change and reads as novel.

Decay is optional. `NoveltyModel::with_decay(factor)` multiplies every count by `factor` (clamped to `[0, 1]`) once every `DECAY_INTERVAL` (512) observations, dropping any template whose count rounds to zero. Batching the sweep every 512 observations keeps its `O(distinct_templates)` cost off the per-line hot path. The effect is a sliding window of attention: a template that was noise but has gone silent is eventually forgotten, so if it returns it re-lights as first-seen. The `decay_forgets_a_silenced_pattern` test hammers a template to 30 sightings, flows four decay-intervals of unrelated traffic past it, and asserts its next appearance scores `1.0` again. A `factor >= 1.0` is normalized to `None` (pure cumulative counting), so `with_decay(1.0)` is exactly `new()`.

Nothing in this mode requires a rule that knows the log format. The masking is driven entirely by which builtin

tokens the generic ruleset already assigns, so a `tail -f` of an unknown, repetitive service self-quiets while an anomalous line — a stack trace, a `SEGFAULT`, an unexpected message — surfaces in bold reverse, with no configuration at all. This is the piece of `zcolorizer` with no `ccze` lineage; `ccze` colors by pattern class only, and the statistical-intensity layer is original.

The color model in the CLI

`--novelty` turns the mode on; `--novelty-decay D` sets the decay factor and *implies* `--novelty`. The flag plumbing is in `build_novelty`: given `D`, a value in `(0, 1]` builds a decaying model, a value outside that range is rejected with `--novelty-decay: must be in (0, 1], got ...`, and with no `D` the presence of `--novelty` alone builds a non-decaying `NoveltyModel::new()`. The model lives on the `LineProcessor` and persists across `--watch` config reloads — it tracks the *stream*, not the theme, so swapping palettes mid-run must not reset the frequencies it has learned. A worked run through the default palette, with the variable fields masked so the four connection lines collapse to one template:

```
$ printf 'Connection from 10.0.0.1 port 22\n\
Connection from 10.0.0.2 port 22\n\
Connection from 10.0.0.3 port 22\n\
Connection from 10.0.0.4 port 22\n\
SEGFAULT in worker pid 991\n' | zcolorizer -C --novelty
<1;7>Connection ... 10.0.0.1 ... port 22      # first-seen: bold+reverse
<1> Connection ... 10.0.0.2 ... port 22      # k=1, score .5:  bold
      Connection ... 10.0.0.3 ... port 22      # k=2, score .33: plain
      Connection ... 10.0.0.4 ... port 22      # k=3, score .25: plain
<1;7>SEGFAULT in worker pid 991              # new template:  blaze
```

(Attribute prefixes shown as `<...>`; hues omitted.) The four `Connection` lines differ only in their IP and are collapsed by the `ip/number` mask into one template, so their scores march `1.0 → 0.5 → 0.33 → 0.25` and their weight drops from bold-reverse to plain across four lines. The `SEGFAULT` line is a template never seen before and blazes even though it carries no `error` keyword the ruleset recognizes.

The theme grammar in depth

A theme is a `name`, a `description`, a `base` style (the fallback for any token not otherwise mapped), and a `styles` map keyed by token name. Token names are plain strings, so a config can invent new tokens freely; the well-known ones the builtin rules emit are enumerated in `theme::tokens` — a large const module covering the generic vocabulary (`date`, `time`, `host`, `ip`, `mac`, `pid`, `process`, `error`, `warning`, `debug`, `info`, `good`, `bad`, `system`, `keyword`, `email`, `uri`, `dir`, `file`, `size`, `version`, `number`, `address`, `percentage`, `signal`, `http_method`, `http_code`, `string`, and more), the `ccze`-plugin tokens (`gettime`, `getsize`, `ftp_code`, `ident`, `ctype`, `subject`, `field`, `chain`, `pkg`, `pkgstatus`, `incoming/outgoing/unique` for `exim`, `repeat`, `swapnum`, and the eleven `proxy_*` squid actions), and the modern-format tokens (`thread`, `level`, `facility`, `duration`, `latency`, `hash`, `json_key`).

The 32 themes are not hand-written token-by-token. Thirty-one of them are generated from `palettes` ported verbatim from the shared MenkeTechnologies theme set (`iftoprs`, itself derived from `storageshower`). A `Palette` is a kebab id, a display name, and six 256-color indices `[c1..c6]` standing for primary, brightest accent, secondary, mid, dim, and darkest. The declaration order matters: `neon-sprawl` is first and is the default (aliased `cyberpunk`), followed by `acid-rain`, `ice-breaker`, `synth-wave`, `rust-belt`, `ghost-wire`, `red-sector`, `sakura-den`, `data-stream`, `solar-flare`, `neon-noir`, `chrome-heart`, `blade-runner`, `void-walker`, `toxic-waste`, `cyber-frost`, `plasma-core`, `steel-nerve`, `dark-signal`, `glitch-pop`, `holo-shift`, `night-city`, `deep-net`, `laser-grid`, `quantum-flux`, `bio-hazard`, `darkwave`, `overlock`, `megacorp`, `zaibatsu`, and `iftopcolor`.

`theme_from_palette` is where six colors become a full styling for roughly sixty tokens. It assigns each token

to a palette role and multiplies the six colors into enough distinct looks by layering bold, italic, and underline: **date** is bold-accent, **time** plain-accent, **host** bold-secondary, **ip** plain-secondary, **uri** underlined-secondary, severity tokens lean on the brightest accent so **error** stays loud in every palette. Where two tokens would otherwise collide on the same palette slot — **process** versus **host** — the code derives an extra distinguishable shade with **shift_lighter**, an index-cube arithmetic function ported from iftoprs that steps a 256-color index one notch brighter (clamping at cube and grayscale-ramp edges). The theme tests assert every palette builds, that **error** is bold in all 31, that **process** and **host** never render identically, and that unmapped tokens fall through to **base**.

The thirty-second theme is **ccze-classic**, a hand-written 16-color map that approximates the original ccze palette (cyan dates, bold-red errors, bold-yellow warnings) for parity and nostalgia. **builtin()** returns the 31 generated palettes chained with **ccze-classic** — 32 in total, confirmed at runtime by **--list-themes | wc -l**. Name resolution is forgiving: **normalize** lowercases, collapses spaces and underscores to hyphens (so "Neon Sprawl", **neon_sprawl**, and **NEON-SPRAWL** all match), and treats **cyberpunk** and **default** as aliases for **neon-sprawl**.

Themes compose. A **[[themes]]** block naming an existing builtin overlays only the tokens it sets via **merged_with**, keeping the rest; a block with a fresh name defines a whole theme (set **base** so unlisted tokens still have a color). That is how the config example makes errors louder without redefining the palette:

```
[[themes]]
name = "cyberpunk"           # overlay the builtin
[themes.styles.error]
fg = "#ff003c"
bold = true
underline = true

[themes.styles.pid]
fg = "#b14bff"               # tone PIDs down to a muted violet
```

The rule library

builtin_generic() in **rules.rs** is the port of ccze's word-colorizer plus a handful of structured-line rules, and it is what runs on any log out of the box. It leads with named-group timestamp rules (ISO 8601, syslog **Mon DD HH:MM:SS**, and **proc[pid]:**), then severity word-lists compiled case-insensitively (**ERROR|ERR|CRITICAL|FATAL|PANIC...** → **error**; **WARNING|WARN** → **warning**; and so on), ccze's **good/bad/system** word lists, HTTP methods and status codes, network identifiers (IPv4, IPv6, MAC, email, URI, hostname), absolute paths, hex addresses, sizes, versions, percentages, signal names, bare numbers, and double-quoted strings. Each **RuleDef** carries a name, a pattern, an optional whole-match token, and an **ignore_case** flag that compiles as an inline **(?i)**. The severity, **good**, **bad**, and **signal** word lists double as the semantic anchors the novelty mode leans on: because they are *not* in **VARIABLE_TOKENS**, a line's error and signal keywords survive templating verbatim, so a line that gains a **SEGFAULT** or a **panic** reads as a structurally new template even when its numbers and timestamps are masked away.

Format modules

The generic ruleset colors anything; a **module** colors a known format's structured fields precisely. A **Module** is a named bundle of **RuleDefs** whose start-anchored named-group patterns pin the shape of a whole line to a format. There are 47 modules (confirmed by **--list-modules | wc -l**), in two families. The classic ccze plugin ports live in **src/modules.rs** — 20 of them: **syslog**, **httpd**, **squid**, **dpkg**, **postfix**, **exim**, **procmail**, **proftpd**, **vsftpd**, **ftpstats**, **xferlog**, **php**, **oops**, **icecast**, **fetchmail**, **apm**, **distcc**, **sulog**, **super**, **ulogd**, each translated from its **vendor/ccze/src/mod_*.c** original with the C source as the spec. The 27 modern modules in **src/modules_modern.rs** cover formats ccze never saw: **json**, **logfmt**, **journald**, **authlog**, **kernel**, **cron**, **auditd**, **nginx**, **haproxy**, **caddy**, **traefik**, **postgres**, **mysql**, **redis**,

mongodb, klog, docker, spring, python, rust-log, rails, named, dnsmasq, dhcpd, fail2ban, ufw, and dovecot.

Modules layer on top of the generic rules and win on overlap, but a free-text message still flows through the generic word-colorizer — ccze's exact module-then-wordcolor design, expressed here as the module rules sitting between user rules and the generic builtins in `resolve_rule_defs`. Enabling every module with `-m all` is safe because each module's structured patterns are `^`-anchored, so they only fire on lines that genuinely match that format. Where ccze chose a color dynamically by substring — squid's `TCP_HIT/TCP_MISS`, an HTTP method — the port emits small follow-up whole-match rules for each variant token, because the Rust regex crate forbids duplicate group names in one pattern (the `apm` module is split into single-field rules for the same reason, noted in its source comment).

The CLI surface and how it feeds

With no file arguments the binary reads stdin, line-buffered, so `tail -f` and `journalctl -f` stream live; with file arguments it colorizes each in turn. Color auto-disables when stdout is not a terminal unless `-C/--force-color` is given (the path for piping into `less -R`), and `--no-color` forces passthrough. The real invocations:

```
tail -f /var/log/syslog | zcolorizer
journalctl -f | zcolorizer --theme cyberpunk
zcolorizer -m httpd access.log | less -R
tail -f app.log | zcolorizer -g '(?i)error' # color, no
tail -f app.log | zcolorizer -H ERROR # dim non-matches
tail -f /var/log/syslog | zcolorizer -m auto --watch
tail -f app.log | zcolorizer --novelty # noise self-dims
tail -f app.log | zcolorizer --novelty-decay 0.9
```

`-g/--grep` filters to matching lines while keeping color, unlike `| grep`, which strips it; `-H/--highlight` keeps every line but renders the non-matches dim so matches stand out in context. Grep and highlight are compiled with `compile_filter`, which surfaces a bad regex tagged with the offending flag name rather than a bare regex-crate error. `-m/--module` is repeatable and merges with any `modules` in the config. The `auto` pseudo-module samples the first 200 lines (`SNIFF_LINES`), fingerprints the format from each module's start-anchored signature, and enables only what is present — no input is dropped, because the sniffed sample is buffered and replayed. Formats identifiable only mid-line (JSON, logfmt) have no anchored signature and are deliberately not auto-enabled; you name them with `-m json`. `-w/--watch re-stats` the config on a 200 ms throttle and rebuilds rules and theme mid-stream when the mtime changes, keeping the previous good config if a half-saved edit fails to parse — the live half of the “pick the theme from zgui” flow. When novelty is active, the reload preserves the running `NoveltyModel`, because the frequencies belong to the stream and not to the config being reloaded.

Running the syslog example through auto-detection produces the paint directly:

```
$ printf 'Jun 27 14:03:11 host sshd[1234]: Failed \
password for invalid user admin from 10.0.0.5 \
port 22\n' | zcolorizer -C -m auto
zcolorizer: -m auto detected: syslog, journald, authlog
<1;38;5;48>Jun 27 14:03:11 <1;38;5;135>host
  <1;38;5;177>sshd [<1;38;5;141>1234]:
  <1;38;5;48>Failed password for invalid user
  <1;38;5;141>admin from <38;5;135>10.0.0.5
  port <38;5;141>22
```

(Escape codes shown as `<...>` for legibility.) The date, host, process, pid, the `Failed password` bad-phrase, the user, the IP, and the trailing port number each land in their neon-sprawl token color.

Several flags exist purely to feed tooling and introspection. `--list-themes` marks the active theme with

*; `--list-modules` prints each module and its description; `--list-rules` prints the effective ruleset after the config merge; `--dump-config` re-emits the resolved config as normalized TOML; and `--themes-json` resolves every available theme to a full struct and emits `{ "active": ..., "themes": [ ... ] }` so a GUI picker can render real swatches from each token's style. The `-h/--help` output is the house-style cyberpunk banner (an ANSI-Shadow wordmark with a cyan-to-magenta truecolor gradient, a status box, and green `//` descriptions) rendered by hand rather than by clap — clap's own help is disabled with `disable_help_flag = true` and `-h/--help` is captured as an explicit `help` field so the hand-drawn banner runs instead. That banner carries its own — NOVELTY — section documenting `--novelty` (“intensity = statistical novelty; noise self-dims”) and `--novelty-decay D` (“age counts (0<D<=1) so silenced patterns relight”).

Integration and stack shape

The crate is built to be embedded, not only invoked. `Cargo.toml` declares `crate-type = ["rlib", "staticlib", "cdylib"]` — an rlib for native use from Tauri hosts and sibling Rust crates, plus staticlib/cdylib for a future C ABI. The library never depends on clap; the CLI binary is gated behind a `cli` feature (`default = ["cli"], cli = ["dep:clap"]`), so a host linking the rlib pulls in none of the argument-parsing machinery. Dependencies are deliberately foundational and vendorable, per the meta durable-deps rule: `serde`, `serde_json`, `toml`, `regex`, `thiserror`, `dirs`, `once_cell`, and `clap` for the binary only. The release profile sets `lto = true` and `strip = true`. The default config path is `$XDG_CONFIG_HOME/zcolorizer/config.toml`, resolved through `dirs`. Together with `--themes-json`, `--dump-config`, and `--watch`, that shapes the intended workflow: a GUI enumerates themes from the JSON, writes the chosen one into the config, and the running `zcolorizer` picks it up live without a restart. The whole `NoveltyModel` type is re-exported from the crate root alongside `Colorizer`, `Color`, `Style`, `Config`, and `Theme`, so an embedding host can drive the statistical-intensity path directly without going through the CLI.

Status

Provisional, from the working tree: `Cargo.toml` version is `0.2.2` (recent commits past the `v0.2.2` tag touch the docs/HUD color schemes and the help footer, not the crate version). `cargo test` passes 94 unit tests, 17 integration tests, and 1 doc-test — 112 total, all green (the unit count grew from the earlier 91 as the `novelty` module's three tests and related engine coverage landed). Runtime enumeration confirms 47 format modules and 32 themes. The README's “31 cyberpunk palettes” and the help line's “31 cyberpunk themes” both count the generated palettes; the 32nd builtin is `ccze-classic`. Working today: the streaming CLI, the regex → token engine with span-ownership precedence and input sanitization, TOML config with custom rules and theme overrides, truecolor/256/named color, all 32 themes, all 47 modules, `-m auto` detection, `--watch` live reload, `--themes-json` for tooling, and the streaming novelty mode (`--novelty` / `--novelty-decay`) with per-template intensity, field-masked templating, and optional exponential decay that persists across watch reloads.

grcrs — a two-binary Rust port of the Generic Colouriser

`grcrs` is a faithful Rust port of `grc`, the Generic Colouriser (upstream version 1.13, by Radovan Garabík). The upstream tool is a pair of Python programs that colourise the output of arbitrary commands by matching lines against regexp/colour rules. `grcrs` reproduces that pair as two compiled binaries — `grc` (the launcher) and `grcat` (the colouriser) — keeping the same option surface, the same config-file search order, the same colourfile grammar, and the same line-carried streaming-colour behaviour. The port declares version `1.13.0` in `Cargo.toml`, aligning its version number with the upstream release it tracks.

The engineering here is not the invention of the colourising model — that model is `grc`'s, and the bundled `grc.conf` plus the 83 colourfiles under `vendor/grc/colourfiles/` are the upstream data files carried verbatim. The engineering is the Rust reimplementation of both programs: reproducing Python's `re` regexp semantics on top of `fancy-regex`, reproducing Python's `surrogateescape` byte handling so non-UTF-8 command output survives round-trip, reproducing `getopt`'s abbreviated-long-option matching, and building an end-to-end test suite whose expectations were cross-checked byte-for-byte against the reference Python `grcat`. The whole thing depends on exactly two crates: `fancy-regex 0.14` and `libc 0.2` (`Cargo.toml`), with a release profile using `lto = true` and `codegen-units = 1`. Beyond the port, the Rust version adds one directive family that upstream has no equivalent for — the streaming `trend/key/metric` extension covered further below — kept strictly additive so that any colourfile written for `grc 1.13` behaves identically here.

What it does

`grc <command>` runs a command and colourises its output. It joins the command and its arguments into a single string, scans `grc.conf` top to bottom for the first regexp that matches, loads the colourfile named on the following line, runs the command, and pipes the command's stdout (and/or stderr) through `grcat <colourfile>`, where per-line regexp-driven ANSI colouring is applied. If no regexp matches, the command runs uncoloured with inherited stdio.

```
grc ping example.com      # colourised ping
grc netstat -tulpn        # colourised netstat
grc df -h                 # colourised disk usage
grc ifconfig              # colourised interface dump
grc make                  # colourised build log
```

The two-program split is the same one upstream uses, and it matters for how the pieces compose. `grc` is only a dispatcher: it decides *which* colourfile applies and *how* to wire the child's file descriptors. `grcat` is a standalone filter that reads a colourfile and stdin and writes coloured stdout — it can be driven directly from a shell pipeline (`some-command | grcat conf.log`), which is how the colourising logic stays independent of the launch logic. The colouriser refuses direct misuse only in the trivial sense: called with anything other than exactly one argument it prints `You are not supposed to call grcatrs directly...` to stderr and exits 1 (`src/grcatrs.rs:548`), matching the upstream guard.

The launcher: `src/grcis.rs`

The `grc` binary is built from `src/grcis.rs`. Its job is option parsing, config resolution, and process orchestration.

Option parsing lives in `parse_opts` and mirrors `getopt`. Short options cluster (`-se` sets both stdout and stderr redirect); `-c` takes its value either attached (`-cconf.x`) or as the next argument (`-c conf.x`); `--` terminates option scanning so everything after it is treated as the command even if it looks like a flag; and option scanning otherwise stops at the first non-option token, which begins the command (`src/grcis.rs:42`). The long options are `stdout`, `stderr`, `config`, `colour`, and `pty`. The port reproduces `getopt`'s prefix behaviour through `resolve_long` (`src/grcis.rs:120`): an exact name wins, otherwise a *unique* prefix resolves to its option, and an ambiguous or unknown prefix returns `None` — which `grc` turns into a `help()` call and exit. So `--conf=name` resolves to `--config`, `--col=off` to `--colour`, and `--pt` to `--pty`, but `--std` is rejected because it could be either `--stdout` or `--stderr`. This is the exact `getopt` contract the Python original inherits, reproduced explicitly rather than approximated.

One consequence of routing every unrecognised long option through `help()` is that `grc --version` is not a dedicated flag at all: `resolve_long("version")` fails to resolve, `help()` fires, and the help banner — whose first line is `Generic Colouriser 1.13` — is what gets printed before the exit-0. That is deliberate and is what the CI smoke test leans on: it runs `grc --version` and greps the output for `Colouriser`, so the “version check” is really a check that the help path still emits the upstream banner string.

The colour default is computed from the terminal: `colour` initialises to `libc::isatty(1) == 1` (`src/grcis.rs:47`), and `--colour=auto` re-evaluates the same `isatty` check, so colour is on when stdout is a tty and off when piped, unless forced with `--colour=on/--colour=off`.

Config resolution runs through `conffile_list` and `find_config`. The `grc.conf` search order is `/etc`, `/usr/local/etc`, `/opt/homebrew/etc`, `$XDG_CONFIG_HOME/grc`, then `~/.grc` (`src/grcis.rs:147`), with `$XDG_CONFIG_HOME` defaulting to `~/.config` when unset. Two precedence rules matter and are reproduced deliberately. Within a single `grc.conf`, the *first* matching regexp wins — `scan_conf_for_cfile` returns the line following the first match (`src/grcis.rs:173`). Across the search path, `find_config` keeps scanning every file and lets the *last* match overwrite earlier ones (`src/grcis.rs:202`), so a higher-precedence user config beats a system config. Each non-comment, non-blank line in `grc.conf` is a candidate regexp and the next line names the colourfile — the same two-line-per-rule format the upstream file uses:

```
# ping command
^([/\w\.\.]+\w)?(io|o|n|h|arp|l2)?ping[236]?\b
conf.ping

# gcc command
^([/\w\.\.]+\w)?(g?cc|[gc]\+|\+)\s
conf.gcc
```

The bundled `vendor/grc/grc.conf` carries 78 such regexp rules mapping onto 71 distinct colourfiles (some commands share a colourfile — `make` and `gcc` both point at `conf.gcc`, for instance). The leading `^([/\w\.\.]+\w)?` fragment recurs across nearly every rule: it lets an absolute or relative path prefix (`/usr/bin/ping`, `./configure`) match the same rule as the bare command name, which is why the launcher matches on the joined command line rather than just `argv[0]`.

Regexp dialect is bridged by `translate_regex` (`src/grcis.rs:220`). Python's `re` has no `</>` word-boundary form — those are literal `</>` characters — but `fancy_regex` reads them as GNU-style boundaries. The translator rewrites `<` and `>` to literals and leaves every other escape untouched, so configs authored for `grc` match the same strings under `fancy_regex`. This is the one place where a naïve reimplementer would silently diverge, and it is handled identically in both binaries — the launcher and the colouriser each carry their own copy of `translate_regex`, deliberately duplicated so neither can drift from the other.

Process orchestration is in `run_coloured` (`src/grcis.rs:303`). The wrapper ignores `SIGINT` (`libc::signal(SIGINT, SIG_IGN)`) so Ctrl-C reaches the child rather than killing `grc` before it can reap

the child and report its status — `grc` still waits on the child and propagates its exit code. The stdout/stderr redirection flags follow `grc`'s rule exactly: stdout is redirected by default; `-e` alone switches to stderr-only; adding `-s` re-enables stdout even alongside `-e` (`src/grcrs.rs:268`). When stdout is redirected, the child's stdout is piped into a spawned `grcat <cfile>`; when stderr is redirected, a second `grcat` reads the child's stderr and writes to a `dup` of fd 2. The `grcat` binary is located next to the running `grc` executable if a sibling `grcat` exists, otherwise it falls back to `grcat` on `PATH` (`src/grcrs.rs:248`) — so a locally built pair works without installation. After spawning, the launcher waits on the child first and only then reaps the one or two `grcat` children it started, returning the child's exit code (falling back to 0 when the status carries no code, e.g. signal death), so the visible exit status is the wrapped command's, not the colouriser's.

The `--pty` mode is marked experimental. It allocates a pseudo-terminal with `libc::openpty` (`src/grcrs.rs:390`), hands the slave end to the child's stdout so the command believes it is writing to a tty (and emits tty-style output), and feeds the master end into `grcat` as stdin. This exists for commands that suppress colour or change formatting when they detect a pipe. When `openpty` fails, the code degrades gracefully to an ordinary pipe rather than aborting.

Error output follows `grc`'s terse format: failures print `grc: <command>: <reason>` to stderr, where `<reason>` is the bare OS error string pulled through `libc::strerror` (`src/grcrs.rs:410`) to match the reference `%s: %s` formatting rather than Rust's default `io::Error` display.

The colouriser: `src/grcatrs.rs`

The `grcat` binary is built from `src/grcatrs.rs`. It reads one colourfile argument, resolves it through a search path, parses it into an ordered list of patterns, then reads stdin line by line and applies colours.

The colourfile search path in `conf_search_path` (`src/grcatrs.rs:231`) is: the empty prefix (so an absolute or relative name passed by `grc -c` is honoured as-is), then `$XDG_DATA_HOME/grc/`, `$XDG_CONFIG_HOME/grc/`, `~/.grc/`, `/usr/local/share/grc/`, `/usr/share/grc/`, and `/opt/homebrew/share/grc/`. The first existing non-directory file wins. `$XDG_DATA_HOME` defaults to `~/.local/share` and `$XDG_CONFIG_HOME` to `~/.config`. A name that resolves nowhere prints `config file [name] not found` to stderr and exits 1.

Each parsed block becomes a `Pattern` (`src/grcatrs.rs:18`): the compiled `Regex`, an optional per-group `colours` vector, the `count` directive string, and the optional `command`, `skip`, `replace`, and `concat` side effects — plus the three `grcrs`-extension fields (`trend`, `key`, `metric`) discussed later. A colourfile is a sequence of these blocks parsed by `parse_config` (`src/grcatrs.rs:261`). A block is a run of `keyword=value` lines; a line starting with `#` or blank is skipped inside a block, and a line whose first character is *not* an ASCII letter ends the block — which is why the `=====` separators in the shipped configs work as block delimiters. The recognised keywords are held in a ten-entry `KEYWORDS` array (`src/grcatrs.rs:265`): `regex`, `colours`, `count`, `command`, `skip`, `replace`, `concat`, plus the extensions `trend`, `key`, and `metric`. US spellings `color`/`colors` and the singular `colour` are all aliased to `colours` (`src/grcatrs.rs:300`), and an unrecognised keyword is a hard parse error. The `count` directive defaults to `more`. A block with no `regex` contributes no pattern.

The `colours` value is comma-split into one entry per capture group; within a group, whitespace-separated tokens are resolved and concatenated, so `bold red` composes the bold and red escapes into one string. Token resolution is `get_colour` (`src/grcatrs.rs:132`): a named token maps through a static ANSI table (`colour_table`, `src/grcatrs.rs:73`) that carries `none`, `default`, `bold`, `underline`, `blink`, `reverse`, `concealed`, the eight base foregrounds and `on_*` backgrounds, `beep`, the sentinel tokens `previous` and `unchanged`, the non-standard `dark/italic/rapidblink/strikethrough`, and the aixterm `bright_*/on_bright_*` codes. The bright codes are emitted with a standard-code prefix (`bright_red` → `\x1b[31;91m`) so a terminal without aixterm support degrades to the base colour instead of showing nothing. A token that is not in the table but is a `"..."` quoted literal is unescaped as a Python string literal would be — `unescape` (`src/grcatrs.rs:144`) decodes octal (`\033`), hex (`\xNN`), and the common single-char escapes, so configs that write `colours="\033[38;5;208m"` for 256-colour codes work unchanged. The octal decoder stops after

three digits and the hex decoder after two, matching Python's literal rules exactly — pinned by `unescape_octal_stops_after_three_digits` and `unescape_hex_caps_at_two_digits_and_bell`.

Here is a shipped colourfile — `vendor/grc/colourfiles/conf.df` — showing the block structure, the `=====` separators, and per-group colours:

```
# F5
regexp=^(?!Filesystem)(\[\/[-\w\d.]+\])+\\s
colours=blue,bold blue
=====
# Size 'G'
regexp=\\s\\d*[.,]?\\dGi?\\s|\\b\\d{7,9}\\b
colours=red
=====
# Use 90-97%
regexp=\\s9[0-7]%\\s
colours=red
=====
# Use 98-100%
regexp=\\s9[89]%|100%\\s
colours=bold red
```

The `regexp` sources exercise features that force the choice of `fancy-regex` over the standard `regex` crate: the negative lookahead `(?!Filesystem)` and the lookaheads used elsewhere in the config set are not expressible in `regex`, which is why `fancy-regex` (a backtracking engine) is the dependency.

The main loop

The main loop (`src/grcat.rs:604`) reads with `read_until(b'\n')` into a raw byte buffer, strips a single trailing `\n\r`, and decodes leniently through `decode_units` (`src/grcat.rs:478`). This is the port's reproduction of Python's `surrogateescape`: valid UTF-8 decodes 1:1, and each *undecodable* byte becomes exactly one `U+FFFD` unit while its original byte is preserved in a parallel `orig` vector. One replacement char per bad byte keeps character indices aligned with what the Python reference produces, and keeping the source bytes means output can re-emit the invalid byte unchanged rather than the lossy replacement character. The net effect: colourising never corrupts non-UTF-8 command output, and this is pinned by dedicated tests (`invalid_utf8_bytes_round_trip_unchanged`, `invalid_utf8_byte_inside_coloured_span_round_trips` in `tests/cli.rs`, plus the unit tests `decode_units_maps_each_invalid_byte_to_one_roundtrip_unit` and `decode_units_splits_multibyte_invalid_run_per_byte`).

For each line, every pattern is tried in order. Matching uses `captures_from_pos` so a pattern can match repeatedly across a line, driven by the `count` directive. The directive semantics are reproduced from `grcat`:

- **more** (default) — colour every match on the line, advancing past each; a zero-width match advances one char boundary to avoid an infinite loop. The reference Python `grcat` loops forever on a whole-line `.*` under `more`; the port's boundary advance is what lets `zero_width_whole_line_pattern_terminates` pass.
- **once** — colour only the first match, then jump to end of line.
- **stop** — colour this match and stop trying further patterns on this line.
- **block / unblock** — set or clear a block-colour flag that carries across subsequent lines; while `blockflag` is set, whole lines are painted `blockcolour` regardless of per-line matches. A `block` match is internally converted to a `stop` so no later pattern runs on the block-start line (`src/grcat.rs:662`), which is why `count_block_halts_subsequent_patterns` expects a following `skip=yes` pattern *not* to suppress the line.
- **previous** — resolve to whatever the prior pattern's count was.

Colours are laid down into a per-character `cline` array of escape strings (`src/grcat.rs:752`), which is where the two carried-across-lines sentinels are honoured: `unchanged` leaves a group's characters untouched,

and `previous/prev` reuses the colour from the previous line's matching group via the `prevcolour` state variable. Group spans are accumulated first into a `clist` of `(char_start, char_end, colour)` triples by `add2list` (`src/grcatrs.rs:519`); groups beyond the colour list reuse colour 0, and non-participating groups are skipped. The final line is assembled in `nline` (`src/grcatrs.rs:785`) by emitting a colour escape only when it changes from the previous character's escape (run-length collapsing the ANSI codes), then the character's *original* bytes from `orig` — except after a `replace` rewrite, where `orig` no longer aligns with the mutated line and the decoded chars are used instead. A trailing `default` reset and newline close each line. `EPIPE` (downstream closed) breaks the loop cleanly, and `grcat` also ignores `SIGINT` itself so the launcher's forwarding scheme is the single owner of Ctrl-C.

The per-block side effects are also reproduced: `command` runs a shell command via `sh -c` on a match; `skip=yes|1|true` suppresses the line from output; `concat` appends the matched line to a file; and `replace` rewrites the line, with Python `\N` backrefs converted to the `{N}` form `fancy-regex` expects (`convert_backrefs`, `src/grcatrs.rs:429`), doubling any literal `$` to `$$` so it is not misread as a group reference. When a block carries a side effect but no `colours`, the inner loop breaks after applying it once, matching `grcat`'s single-shot semantics for pure side-effect blocks.

Streaming trend colour (grcrs extension)

The one place `grcrs` goes past the upstream grammar is a directive family for colouring a number by its *motion* over the stream rather than by which regexp matched it. Three keywords participate — `trend`, `key`, and `metric` (`src/grcatrs.rs:18` fields, parsed at `src/grcatrs.rs:332`). `trend=` is a comma-separated list of `direction:colour` pairs where the directions are `rising`, `falling`, and `steady`; each resolves through the same `get_colour` token machinery into a `TrendSpec` (`src/grcatrs.rs:39`). `key=` names the capture group whose text keys a per-key last-value map, and `metric=` names the capture group whose text is parsed as an `f64`. On each match, `trend_colour` (`src/grcatrs.rs:48`) compares the parsed value against the last value seen for that key and recolours the metric group green/red/dim (or whatever the spec says) by the sign of the delta, then stores the new value. The first observation of a key and any repeated equal value both count as `steady`; a group that does not parse as a number is left with its ordinary `colours` entry. The per-pattern map is composited into a single key of the form `pattern_idx \0 group_text` so two different patterns cannot alias one another's series.

```
regexp=(\w+)\s+(\d+)
colours=default,default
trend=rising:green,falling:red,steady:dark
key=1
metric=2
```

Under this block, group 1 (a name) keys the map and group 2 (a number) is recoloured by its own trend under that name, so each name's series climbs and falls independently — verified at the unit level: `parse_config_trend_key_metric` confirms the spec resolves to `rising=\x1b[32m`, `falling=\x1b[31m`, `steady=\x1b[2m`, and `trend_colour_is_isolated_per_key` confirms the per-key isolation. The extension is strictly additive: a colourfile that never mentions `trend/key/metric` takes the plain `add2list` path (`src/grcatrs.rs:690`) and behaves exactly as upstream `grcat`.

Worked examples

The end-to-end tests double as the clearest specification, because each asserts the exact ANSI byte stream. Basic single-match wrapping, with the `\x1b[0m` reset that `grcat` emits ahead of every colour run so previous state is cleared:

```
config: regexp=foo\n colours=red
input:  a foo b
output: \x1b[0ma \x1b[0m\x1b[31mfoo\x1b[0m b\x1b[0m
```

A lookahead — the feature that forces `fancy-regex` — colouring a number only when `Mar` follows it:

```
config: regexp=\d+(?=\sMar)\ncolours=green
input:  size 344 Mar
output: \x1b[0msize \x1b[0m\x1b[32m344\x1b[0m Mar\x1b[0m
```

`count=block/unblock` carrying colour state across lines, from `block_state_carries_across_lines`:

```
before      -> \x1b[0mbefore\x1b[0m
START here  -> \x1b[3m\x1b[30mSTART here\x1b[0m
inside block -> \x1b[3m\x1b[30minside block\x1b[0m
END now     -> \x1b[0m\x1b[0mEND\x1b[0m now\x1b[0m
after      -> \x1b[0mafter\x1b[0m
```

And the invalid-byte round-trip, where the raw `0xff` survives inside a coloured span rather than becoming a replacement character (`invalid_utf8_byte_inside_coloured_span_round_trips`, `regexp=o.n` over `no\xffne`):

```
output bytes: \x1b[0mn\x1b[0m\x1b[31mo\xffn\x1b[0me\x1b[0m
```

Running the full launcher, an absolute `-c` config skips the `grc.conf` scan so the test needs no system install:

```
grc -c "$CONF" --colour=on sh -c "printf 'a foo b\n'"
# -> \x1b[0ma \x1b[0m\x1b[31mfoo\x1b[0m b\x1b[0m
```

Man pages, config data, and packaging

The runtime data — `grc.conf` and the colourfiles — is not bundled into the binaries; it lives in the `vendor/grc` submodule and must be installed on one of the search paths. Section-1 man pages for both binaries ship in `man/grc.1` and `man/grcat.1`; the `grcat.1` page documents all ten colourfile keywords, including the `trend/key/metric` extension flagged there explicitly as “grcrs extension” and “not present in upstream grc”. The pages can be read without installing (`man ./man/grc.1`).

The release pipeline (`.github/workflows/release.yml`) fires on `v*` tags and builds a six-target matrix: `aarch64/x86_64 apple-darwin`, `x86_64/aarch64 unknown-linux-gnu`, and the two static `unknown-linux-musl` variants (the musl targets are marked `allow_failure` so a musl toolchain hiccup does not block the release). Each target’s package step stages the two stripped binaries alongside `grc.conf` and every `colourfiles/conf.*` into one tarball, so the archive carries the runtime data the binaries expect to find at their search paths. A `publish` job creates the GitHub release from the artifacts, and a `homebrew` job computes per-target SHA-256 sums and regenerates the `Grcrs` formula in the `MenkeTechnologies/homebrew-menketech` tap — emitting `on_macos/on_linux` blocks only for the arches that actually built, referencing the musl tarballs as trailing comments (Homebrew has no libc gate), and installing `grc/grcat` to `bin`, `grc.conf` to `etc`, and the colourfiles to `share/grc`. That matches the `/opt/homebrew/...` entries baked into the search paths, so `brew install menketech/menketech/grcrs` yields a working install without manual data placement. In the wider stack, `grc` is the front for the shell’s command colourising, invoked either directly or through the aliasing shims the upstream project ships (`grc.zsh`, `grc.sh`, `grc.fish`) that wrap common commands.

Status and parity

Parity against upstream is enforced by the test suite, currently **71 tests** total (counted by `#[test]` attributes): 17 launcher unit tests in `src/grcrs.rs` (option parsing, abbreviated long options, regexp translation, `grc.conf` matching and precedence), 27 colouriser unit tests in `src/grcatrs.rs` (colour table, string unescape, back-ref conversion, config-block parsing, `surrogateescape` decoding, and the streaming-trend logic), and 27 end-to-end tests in `tests/cli.rs` that spawn the built binaries and assert on the exact coloured bytes. The end-to-end tests cover the observable behaviours directly: `count=stop/once/more/block`, `unchanged` and `previous` colours, `replace` then colour, quoted-literal colours, `\</\>` literals, `skip`, `concat`, CRLF

stripping, multibyte char spans, stderr redirection, exit-status propagation, and the invalid-UTF-8 round-trip cases. CI ([.github/workflows/ci.yml](#)) runs four jobs: `cargo fmt --all --check`, `cargo clippy --all-targets --locked -- -D warnings`, `cargo doc --locked --no-deps` under `RUSTDOCFLAGS=-D warnings`, and a build+test matrix on `ubuntu-latest` and `macos-latest` that finishes with the `grc --version | grep -q Colouriser` smoke test.

The licence is **MIT**, stated consistently across the source tree: `Cargo.toml` declares `license = "MIT"`, the README's badge and licence section both read MIT, the `LICENSE` file carries the MIT text, and the Homebrew formula generated by `release.yml` writes `license "MIT"`. `grcrs` is original Rust code — it reimplements `grc`'s behaviour and its config-rule model rather than deriving from `grc`'s source, and the upstream tree kept under `vendor/grc/` is a read-only reference for parity work, not what the binaries are built from.

zpwrcode-host and zwire-host — the browser-adjacent command line

The two subjects of this final chapter sit at an awkward border. They are launched, most of the time, by a web browser, and they exist because a browser extension needs a way to reach the local machine. That lineage tempts you to file them under “browser tooling” and move on. But a browser extension is a bundle of manifest JSON and service-worker JavaScript that Chrome loads; it has no binary you build, no `--help`, no prompt, and no life outside the browser process. The two things this chapter covers — `zpwrcode-host` and `zwire-host` — are Rust programs. You compile them with `cargo`, they carry `--help` and `--version`, they run on macOS, Linux, and (for `zwire-host`) Windows, and at least one of their run modes has nothing to do with a browser at all. They belong in a book about the command line because that is what they are: command-line binaries that happen to also speak the browser’s native-messaging dialect. The extension is the client; these are the tools.

Native messaging, in one paragraph

Chrome’s native-messaging mechanism is the reason both binaries exist. An extension declares the `nativeMessaging` permission, and the browser is told — through a small JSON manifest installed in a per-browser directory — that a given host name maps to an executable on disk with a fixed set of allowed extension origins. When the extension calls `chrome.runtime.sendMessage` or `chrome.runtime.connectNative`, Chrome spawns that executable and talks to it over stdin/stdout using `u32`-length-prefixed JSON frames. Neither of these binaries invented that protocol, and the chapter does not claim they did; the engineering is what each does on top of it. `zpwrcode-host` ports a specific upstream protocol into Rust and adds actions to it. `zwire-host` takes the single stdio transport Chrome gives you and unifies it with two more (a local socket and a client), so the same capabilities are reachable whether the caller is a browser or a shell script.

zwire-host — one dispatcher over three transports

`zwire-host` is v0.3.14, MIT-licensed (`zwire-host/Cargo.toml:3-8`). One crate produces two artifacts: a `[[bin]]` named `zwire-host` and a `[[lib]]` named `zwire_host` (`zwire-host/Cargo.toml:15-21`). The binary is a five-line entry point — `main` collects args and calls `zwire_host::run` (`zwire-host/src/main.rs`) — so that every other Rust program can embed the exact same host by pulling in the library. The docstring is blunt about the scope creep that gave the crate its name: it began as “the Chrome native-messaging host for the zwire HUD” and became “a universal local endpoint” reachable “from a browser extension *and* from tmux, emacs, desktop apps, plugins, and any language” (`zwire-host/src/lib.rs:4-12`).

The three run modes are selected by the first positional argument in `run` (`zwire-host/src/lib.rs`):

```
# 1. native messaging on stdio - no subcommand, what Chrome spawns
zwire-host

# 2. the NDJSON socket daemon
zwire-host serve

# 3. a one-shot client against that daemon
zwire-host call '{"cmd":"hostinfo"}'
```

Modes 2 and 3 are ordinary CLI usage with no browser in sight. The daemon listens on a Unix domain socket on macOS/Linux and a named pipe on Windows (`zwire-host/src/transport.rs`), speaking newline-delimited JSON — “the lingua franca every tool already has” (`zwire-host/src/lib.rs:16-18`). The socket path defaults to a runtime dir, never the persistent state dir, and is overridable with `$ZWIRE_HOST_SOCKET`; the code walks `$XDG_RUNTIME_DIR`, then macOS `$TMPDIR`, then a per-user name in `/tmp`, binding it `0600` because “the socket exposes `exec/fs/pty`, so no other local user may connect” (`zwire-host/src/lib.rs`, `zwire-host/src/transport.rs`). A client is trivial to drive from a shell:

```
zwire-host serve &
zwire-host call '{"cmd":"fs_walk","path":"/src","ext":"rs"}'
echo '{"cmd":"exec","program":"git","args":["status"]}' \
| zwire-host call
zwire-host call --stream '{"cmd":"sysinfo_start"}'
```

The last line shows the streaming variant: `--stream` relays every reply frame instead of just the first, which is how `sysinfo` and `PTY` output flow back (`zwire-host/src/lib.rs`, `help body`). Beyond the local socket, `serve` can also open a TCP listener guarded by a shared `--token` and dial named `--peer` hosts, federating the bus across machines (`zwire-host/src/transport.rs`, `ServeConfig`) — but the token gate exists precisely because local socket clients are trusted and TCP ones are not.

Every transport feeds the same dispatcher. A `Session` is per-connection state, and `Session::handle` takes one decoded request plus the connection’s output sink; “both the Chrome stdio loop and the socket daemon drive it the same way, so every capability is reachable from every client with no per-transport code” (`zwire-host/src/session.rs:1-7`). The capability set is advertised in the `hello` reply so a client can feature-test, and it reflects which optional features were compiled in (`zwire-host/src/session.rs:14-43`):

```
hello kv fs exec jobs bus peer watch
open clipboard notify hostinfo scheme ui palette
sysinfo procs      (feature "sysinfo-caps")
pty                (feature "pty")
```

The command arms behind those tags include `kv_get/kv_set/kv_merge` for a namespaced key/value store, `fs_*` crawl and watch, `exec`, `job_*`, `ps/kill/which`, `pty_spawn/pty_write/pty_resize/pty_kill`, `sysinfo_start/sysinfo_once`, plus `open/clipboard_get/clipboard_set/notify/hostinfo` (`zwire-host/src/session.rs:188-538`). The optional capabilities are real Cargo features: `sysinfo-caps` pulls `sysinfo`, `pty` pulls `portable-pty`, and both are in the default set, but an embedder can drop them (`zwire-host/Cargo.toml:24-40`).

The pub/sub bus

The capability that turns the host from an RPC endpoint into a coordination hub is the bus (`zwire-host/src/bus.rs`). Any long-lived connection can `sub` to a topic and receive `{"ev":"pub",...}` frames whenever another connection `pubs` to the same topic. Because the broker is process-global, “a message published by the browser HUD reaches a subscribed `zmax`, `tmux` widget, or plugin — they never talk to each other directly, only through the host” (`zwire-host/src/bus.rs:1-9`). The daemon also publishes on well-known topics itself — `scheme`, `ui`, `palette`, `schemes` — so a new subscriber gets a snapshot on subscribe and every change afterward, which is live theme sync with no polling. Delivery is best-effort fan-out; publish collects target sinks under the broker lock and writes to them outside it, so IO never deadlocks a publisher that is also a subscriber (`zwire-host/src/bus.rs:79-95`).

zpwchrome-host — a browserpass port plus additive actions

`zpwchrome-host` is v0.10.2, MIT-licensed (`zpwchrome-host/Cargo.toml:1-6`). It is a Rust port of the Go program `browserpass-native`, the native host that the browserpass browser extension talks to for UNIX `pass` integration. The port is deliberate and disciplined: `src/ported/` is a 1:1 mirror of the upstream Go source, one Rust file per Go file, PascalCase identifiers preserved verbatim, every function carrying a `/// Port of <name>() from <go_file>:<line>` comment, drift caught by pinning tests (`zpwchrome-host/README.md:27-39`). It implements every action in browserpass-native's `PROTOCOL.md v3.1.2` — `configure`, `list`, `tree`, `fetch`, `save`, `delete`, `echo` — with error codes 10–32 pinned to the same integers and the version reported as `3.1.2` (`zpwchrome-host/README.md:11-25`). Because it is wire-compatible, it is a drop-in replacement for the Go binary and can be registered under the upstream host name for the stock browserpass extension. The shipped code is original MIT Rust; the lineage is credited, and that is the whole of the debt.

The original engineering lives in `src/extensions/`, which the README calls additive because the upstream extension never sends these action names, so wire compatibility is preserved (`zpwchrome-host/README.md:41-43`). Three additive families matter most:

- `otp` — shells `pass otp <entry>` with `PASSWORD_STORE_DIR` set to the matching store and returns the current TOTP code.
- `search` — host-side fuzzy plus substring scoring across every configured store, faster than shipping the whole tree to the client for `zfz`.
- `dl.*` — a segmented download manager, the largest addition, described below.

The dispatch order in `main` sends any `dl.`-prefixed action to the download manager first, then `otp/search`, then `run.spawn`, then `host.crawl/host.exec`, then `zcite.save`, and only then falls through to the ported browserpass switch (`zpwchrome-host/src/bin/zpwchrome_host.rs:362-421`). The message is parsed twice — once as a raw `Value` for the extension actions that take their own shape, once as the ported request struct — so the strict port stays free of extension fields (`zpwchrome-host/src/bin/zpwchrome_host.rs:316-320`).

The download manager's file-state model

`dl.add` does not download in the host process. It spawns a detached worker — `zpwchrome-host --dl-worker <gid>` — and the process model is a CLI subcommand of the same binary (`zpwchrome-host/src/bin/zpwchrome_host.rs:246-257`). State for each job lives in a JSON file at `$XDG_CACHE_HOME/zpwchrome/dl/gid_NNNNNN.json` (`zpwchrome-host/src/extensions/dl.rs:104-105`), and that file, not a long-lived connection, is the coordination medium. This matters because native messaging spawns a fresh host process per one-shot message; there is no persistent port for downloads. Every control action — `dl.pause`, `dl.resume`, `dl.cancel`, `dl.remove`, `dl.clear`, `dl.list` — reads or mutates flags in the state file, and the worker polls those flags between chunks. The `JobState` struct carries `gid`, a `status` string (`pending|active|paused|done|failed|cancelled`), `paused` and `cancelled` bools, a `paused_offset_ms` accumulator so a resumed job reports accurate elapsed time, and a `worker_pid` (`zpwchrome-host/src/extensions/dl.rs:50-83`). On `dl.resume`, the host checks `kill(worker_pid, 0)`: a live worker just needs the paused flag cleared, a dead PID means the service worker was suspended and a fresh worker must be spawned — self-healing without the extension knowing (`zpwchrome-host/README.md:74-83`). Segmentation kicks in only when the total is at least `MIN_SEGMENT_BYTES` (1 MiB), the server sends `Accept-Ranges`, and more than one segment is requested; otherwise it degrades to a single stream (`zpwchrome-host/src/extensions/dl.rs:39-44`, README download-manager section). Spawn isolation is load-bearing: `setsid()` plus closing every `fd >= 3` before `exec`, because otherwise the worker inherits Chrome's native-messaging stdout pipe and Chrome reports “Native host has exited” even on success (`zpwchrome-host/src/bin/zpwchrome_host.rs`, README worker-process-model). The HTTP client is `ureq` with `rustls`, chosen so there is no `aria2`, no system OpenSSL, and no Go toolchain at

runtime (zpwchrome-host/Cargo.toml, zpwchrome-host/README.md:9).

Embedding zwire-host as a library

The `host.crawl` and `host.exec` actions are where the two binaries meet. Rather than re-implement a recursive filesystem walk or a capture-stdout exec, `zpwchrome-host` depends on the `zwire-host` crate and calls `zwire_host::api::walk` and `zwire_host::api::exec` — “the same code path the `zwire-host` daemon runs” (`zpwchrome-host/src/extensions/host.rs:1-8`). The dependency is declared `default-features = false` (`zpwchrome-host/Cargo.toml`, dependency comment), which is why the feature split in `zwire-host` exists: the embedder wants only the light filesystem/exec half and none of the heavy `portable-pty` or `sysinfo` weight, verified with `cargo tree`. So one of these two CLI tools is literally built out of the other, at the library layer, with the transport and heavy capabilities compiled out.

The wiring that joins the extension to both hosts

The `zpwchrome` extension — manifest v3, an MV3 service worker, the client and not a subject here — opens two native-messaging channels, and that is why the two hosts belong in the same chapter. For `pass`, `OTP`, `search`, and `downloads` it uses one-shot `sendNativeMessage` to the host name `com.menketotechnologies.zpwchrome`, which resolves to `zpwchrome-host`; each call spawns a fresh host process with no long-lived port (`zpwchrome/background.js:3726-3789`). Separately, for theme sync, it opens a persistent `connectNative` port to `com.zwire.hud`, which resolves to `zwire-host`, and `subs` to the `scheme/ui/palette/schemes` topics on the bus; the host answers with a snapshot and then pushes every change from any app in the fleet (`zpwchrome/background.js:4287-4354`). That is the same pub/sub bus described above, consumed from JavaScript. If `zwire-host` is not present — the extension loaded in vanilla Chrome — the port disconnects and the extension backs off harmlessly, its own theme still working from storage (`zpwchrome/background.js:4333-4354`).

The manifest side of that wiring is what `--install` writes. `zpwchrome-host --install <ext-id>` drops `com.menketotechnologies.zpwchrome.json` into every Chromium-family config dir it finds, plus `zwire`’s own profile dir, with `allowed_origins` scoped to the calling extension so only it can spawn the host (`zpwchrome-host/src/bin/zpwchrome_host.rs:258-283, 513-634`). The written manifest is the standard native-messaging shape:

```
{
  "name": "com.menketotechnologies.zpwchrome",
  "description": "zpwchrome native host (BP protocol)",
  "path": "/Users/user/.cargo/bin/zpwchrome-host",
  "type": "stdio",
  "allowed_origins": [
    "chrome-extension://<ext-id>/"
  ]
}
```

Installation is `cargo install`, upgrade is `cargo install --force`, and the path baked into the manifest survives a Homebrew keg change because `stable_exe_path` rewrites a versioned Cellar path back onto the version-independent `opt` symlink — a keg path would die on the next `brew upgrade` and silently drop every download back to Chrome’s own downloader (`zpwchrome-host/src/bin/zpwchrome_host.rs:491-511, zpwchrome-host/README.md:103-132`). None of that — `cargo install`, `--install`, the `--help` banner, the keg-path repair — has an analogue in the extension. It is all command-line surface, which is the point of the distinction this chapter opened with.

Both binaries carry the same styled `--help` screen and a `--version/version` path: `zpwchrome-host --version` prints the browserpass protocol version (3.1.2) it speaks, distinct from its own crate version (`zpwchrome-host/src/bin/zpwchrome_host.rs:307-309, 454-456`), and `zwire-host version` prints its crate version (`zwire-host/src/lib.rs, run`). Test coverage is reported at 121 tests for `zpwchrome-host` including a 2 MiB segmented download against a local Range-capable HTTP server

([zpwchrome-host/README.md:179-188](#), count provisional — taken from the README, not verified by a build here). The version numbers above are read from each crate's [Cargo.toml](#) rather than confirmed against a compiled artifact, so treat them as the declared versions. What is not provisional is the shape of the thing: two Rust programs, three transports between them, one embedded inside the other, both reachable from a browser and one of them equally reachable from a shell.

Matching the upstream — the parser-compatibility layer, tool by tool

The shape-of-the-fleet chapter made the argument-parsing point in a paragraph: everyone matches the upstream parser, using `clap` where `clap` can express the grammar and refusing or pre-processing it where it cannot. That paragraph is true, but it hides the interesting part — the specific ways each parser is *wrong* for a modern derive-based CLI, and the specific machinery each tool grows to be wrong in exactly the same way the original was. This chapter reads all of it across the grain. It is the one place in the book that treats the eight tools' front ends as a single subject, because the front end is where a port either earns the word “drop-in” or forfeits it, and the failure mode — accepting a real upstream command line and doing something subtly different with it — is invisible until a fifteen-year-old script breaks.

Why `clap` is the wrong shape for a ported CLI

`clap` 4's derive parser is an excellent argument parser for a program designed around it. It expects options to be separated from their arguments (or joined with `=`), it expects short flags to cluster only when they are all boolean, it reserves `--help/--version`, and it errors on anything it does not recognize. Every one of those reasonable defaults is a liability for a tool whose grammar was frozen decades ago by a different parser. Isov's grammar is not `getopt-clean`; Nmap's short options are glued (`-sS`, `-PS80`); `htop` and `grc` were written against `getopt/getopt_long`, whose abbreviated-long-option and option-scanning rules `clap` does not reproduce; and every one of these tools has flags that a script passes and that must be *accepted and ignored* rather than rejected. So the fleet splits into three postures toward `clap`, and the split is not stylistic — it tracks how far each upstream grammar sits from what `clap` can express.

Refuse `clap` entirely (`htoprs`, `grcrs`). Two of the five reimplementations do not let `clap` near their argv, because their upstream parser is the specification and `clap` cannot reproduce it. `htoprs` calls the real `libc::getopt_long` symbol — the identical library routine `htop`'s own `getopt_long` fills — marshalling `argv` into a null-terminated C `char**` and building the same `option` array with `has_arg` 0/1/2, so the parse is not merely similar to `htop`'s, it *is* `htop`'s, produced by the same C code (`src/ported/commandline.rs`). `grcrs` hand-mirrors `getopt` in `parse_opts`, reproducing two subtleties `clap` gets wrong: abbreviated long options resolved through `resolve_long` (an exact name wins, a unique prefix resolves, an ambiguous prefix is refused — `--conf=name` → `--config`, `--col=off` → `--colour`, but `--std` rejected as ambiguous between `--stdout/--stderr`), and option scanning that stops at the first non-option token and treats everything after `--` as the command even when it looks like a flag (`src/grcrs.rs:42`, `:120`).

Pre-process, then hand `clap` a clean line (`lssofrs`, `nmaprs`). Two more keep `clap` downstream but run a hand-written normalizer *before* it, because their grammar is glued or `+-`prefixed rather than `getopt-abbreviated`. `lssofrs`'s `normalize_lsof_args` (`cli.rs:368`) splits `--clusters` `getopt-style`, expands the `+L/+d/+D` family, peels attached arguments (`-o9`, `-sTCP:LISTEN`, `+L1`), maps each Isov option to its `clap` equivalent or to nothing, and passes everything after `--` untouched. `nmaprs`'s `expand_nmap_style_argv` (`argv_expand.rs`) walks `argv` token by token and separates Nmap's compressed grammar — `-sS` → `--scan-type S`, `-Pn` → `--no-ping`, `-PS80,443` → `--ping-S 80,443`, `-T4` → `--timing 4`, stacked `-vv/-dd` collapsed to `--verbosity=N/--debug=N` by counting letters — with the match-arm ordering commented as load-bearing (`-s0` before the generic `-sX` arm, `-P0` before the generic `-P*` arm).

Use `clap` as designed (`iftoprs`, `storageshower`, `temprs`, `zcolorizer`). The remaining tools have grammars `clap` can express directly, so they lean on the derive parser and add only targeted overrides. `iftoprs`

and `zcolorizer` set `disable_help_flag/disable_version_flag` so their hand-drawn cyberpunk help screens render instead of clap's default (`iftop.rs:68`; `zcolorizer` captures `-h/--help` as an explicit field). `storageshower` uses clap's `overrides_with` to pair every visible flag with a hidden `--no-` inverse so the last one on the line wins and only one boolean survives to `apply_to` (`cli.rs:383`). `temprs` sets `allow_hyphen_values(true)` on almost every index-taking flag, which is the single setting that lets `-1`, `-2` arrive as *values* rather than being read as short flags — negative stack indexing works precisely because of it.

The accept-and-ignore contract

The subtlest obligation is the one that does nothing visible: accepting flags that have no meaning in the port so that scripts passing them keep running rather than erroring. `lsf.rs` is the clearest case and its normalizer is where the contract lives. Single-char options `-b -O -C -M -X -E -L` are consumed inside a cluster and dropped (`cli.rs:452`); `-z -Z -S -f` drop any attached argument and end the cluster (`:456`); `-A -k -m -D -e` are treated as taking a required argument so the following token is swallowed if none is attached (`:460`); on the `+` side, `+w +f +E +M +X +m` are silent toggles and `+e` consumes a separate argument (`:564`). A script that runs `lsf -b -O` (avoid kernel blocking, bypass overhead) runs under `lsf` without error even though neither flag has an implementation behind it. The book's own phrase for this is exact and worth keeping: the flags are *honored syntactically, not behaviorally*. `nmap.rs` does the same at a different layer — its `effective_*` accessors and the extended long-option set from `nmap.cc` (`--resolve-all`, `--defeat-rst-ratelimit`, `--nsock-engine`, `--release-memory`, and the rest) are accepted for parity and logged at `debug` via `trace_nmap_compat` rather than acted on, so an Nmap command line that leans on them parses and scans rather than aborting.

The reason this is the correct trade, and not a lie, is that the alternative is worse in exactly the way a port exists to avoid. A parser that rejects `-b` because the port has no kernel-blocking behavior forces every caller to audit and rewrite their invocations — which is the migration cost the drop-in replacement was supposed to eliminate. Accepting the flag and doing nothing is the honest behavior *provided the tool says so*, which every chapter does: `lsf`'s parity section names `-b/-O/-D` as syntactic no-ops, and `nmap.rs`'s capability matrix marks the accepted-but-unwired options.

Where the parsers deliberately diverge

Two divergences are intentional and both are about branding, not grammar. `htop.rs -h` and `htop.rs -V` short-circuit in a thin `src/main.rs` wrapper before the ported `getopt` path ever runs — `-V` calls the ported `printVersionFlag` (prints `htop 0.5.7`), `-h` calls `extensions::help::print_help` (the figlet-bannered house-style screen) — while the plain ported `printHelpFlag` is retained verbatim as the spec the styled screen tracks. `nmap.rs` intercepts clap's help/version errors and routes them into `help_tp::print_help/print_version` so the styled CYBERPUNK help appears instead of clap's plain renderer, and re-derives the binary basename from `argv[0]` so `nms` shows `nms` in its help. These are the only places the fleet's parsers do something the upstream did not, and in both cases the upstream behavior is still reachable underneath — the divergence is a presentation layer over an unchanged parse.

The one-token overload problem

The hardest single thing a ported parser does is disambiguate a flag whose meaning depends on what follows it, because that is where a token-by-token pre-processor and a grammar-driven parser disagree most. `lsf`'s `-s` is the canonical example and `normalize_lsof_args` handles all three readings: `-s` with no argument means “always show size,” `-s` with a colon spec (`-sTCP:LISTEN`) filters sockets by state, and `-s` followed by a non-colon rest is size-always with the cluster continuing (`cli.rs:479`). `-o` optionally takes a leading digit count for offset precision (`-o9` = nine digits), peeled character by character and re-emitted as `--offset-always` plus `--offset-digits` (`:465`). `+L` alone lists the NLINK column while `+L1` additionally selects files with link count below 1 — the unlinked-but-open case (`:521`). None of these is expressible as a clap argument type;

they exist only because a human wrote a state machine that reads the same bytes Isov's own option loop reads. The lesson the fleet encodes is that a compatible parser is not a compatible *option set* — it is a compatible *reading* of the exact byte sequence the upstream accepted, overloads and all, and the only way to get it right is to treat the original parser as the spec and reproduce its decisions rather than its intentions.

The output contract — TTY-awareness, ANSI, and the cyberpunk chrome

Every tool in this book is a filter in the Unix sense — `argv` and `stdin` in, bytes out — and the single convention that lets eight independently-authored programs compose into one pipeline is not a framework or a shared crate. It is an agreement about output: colour and interactive chrome are gated on whether the relevant descriptor is a terminal, and they vanish the moment output is piped. The shape-of-the-fleet chapter stated the rule; this chapter is the mechanism behind it, because the mechanism is where the rule is either honored precisely or broken silently, and a tool that leaks an escape code into a `jq` pipeline is not composable no matter how good its colours look on a terminal.

The `isatty` gate, eight ways

The gate is the same idea everywhere — ask the OS whether a descriptor is a TTY, and branch — but the tools reach for it through four different surfaces, and the differences are instructive.

- `grc.rs` initialises its colour default straight from the C call: `colour = libc::isatty(1) == 1` (`src/grc.rs:47`), and `--colour=auto` re-evaluates the same `isatty` check, so colour is on to a terminal and off to a pipe unless forced with `--colour=on/--colour=off`. It is the most literal reproduction because it is porting `grc`'s own `isatty`-driven default.
- `temprs` branches on the Rust `IsTerminal` trait at both ends of the pipeline: `input()` reads `stdin().is_terminal()` so `CMD | tp` reads the pipe while a bare `tp` at a TTY waits for typed input, and `output()` mirrors it with `stdout().is_terminal()`, choosing the boxed cyberpunk renderer for a terminal and a plain byte-faithful `print!` for a pipe. It then *memoizes* the `stdout` check in a `thread_local! Cell<Option<bool>>` (`is_tty()` in `utils.rs`) so the syscall is paid at most once per run rather than once per rendered chunk.
- `lsOfrs` gates the whole columnar-output styling on the check (`main.rs:57`): off a TTY it drops its colours and cyberpunk headers and prints plain, script-safe text, and — importantly — its live modes never enter the alternate screen non-interactively; they print one snapshot and exit. The `--color` flag forces `always/never/auto` over the auto-detected default.
- `zcolorizer` auto-disables colour off a TTY unless `-C/--force-color` forces it (the path for piping into `less -R`), and `--no-color` forces passthrough. `nmap.rs`'s styled help suppresses all escapes on a non-TTY or when `NO_COLOR` is set.

The `NO_COLOR` environment convention appears in `nmap.rs` explicitly, and the pattern across the fleet is the union of three signals: is the descriptor a TTY, did the user force colour on or off, and is `NO_COLOR` set. Any one of them can turn colour off; only an explicit force turns it on against a pipe.

Two output paths from one renderer

The tools that draw decorated output do not have a “colour” path and a separate “plain” path that can drift apart — they have one renderer that collapses. `temprs`'s `cyber_*` family in `utils.rs` is the clearest specimen. To a terminal, `cyber_print_content` frames a payload in a `┌ ... ┐` box, `cyber_hr` draws a cyan

rule, and indexed listings print `[NN]` markers in yellow with `@tag` names in a second colour. To a pipe, every one of those routines degrades to plain, byte-faithful text: `cyber_path` prints just the path, `cyber_hr` prints eighty dashes, `cyber_print_content` becomes a bare `print!`. That single-source split is what makes `tp -o 1 | jq` safe — the machine-consuming path is not a re-implementation that might forget an escape, it is the same function with the decoration compiled out by a branch. `grcrs`'s colouriser carries the analogous guarantee at the byte level: `grcat` emits a colour escape only when it changes from the previous character's escape (run-length collapsing the ANSI codes) and re-emits the character's *original* bytes, so an uncoloured token produces no escape at all and a non-UTF-8 byte survives round-trip.

Structured output for the pipeline that wants data, not colour

TTY-awareness handles the human-versus-pipe axis, but two tools go further and emit structured data specifically for downstream tooling, which is a different contract: not “plain text when piped” but “machine-parseable format on request.” `iftoprs --json` (`run_json_mode`, `main.rs:762`) stands up the same capture and proc-lookup threads, skips the TUI entirely, and emits one JSON object per line — NDJSON, one `JsonSnapshot` per refresh with an RFC-3339 timestamp, flow count, cumulative totals, peaks, and a `flows` array, with `process_name/pid` omitted when unknown via `skip_serializing_if`. `nmaprs` writes Nmap-compatible XML (`-oX`, an `<nmaprun>` root with the full element set) and grepable (`-o6`) output so existing XML- and grep-consuming tooling has a chance of parsing it. `lsofrs` adds `-J/--json` (serde array) and `--csv` (RFC 4180) alongside its lsof field-output mode (`-F`), and `temprs`'s `--path INDEX` exists purely to hand a backing file path to another program (`cat "${tp --path 1}"`). The pipeline stages the book demonstrates fall directly out of these:

```
sudo iftoprs --json | jq -c '.flows[] | {dst, sent_2s}'
kill "${lsf -t -c nginx}"           # bare PIDs feed kill
ps aux | tp -w procs                # push tagged onto the stack
tp -g python procs && echo hit      # grep the entry, compose in &&
journalctl -f | zcolorizer          # repaint a live stream
```

The cyberpunk chrome, and where it is not allowed to go

The house visual style — neon 256-colour palettes, `[ SYSTEM://DISK_MATRIX ]` and `NMAPRS // GRID SCANNER` chrome, figlet/ANSI-Shadow wordmarks with cyan-to-magenta truecolor gradients, NOMINAL/WARNING/CRITICAL badges — is real and shared, and the tools that draw help screens by hand (`nmaprs`'s `help_tp`, `zcolorizer`'s hand-drawn banner, `temprs`'s `CYBERPUNK_TEMPLATE`, `htoprs`'s figlet help, `storageshower`'s hand-formatted `--help`) all disable clap's default renderer to run it. But the chrome is bounded by the same TTY gate as everything else, and this is the discipline that keeps it from being a liability: `nmaprs`'s styled help suppresses every escape on a non-TTY, `zcolorizer`'s banner is only drawn on the interactive path, and every tool's error output drops to the terse `<tool>: <thing>: <reason>` shape on stderr regardless of chrome. `grcrs` even reproduces the upstream's error *formatting* specifically — `grc: <command>: <reason>` where `<reason>` is the bare OS string pulled through `libc::strerror` (`src/grcrs.rs:410`) to match the reference `%s: %s` rather than Rust's default `io::Error` display. The chrome lives on the human path and nowhere else; the moment a descriptor is a pipe, all eight tools become quiet, plain, script-safe filters, which is the only reason the aesthetic is affordable at all.

The theme engine – the 31-palette lineage and `shift_lighter`

The shape-of-the-fleet chapter called the shared theme set “a genuine shared component, not merely a shared aesthetic,” and traced its path: the 256-colour palette set originates in `storageshower`, is carried into `iftoprs`, `lsofrs`, and `htoprs`, and is inherited explicitly by `zcolorizer`. That is worth a chapter of its own, because it is the clearest example in the whole book of code — not just convention — moving between tools, and because the details of *how* thirty-odd palettes stay consistent across five independently-shipped binaries reveal a small, sharp piece of engineering that recurs verbatim: `shift_lighter`.

One palette set, five consumers, an enum-length invariant

The palette set is a fixed roster of named 256-colour schemes running from `NeonSprawl` through `Zaibatsu`. The count is not identical across the tools, and the differences are meaningful rather than sloppy:

Tool	Palettes	Notes
<code>storageshower</code>	30 <code>ColorMode</code> variants	the origin; display names “Neon Sprawl”...“Zaibatsu”
<code>iftoprs</code>	31	adds the parity palette <code>Iftopcolor</code>
<code>lsofrs</code>	31	<code>ThemeName::ALL.len() == 31</code> , asserted (<code>theme.rs:860</code>)
<code>htoprs</code>	31 six-colour palettes	applied at the <code>Ncurses::to_color</code> choke point
<code>zcolorizer</code>	31 generated + 1 hand-written	<code>ccze-classic</code> added for parity → 32 total

The important discipline here is not the count but the *assertion* of the count. `lsofrs` pins `ThemeName::ALL.len() == 31` with a compile-adjacent test, and `iftoprs` verifies `ThemeName::ALL.len() == 31` (`theme.rs:335`); `zcolorizer` confirms 32 at runtime via `--list-themes | wc -l`. A palette roster that is supposed to match across five tools is exactly the kind of thing that drifts silently — someone adds a scheme to one binary and forgets the others — so the tools that carry the set assert its length rather than trusting it, which is the same posture the parity chapters take toward test counts and the port gate takes toward function names.

The six-colour model

Every palette in the set is six 256-colour indices, and the six slots carry a fixed meaning that lets one palette drive an entire UI. `zcolorizer` names them most precisely — a `Palette` is a kebab id, a display name, and six indices `[c1..c6]` standing for *primary*, *brightest accent*, *secondary*, *mid*, *dim*, *darkest*. `storageshower`’s

custom-theme struct spells the same six as named channels (`blue`, `green`, `purple`, `light_purple`, `royal`, `dark_purple`), and its theme editor is a six-slot picker over the 256-colour palette seeded from the current effective palette via `palette_for_prefs`. `iftoprs`'s `CustomThemeColors` is six palette slots; `htoprs`'s are six-colour palettes; `lsotrs`'s theme editor builds custom six-colour palettes and writes them to `~/.lsotrs.conf`. Six is the shared width because it is the smallest number that gives a UI a primary, two accents, and a three-step dim ramp — enough to colour a bar, a header, a selected row, and a muted secondary without the palette becoming a full 16-colour theme the user has to design.

shift_lighter — the function that moves with the palettes

The single piece of code that most clearly *is* shared rather than merely resembling a shared idea is `shift_lighter`, an index-cube arithmetic helper that steps a 256-colour index one notch brighter, clamping at the cube and grayscale-ramp edges. The book records its migration explicitly: it “moves with the palettes from `iftoprs` into `zcolorizer`.” Its purpose is collision avoidance. When a palette assigns two logically-distinct tokens to the same slot — `zcolorizer`'s `process` versus `host`, both landing on the secondary colour — rendering them identically would lose information, so `theme_from_palette` derives an extra distinguishable shade by pushing one of them one step brighter through `shift_lighter` rather than by adding a seventh palette colour. The theme tests pin the outcome: `zcolorizer` asserts `process` and `host` never render identically across all 31 generated palettes, and that `error` is bold in every one. The elegance is that the palette stays six colours wide — the extra shades are *computed* from the six at render time, not stored — so a user editing six slots still controls the whole look, and the derived shades track their parents automatically.

Generated, not hand-authored

The reason a six-colour palette can style sixty tokens is that the mapping from palette to token-styling is a function, not a table. `zcolorizer`'s `theme_from_palette` is the fully-worked example: it assigns each token to a palette role and multiplies the six colours into enough distinct looks by layering bold, italic, and underline — `date` is bold-accent, `time` plain-accent, `host` bold-secondary, `ip` plain-secondary, `uri` underlined-secondary, and the severity tokens lean on the brightest accent so `error` stays loud in every palette. Thirty-one of `zcolorizer`'s thirty-two themes are generated this way from the ported palettes; only `ccze-classic` is hand-written, a 16-colour map approximating the original ccze look (cyan dates, bold-red errors, bold-yellow warnings) for parity and nostalgia. The same generate-don't-enumerate principle is why `storageshower` can ship 30 builtins while checking only the first ten into `themes/` as ready-to-copy TOML — the rest are computed from their `ColorMode` variant, and a `next()` method that indexes into `ColorMode::ALL` and wraps modulo its length drives in-app cycling with the display-name-uniqueness and clean-wrap asserted by tests.

Persistence, and forward-compatible config

A theme is only useful if it survives a restart, and the fleet's persistence story is uniform with two honest exceptions covered in the packaging chapter. The three TUI tools that keep user preferences auto-save the selected theme (and every other toggle) to a TOML dotfile on every in-app change with no explicit save step — `~/.lsotrs.conf`, `~/.iftoprs.conf`, `~/.storageshower.conf` — while `htoprs` stores JSON under `~/.config/htoprs/prefs.json` and `zcolorizer` uses TOML under XDG at `$XDG_CONFIG_HOME/zcolorizer/config.toml`. The load path is where the forward-compat guard lives: every persisted field carries `#[serde(default)]`, so a config written by an older build that is missing a newer key still loads, the missing field falling back to its default rather than erroring. `lsotrs`'s `load()` goes further and silently returns defaults on any parse error at all (`config.rs:88`), and its `save()` is compiled out under `#[cfg(test)]` so the test suite never touches the user's real config. For a file the tool rewrites itself on every toggle, tolerant loading is not laxness — it is the only behavior that lets the palette roster and the config schema evolve without stranding a user on a config their new binary refuses to read.

Talking to the kernel — procfs, libproc, pcap, raw packets, sysctl

The system tools in this book do not learn about the machine by running other programs. That is a single design value stated once in the shape-of-the-fleet chapter — the anti-subprocess discipline — and it is worth a chapter because it is where the ports do their least glamorous and most load-bearing work: reading the kernel directly, per operating system, by hand, from the same interfaces the C originals read. The payoff of doing it this way is a single self-contained binary with no runtime dependency on `lsof`, `ss`, `netstat`, or `diskutil` being installed and on `PATH`; the cost is that each tool carries a per-OS backend tree and hand-parses kernel data structures that a shell-out would have let it treat as opaque text. This chapter reads those backends across the grain.

The Linux path: hand-parsed procfs, no crate

Everywhere Linux appears in the fleet, the data comes from the `/proc` and `/sys` pseudo-file systems, parsed by hand against the files — and, notably, **none of the tools uses the `procfs` crate**. The parsing is done directly:

- `lsofrs` walks `/proc/<pid>/fd` for numbered descriptors and reads the `cwd`, `root`, and `exe` symlinks into the `cwd/rtd/txt` pseudo-descriptors `lsof` users expect, correlating sockets through a prebuilt socket map keyed by inode. Because `/proc/net/tcp` exposes queue occupancy but not the socket buffer limits, `send_buf_size/recv_buf_size` stay `None` on Linux — a gap the stall classifier degrades around rather than fabricating.
- `iftoprs` parses `/proc/net/tcp`, `/proc/net/tcp6`, `/proc/net/udp`, and `/proc/net/udp6` for the inode → port map, then walks `/proc/[pid]/fd/` for inode → PID and reads `/proc/[pid]/comm` for the process name (`procinfo.rs:127`) — the whole process-attribution table built without a subprocess, the `Cargo.toml` comment noting this compiled-in path replaces a runtime `lsof`.
- `storageshower` reads `/proc/mounts` for the mount table, `/proc/diskstats` for sector counts (mapping devices to mounts via `mount_to_device_map`), and `/sys/block/*/device/state` for SMART-style health.
- `htoprs` carries the full Linux backend `htop` carries (sixteen modules), reading the kernel through three pure-Rust, no-FFI crates rather than C libraries: `libmedium` for hwmon sensors, `neli` speaking generic-netlink for the TASKSTATS delay-accounting path (`linux/LibNL.c`), and `zbus` speaking D-Bus directly for the `SystemdMeter` (`linux/SystemdMeter.c`). Both `neli` and `zbus` speak their wire protocols directly over a socket rather than linking a C library — a deliberate choice to keep the build free of C dependencies and vendorable long-term.

The through-line is that `procfs` is a text (and symlink) interface, and the fleet treats it as one it parses itself. There is no shared `procfs` abstraction; each tool reads exactly the files it needs, which keeps the parsing auditable against the kernel's documented `/proc` format rather than against a crate's model of it.

The macOS path: direct libproc and sysctl FFI

macOS has no `/proc`, so the same data comes from `libproc` and `sysctl`, declared as raw FFI and called against `repr(C)` structs matched to the kernel headers:

- `lsoprof` declares `proc_listpids` and `proc_pidinfo` as raw `extern "C"` and calls them twice — first to size the PID buffer, then to fill it — with no shelling out and no parsing of another tool's output. Because macOS exposes socket send/rcv buffer *limits* alongside occupancy, the Darwin path fills the `send_buf_size/rcv_buf_size` fields that the stall classifier needs (which is why `--stall` uses a fill-ratio rule on macOS and a trend rule on Linux).
- `iftoprs` uses the `libproc` crate to read each process's open sockets via `pidfdinfo::<SocketFDInfo>`, selecting the correct arm of the `soi_proto` union by `soi_kind` (TCP vs datagram), converting the network-order `insi_lport` with a local `ntohs`, and skipping anything that is not TCP(6)/UDP(17) or has a zero local port (`procinfo.rs:47–124`).
- `storageshower` calls `getmntinfo(&mut mntbuf, MNT_NOWAIT)` through `libc`, walks the returned `statfs` array to compute `total/avail/used` with saturating subtraction, and reads IOKit's `IOBlockStorageDriver` byte counters for I/O rates; SMART status on macOS is the one place it does shell out (`diskutil info`, reading the `SMART Status` field), which the chapter names rather than hides.
- `htoprs` carries the Darwin backend (`darwinmachine.rs`, `darwinprocess.rs`, `darwinprocesstable.rs`) and uses `libc::sysctlbyname` with byte-string constants spelled out including their NUL (`b"kern.osrelease\0"`, one of the sixteen documented clippy-allow entries) so the literal matches the C source exactly.

The `repr(C)` struct-matching is the discipline that makes this safe: the FFI is not “call a syscall and hope,” it is a struct laid out to match the kernel header, filled by the kernel, and read field by field — the same contract the C original relies on, transcribed rather than reinvented.

The wire: pcap and raw packets, and graceful privilege degradation

Two tools go below the process table to the network itself, and they are the only two that need elevated privilege, so they are also the two that degrade most carefully when they do not have it.

`iftoprs` is the only tool that uses `pcap`. It opens the interface with a 256-byte snaplen — enough for L2–L4 headers, never payload, so it caps the copy — and decodes four real link types (`DLT_EN10MB` Ethernet, `DLT_NULL` BSD loopback, `DLT_LINUX_SLL` Linux cooked capture, `DLT_RAW` raw IP) rather than assuming Ethernet, pushing decoded packets across a `tokio::sync::mpsc` channel off the capture thread. Its capture loop is resilience-first: a device-open failure or a non-timeout pcap error is treated as transient, retried with exponential backoff from 250 ms capped at 30 s, resetting on any successful reopen, with `TimeoutExpired` treated as “continue” rather than an error and the receiver being dropped as the only clean exit.

`nmaprs` is the only tool that constructs raw packets, via `pnet/pnet_sys`, and the only one with a genuinely async core (`tokio` with `full`). It detects `geteuid()` and silently falls back from SYN to TCP-connect scan when unprivileged; `--privileged` forces raw mode and `--unprivileged` forces connect mode, rejecting raw-only techniques (`--s0`, `--sI`, SCTP) with an error rather than a wrong answer. Its raw paths run a dedicated rcv thread per address family, register the reply key in a `DashMap` *before* each send to avoid a rcv race, and shard work across up to 16 independent pipelines per family. Interestingly — a point the connect-scan chapter makes — `nmaprs`'s *default* connect scan does not use async sockets at all but a worker pool of OS threads running blocking `connect_timeout`, because the blocking probe is three syscalls (`socket + connect + close`) where the async path adds an `ioctl(FIONBIO)` and a reactor registration, and the lock-free `UnsafeCell` result array lets each worker write its own slot exactly once with no contention.

Both privileged tools share the same posture: capability is detected, not assumed, and absence of privilege produces a lesser-but-correct result (connect scan instead of SYN, retry instead of crash) rather than an error or a fabricated answer.

Bounded memory and poison tolerance

Two properties recur across the kernel-facing backends because reading a live kernel means reading an unbounded, mutating source. The first is a fixed memory ceiling: `iftoprs`'s accounting model is a 40-slot ring buffer per flow (two `VecDeque<u64>`, sent and received, capped at `HISTORY_SLOTS = 40`) with 60-second idle eviction, so history cannot grow with connection lifetime and idle flows age out of the table; `storageshower`'s directory sizer walks with `read_dir` and `saturating_add` and refuses to recurse through symlinks (a symlink counts as its own `symlink_metadata().len()`), which is what stops a cycle from driving runaway — pinned by `scan_directory_symlink_cycle_completes_with_finite_sizes`. The second is poison tolerance: `iftoprs`'s shared `FlowTracker` locks its mutex everywhere with `lock().unwrap_or_else(|e| e.into_inner())` so a panic in one thread cannot wedge the tracker. Both properties exist because the data source never stops changing, and a tool that reads it on a background thread and renders it on another has to bound what it keeps and survive a partial failure on either side — which is exactly the shape every one of these backends takes.

Measuring the fleet — benchmark and test methodology, honestly

Every per-tool chapter in this book reports numbers — versions, test counts, benchmark ratios — and every one of them marks those numbers as provisional. That is not hedging for its own sake. It is a methodology, stated once here so the per-tool chapters can keep repeating the caveat without re-deriving it, and it is worth reading as a subject because the way the fleet handles its own numbers is the clearest evidence of the honesty the book keeps claiming. A book that printed “1,468 tests, all passing” and “5–21× faster than Isof” as flat facts would be lying in two distinct ways, and the tools’ own discipline names both.

What “provisional” means, precisely

A test count in this book is, almost everywhere, a count of `#[test]` (and `#[tokio::test]`) *attributes* found by `grep` over the source tree — not the `test result:` line of a `cargo test` run. Those are different quantities, and the gap between them is real. A `grep` of attributes counts intentions: it includes `#[ignore]`d tests, it includes helper tests, it cannot tell a passing test from a failing one, and in at least one case it counts a generation. `temprs`’s ~5,059 unit tests are dominated by 4,449 *generated* flag-recognition tests in `opts.rs` — a machine-emitted battery, not 4,449 hand-written cases — which is why the chapter states the breakdown rather than the headline. The correct reading of every test number in this book is therefore “count of test *attributes* present in the source, as of the read, treat as inventory not as a green-suite claim; the authoritative figure is the `test result:` line of `cargo test --locked`.” The one place the book verified counts in-session is `storageshower` (3,101 in `tests/` + 596 in `src/`), and even there the note is explicit that the full suite was not compiled and run, so the *passing* count is unverified.

The consolidated table of these numbers lives in Appendix C. What matters here is the rule that produced them: a number goes in the book with the tool that generated it named (`grep`, `wc -l`, runtime enumeration, or a `cargo test` run) so a reader can tell an intention from a result.

Benchmark numbers are the author’s, on one machine

The benchmark claims are held to the same standard and it is a strict one. Three tools report head-to-head figures, and all three are marked as the project’s own measurements on one machine and configuration, not reproduced for the book:

- `lsufrs` — the README publishes hyperfine benchmarks claiming a 5–21× advantage over `Isof 4.91` and `Isofng` on macOS, on a machine with roughly 470 processes and 5,000 open files (consistent with the 431/4,032 snapshot the chapter captures). Not reproduced here; read as the author’s reported figures.
- `nmaprs` — hyperfine comparisons against `nmap 7.99` on macOS arm64, all against `127.0.0.1` with `-n -Pn --min-rtt-timeout 50ms --max-retries 0` to `/dev/null`. The headline is 1.5–5.1× faster across port counts, with the largest gains (3–5×) on small scans where Nmap’s startup cost dominates and narrowing to 1.5–1.6× on full 65536-port sweeps where probe work dominates. A Criterion microbenchmark of a three-closed-port localhost connect scan reports ~74–82 µs.
- `storageshower` — the README reports formatting operations in the tens of nanoseconds and full disk-list rendering for 200 mounts around 15.7 µs on Apple Silicon.

The uniform posture — “these are the author’s own measurements on one machine; treat them as directional rather than as guaranteed cross-platform figures” — is not false modesty. A benchmark ratio is a measurement of two binaries on one machine under one workload, and reprinting it as a property of the tool (“lsofrs is 5–21× faster than lsof”) drops every qualifier that makes it true. The book’s rule is to keep the qualifiers attached: which upstream version, which machine, which workload, and who measured it. Where the chapter can explain *why* a number is plausible it does — nmaprs’s small-scan advantage is attributed to Nmap’s startup cost (Lua/NSE init, libpcap, database loads) that a Rust binary with none of that pays once and cheaply — but the explanation is a mechanism, not a re-measurement, and the chapter says so.

The parity spectrum: only one tool enforces it at build time

The most over-statable thing in the whole book is “the fleet’s parity discipline,” because it is tempting to describe it as a shared harness and it is not one. Parity varies enormously across the tools, and reading it as a single thing would paper over the difference between a build gate and an aspiration. The honest spectrum, strongest to weakest:

1. **htoprs — a build gate plus a byte-diff oracle.** The port-purity gate in `build.rs` runs on every `cargo build/test/check`: it extracts every free `fn` name under `src/ported/` and checks it against a 4,132-entry snapshot of htop’s real C function names, and a function with no upstream counterpart *fails the build*. It fires even under `cargo test --test X` so a reimplementer cannot route around it, the allowlist for genuine architectural helpers holds zero entries, and the brace-counting collector is comment-, string-, and raw-string-aware so it cannot be tricked by a “{” in a test. On top sits `tests/parity/cli_parity.rs` diffs htoprs against the real C htop byte-for-byte (modulo the deliberate rebrand via `canon()`), and `xutils_parity.rs` compiles a C harness against htop’s *genuine* `XUtils.c` and diffs the Rust port against that compiled oracle function by function. Both skip gracefully when no htop 3.5.x or C compiler is present, so CI stays green. This is the strictest apparatus in the book by a wide margin.
2. **grcrs — a byte-for-byte oracle without a build gate.** Its 27 end-to-end tests in `tests/cli.rs` spawn the built binaries and assert the exact coloured byte stream, with expectations cross-checked against the reference Python `grcat` — covering `count=stop/once/more/block`, `unchanged/previous` colours, `replace-then-colour`, `\</\>` literals, CRLF stripping, and the invalid-UTF-8 round-trips that pin its `surrogateescape` reproduction.
3. **ztmux (Part I) — a parity suite plus an anti-drift gate.** Reported at 1107/1107 cases passing against the vendored tmux, with a `tests/ported_fn_names_match_c.rs` gate that fails the build on a free `fn` with no tmux C counterpart. It earned the number by root-causing real divergences (a `#{l:...}` crash from a dropped pointer increment, layout rounding, a macOS `#{pane_current_command}` case).
4. **nmaprs, zcolorizer — documented parity, contract-tested.** `nmaprs` documents parity as a capability matrix (which port-state each scan technique produces) backed by contract tests over the port database, scanflags, OS-DB parsing, and target expansion. `zcolorizer` keeps ccze’s C source vendored as reference and pins its precedence model (`earlier_rule_wins_overlap`) and sanitization behaviour with engine tests — but it colours formats ccze never saw, so byte-parity is not even the goal for most of its surface.
5. **lsofrs, iftoprs — reproduce the grammar and vocabulary, no running-upstream diff.** `lsofrs` reproduces lsof’s option grammar, type vocabulary, and output formats and asserts them with its own tests but does not diff against a running `lsof`. `iftoprs` ships a parity palette and mirrors iftop’s keybinds and flow-table model, but its 2s/10s/40s columns are *summed byte counts over trailing one-second slots scaled to bits*, not slot-averaged bytes/sec — a difference the chapter states precisely rather than calling it identical.
6. **storageshower, tempis — originals, so nothing to be parity-tested against.** Their large suites pin *their own* invariants — symlink-cycle termination in `storageshower`, master-record tag alignment and round-trips in `tempis` — which is a different kind of discipline entirely.

The single sentence that keeps this honest is the one the parity chapter already uses: treat a test count as a count of intentions until a `test result:` line confirms it, and treat “parity” as whatever the specific tool actually

enforces — a build gate for one tool, a byte oracle for two, a capability matrix for two, grammar-reproduction for two, and self-invariants for the two originals. There is no fleet-wide parity harness, and saying there is would be the exact kind of convenient generalization the shape-of-the-fleet chapter warned against.

The audit tools are themselves guarded

One detail deserves its own note because it is where a coverage number could most easily become a lie. `htoprs`'s coverage figure is not typed into a README — it is derived from source at run time by `scripts/gen_port_report.py`, which walks both the C source and the Rust port and writes `docs/port_report.html` with per-file and overall numbers, and which counts a `todo!()` or `unimplemented!()` body as **stubbed, never ported**. That last rule is the load-bearing one: it means scaffolding cannot inflate the coverage figure, because the generator classifies an unimplemented body as a gap no matter how the module around it is named. The stub scaffold generator (`scripts/gen_port_stubs.py`) lays out the full port surface — one module per C file, one `todo!()` stub per C function named to satisfy the gate — and never overwrites an already-ported module, so the port surface is enumerated without the enumeration masquerading as completion. A report that derives its number from source at run time, and that scores a stub as a stub, is the correct shape for an audit tool: the number moves only when the code moves, which is exactly what makes it worth printing.

Packaging the fleet – binaries, completions, man pages, release

Below the architecture sits a layer of packaging conventions that make the fleet feel like one product installed from one place rather than eight separate downloads, and — as with everything else in this book — the conventions come with honest exceptions that are worth naming rather than smoothing over. This chapter collects the packaging surface: how each tool ships its binaries, its shell completions, its man pages, its config, and its releases.

The short-alias binary convention

Frequently-typed tools get a two- or three-character second name built from the same source, byte-identical to the long form. `lsotrs` also builds `lsf`, `nmaprs` also builds `nms`, and `temprs` also builds `tp` — each declared as a second `[[bin]]` target pointing at the same entry point, so the alias is not a symlink or a shell alias but a second compiled artifact that parses `argv` identically. `lsotrs` copies `argv[0]` verbatim through its normalizer so `lsf` and `lsotrs` behave the same, and `nmaprs` re-derives the basename so `nms --help` shows `nms`. The rationale is muscle memory: a tool you reach for dozens of times an hour needs a name your fingers can hit without thinking, and `temprs`'s chapter is blunt that `tp` “exists purely because a temporary-file tool that you reach for dozens of times an hour needs a two-keystroke name.”

`grcrs` is the exception that proves the convention is about role, not length. It builds two binaries — `grc` the launcher and `grcat` the colouriser — that are *functionally distinct* rather than aliases, mirroring the same split upstream `grc` uses so the colouring logic stays a standalone filter you can drive straight from a pipeline (`some-command | grcat conf.log`). The remaining four tools (`iftoprs`, `htoprs`, `storageshower`, `zcolorizer`) ship a single binary. And the two native hosts each produce one binary plus a library: `zwire-host` builds the `zwire-host` binary and the `zwire_host` lib so other Rust programs can embed the same host, and `zpwirchrome-host` similarly exposes its internals — the download worker is a `--dl-worker` subcommand of the same binary rather than a second artifact.

Shell completions, and the two that are live

Six of the eight ship a checked-in zsh completion under `completions/_<tool>` with a `#compdef` header: `lsotrs`, `nmaprs`, `iftoprs`, `htoprs`, `storageshower`, and `temprs`. Several are kept in lockstep with the parser, and two are genuinely dynamic rather than static tables:

- `lsotrs`'s `_lsotrs` is a real `#compdef lsotrs lsf` covering both names, built on `_arguments -C -s -S` with dedicated state handlers (`->filedes` for `-d`, `->addresses` for `-i`, `->states` for `-s`, `->fields` for `-F`) that reuse the shell's own `_pids/_users/_pgids` completers with comma-sequencing.
- `htoprs`'s completion keeps its option set in lockstep with the getopt table in `commandline.rs` and completes sort columns *dynamically* — the `sort-keys` state calls `_call_program sort-keys $words[1] --sort-key help` and parses the tool's own live column list rather than hardcoding one, so the completion cannot drift from the binary's real column set.
- `temprs`'s `_temprs` (`#compdef tp temprs`) is *live against the running stack*: its `_tp_indices` helper reads the actual `temprs-stack` master file out of `${TEMPRS_DIR:-...}` and offers the current indices, file basenames, and `@`-prefixed tag names, so tab-completion reflects the contents of your stack at the

moment you press Tab. It even solves the NUL problem inline — zsh variables cannot hold NUL, so it pipes the master record through `tr '\0' '\x1e'` before splitting.

`iftoprs` both ships a checked-in `_iftoprs` and can emit completions at runtime for five shells (`--completions zsh|bash|fish|elvish|powershell`), the shell set coming from `clap_complete::Shell` directly rather than a hand-rolled enum. `zcolorizer` and `grcrs` ship no in-tree completion — `grcrs` relies on the upstream `grc.zsh/grc.sh/grc.fish` shims that wrap common commands, which is the same integration path upstream `grc` uses.

The zshall-modelled man-page pair

The same six that ship completions also ship the paired man-page convention: a short `<tool>.1` and a comprehensive `<tool>all.1` modelled on `zshall(1)`, folding invocation, keybinds, themes, and architecture into one greppable document. `lsotrs.1` is the short reference and `lsotrsall.1` documents every flag, mode, theme, and architectural note; `lsotrs.1` even defines a real exit-status contract (`0` success, `1` on error or no match) and a `SEE ALSO` pointing at `lsot(8)`, `ss(8)`, `fstat(1)`, `netstat(1)`, `pstree(1)`, `top(1)`, and `proc(5)`. `htoprsall.1` is eighteen concatenated sections and cross-references the rest of the fleet in `SEE ALSO` (`iftoprsall`, `lsotrsall`, `nmaprsall`, `zshrsall`), which is the detail that makes the pair a *fleet* convention rather than a per-tool habit — the manuals point at each other. The exceptions again: `grcrs` ships per-binary `grc.1` and `grcat.1` (the `grcat.1` page documenting all ten colourfile keywords including the `trend/key/metric` extension, flagged there explicitly as “grcrs extension”), and `zcolorizer` ships no `man/*.1` in the tree searched here. Two chapters also note their man pages have gone *stale* — `htoprs`’s both carry an early-stage PORT STATUS caveat that predates the 97.8% figure and should be resynced — which the book records rather than hides, because a stale man page is a real defect even when the code is finished.

Config persistence, split by interactivity

Where a tool keeps user preferences follows one rule — interactive tools persist, non-interactive tools do not — with the storage location split by tool:

Tool	Config	Format
<code>lsotrs</code>	<code>~/.lsotrs.conf</code>	TOML, auto-saved on every toggle
<code>iftoprs</code>	<code>~/.iftoprs.conf</code>	TOML, auto-saved
<code>storageshower</code>	<code>~/.storageshower.conf</code>	TOML, auto-saved
<code>htoprs</code>	<code>~/.config/htoprs/prefs.json</code>	JSON
<code>zcolorizer</code>	<code>\$XDG_CONFIG_HOME/zcolorizer/config.toml</code>	TOML (XDG)
<code>nmaprs</code> , <code>grcrs</code> , <code>temprs</code>	none	—

The three non-interactive tools keep no preference file: `grcrs` reads the upstream `grc.conf` data files (shared config, not per-user state), and `temprs` keeps its *stack* on disk (`$TMPDIR/temprs`, overridable with `TEMPRS_DIR`) rather than a settings file. Every persisted config, as the theme chapter noted, guards each field with `#[serde(default)]` for forward-compatibility.

Release, distribution, and the licence floor

Distribution is uniform: every tool is published to crates.io (`cargo install <tool>`) and carries a Homebrew tap install path (`brew install <tool>` or the fully-qualified `MenkeTechnologies/menketech/<tool>`), and the two native hosts install via `cargo install` with `--force` for upgrade. The release automation rhymes across the fleet — a `release.yml` that fires on `v*` tags, cross-compiles a per-target matrix (macOS aarch64/x86_64, Linux gnu, and musl via `cross`, with the musl targets typically marked `allow_failure` so a toolchain hiccup does not block a release), packages per-target tarballs, publishes a GitHub Release, and auto-updates the Homebrew tap formula with freshly computed SHA-256 sums. `grcs`'s release is the most elaborate because it must ship *data*: each tarball stages the two stripped binaries alongside `grc.conf` and every `colourfiles/conf.*` so the archive carries the runtime data the binaries expect at their search paths, and the generated formula installs `grc/grcat` to `bin`, `grc.conf` to `etc`, and the colourfiles to `share/grc` — matching the `/opt/homebrew/...` entries baked into the search paths so `brew install` yields a working install with no manual data placement.

The licence floor is the simplest fact in the book and the one most worth stating plainly: every tool in the fleet is **MIT**, the Rust in each is original code, and where a tool reimplements a Unix program it keeps the upstream source under a read-only `vendor/` tree purely as a parity reference — the binary is built from the tool's own `src/`, never from `vendor/`, and the vendored trees are excluded from the published crates. There is no copyleft anywhere in the CLI fleet and no per-tool licence divergence to track. That uniformity is not incidental to the packaging story; it is what lets the whole shelf install from one tap under one licence, which is the concrete meaning of “one product, eight binaries.”

Appendix A – the complete command & flag reference

The per-tool chapters describe each tool's flags in prose, where the surrounding sentences explain *why* a flag reads the way it does. This appendix is the other view: every command and flag the book documents, gathered into per-tool tables so a reader who already knows what a tool does can find the one invocation they need without re-reading the chapter. Nothing here is new — every entry traces to the chapter of the same name — and where a flag has a subtlety the prose already stated (an accept-and-ignore no-op, a TTY gate, a divergence from the upstream), the table carries a one-line note rather than repeating the paragraph. Flags accepted for parity but not wired to behavior are marked (*no-op*); where a tool's short binary alias exists it is named in the heading.

lsOfrs / lsf

Selection and the lsof-compatible core:

Flag	Effect
<code>-p PID[,PID]</code>	files held by the given PIDs (^ negates)
<code>-c CMD / -c /regex/</code>	command by prefix or /regex/
<code>-u USER[,USER]</code>	files for the given users (^ negates)
<code>-d SPEC</code>	descriptor set, e.g. <code>0-10</code>
<code>-i [46][proto][@host][:port]</code>	network selector (<code>-i 4TCP:22</code> is one token)
<code>-a</code>	flip selection from OR to AND
<code>-s [proto:state]</code>	show size always, or filter sockets by state
<code>-o[N]</code>	offset always, optional N-digit precision (<code>-o9</code>)
<code>+L[n]</code>	list NLINK column; <code>+L1</code> selects unlinked-but-open files
<code>-t</code>	terse — bare PIDs, one per line
<code>-F [pcfnu...]</code>	lsof field output (<code>p f n</code> default set)
<code>-0</code>	NUL-terminate fields instead of newline
<code>-J/--json, --csv</code>	JSON array / RFC 4180 CSV
<code>-V</code>	report search items that matched nothing
<code>-v</code>	version — prints <code>lsOfrs 4.9.1</code>
<code>--color always never auto</code>	force colour policy

Flag	Effect
<code>-K</code> → <code>--list-tasks</code> , <code>-T</code> → <code>--tcp-info</code> , <code>-x</code> → <code>--cross-over</code>	Isf-only flags mapped to real behavior
<code>-b -O -C -M -X -E -D ...</code>	(<i>no-op</i>) accepted-and-ignored for script compatibility

Derived views (single-shot when piped, interactive on a TTY; all accept the selection filters above):

```
lsf --tree          lsf --summary      lsf --ports
lsf --net-map       lsf --pipe-chain   lsf --stale
lsf --top 10        lsf --watch PATH   lsf --follow PID
lsf --leak-detect=10,5 lsf --stall       lsf --monitor
lsf --delta -r 2    lsf --tui
```

nmaprs / nms

Scan-type selectors (glued Nmap tokens accepted; expanded before clap):

Flag	Technique
<code>-sT</code>	TCP connect (default)
<code>-sS</code>	SYN half-open (needs privilege; falls back to <code>-sT</code> unprivileged)
<code>-sU</code>	UDP
<code>-sN / -sF / -sX</code>	NULL / FIN / Xmas
<code>-sA / -sW / -sM</code>	ACK / Window / Maimon
<code>--s0</code>	IP-protocol scan
<code>--sI ZOMBIE</code>	idle (zombie) scan
<code>-sY / -sZ</code>	SCTP INIT / COOKIE-ECHO
<code>-b</code>	FTP bounce
<code>-sn</code>	ping scan only, no ports
<code>-sV</code>	version detection (skips Perl-only regex constructs)
<code>-O</code>	OS detection (IPv4 partial; IPv6 TTL heuristic)
<code>-A</code>	<code>-O</code> + <code>-sV</code> + default <code>-sC</code> + <code>--traceroute</code>

Ports, timing, discovery, output:

Flag	Effect
<code>-p SPEC / -p -</code>	port list / all 65536
<code>-F, -r, --top-ports N</code>	fast list / sequential / top-N by frequency

Flag	Effect
<code>-T0...-T5</code>	timing template (default <code>-T3</code> → 256 workers, 1500 ms)
<code>--min-rate/--max-rate,</code> <code>--min/--max-parallelism</code>	rate and concurrency controls
<code>-Pn</code>	skip host discovery
<code>-PS/-PA/-PU/-PY/-PO/-PE/-PP/-PM</code>	the <code>-P*</code> discovery probe family
<code>-oN/-oG/-oX/-oA/-oS</code>	normal / grepable / XML / all / script-kiddie
<code>-oM, -oH</code>	partial: grepable-style / placeholder
<code>-iL FILE, -iR N, --resume STATE</code>	input list / random hosts / checkpoint
<code>--proxies, --dns-servers, --system-dns</code>	proxy chain / custom DNS
<code>-f/--mtu, -D, -S, -g, --data*, --ttl, --badsum,</code> <code>--spoof-mac</code>	evasion / raw-packet controls
<code>--privileged / --unprivileged</code>	force raw / force connect mode
<code>--resolve-all, --nsock-engine,</code> <code>--release-memory, ...</code>	(<i>no-op</i>) accepted for parity, logged not acted on

temprs / tp

Thirty-eight operations; index arguments accept a signed position (`-1` = top) or a tag name interchangeably.

Group	Flags
I/O	<code>-i IDX</code> (write into entry), <code>-o IDX</code> (read to stdout), <code>-A IDX</code> (append), <code>-v, -q</code> (<i>records intent; runtime does not yet suppress</i>)
Stack algebra	<code>-p/--pop, -s/--shift, -u/--unshift, -a IDX, -r IDX, -M FROM TO, -S A B, -x IDX, --rev, --sort name\ size\ mtime</code>
Inspection	<code>-l, -n, -L, -N, -k/--count, -I IDX, -d/--dir, -m/--master, --path IDX</code>
Coreutils-analogue	<code>--head IDX N, --tail IDX N, --wc IDX</code> (line count only), <code>--size IDX, -g PATTERN</code> (literal substring; exit 0/1), <code>-D A B</code> (shells <i>diff -u</i>), <code>-C IDX...</code> (cat)
Editing / lifecycle	<code>-e IDX (\$EDITOR), --replace IDX PAT REPL, -c/--clear, --expire HOURS</code>
Naming	<code>-w/--name NAME, -R/--rename OLD NEW</code>

Hardening worth remembering: `--clear` refuses unless the directory basename contains `temprs`; `--expire` rejects negative/NaN/inf before the cast; `--replace` rejects an empty pattern.

```

ps aux | tp -w procs      # push, tagged "procs"
tp -g python procs && echo hit
tp --path procs          # print backing file path
tp -D procs -l           # diff two entries by name/index
tp --expire 1            # drop entries older than 1h

```

iftoprs

Single binary; raw capture needs root. Non-TUI modes are handled before the terminal is touched.

Flag	Effect
-i NAME	interface to monitor
-f EXPR	BPF filter, e.g. " tcp port 80 "
-F CIDR	IPv4 local network scope
-p	promiscuous mode
-n / -N	disable reverse DNS / port → service names
-b / -B	disable bars / show bytes not bits
-P / -Z	hide ports / hide process column
--json	stream NDJSON, no TUI
-c FILE	config path (default ~/.iftoprs.conf)
-l / --list-colors	list interfaces / preview all 31 themes
--completions SHELL	zsh bash fish elvish powershell

In-TUI keys (selection): **1/2/3** sort by 2s/10s/40s, **</>** sort by name, **r** reverse, **o** freeze order, **t** cycle line display, **b** cycle bar style, **B** bits/bytes, **Tab** flow ↔ process view, **/** filter, **c/C** theme chooser/editor, **i** interface chooser, **F** pin, **y** copy, **e** export, **P** pause.

htoprs

CLI mirrors htop's **getopt_long** table exactly (driven through the real **libc::getopt_long**); **-h/-V** route to branded screens.

Flag	Effect
-d DELAY	update delay, tenths of a second (1..100)
-n NUMBER	exit after N refresh iterations
-s COLUMN	sort by column (--sort-key help lists)
-u [USER]	only this user's processes
-C/--no-color	monochrome
-M/--no-mouse	disable mouse
-U/--no-unicode	ASCII graph meters
-t/--tree	tree view
-p PID,...	show only the given PIDs
-F FILTER	fixed-string filter (multi-term)

Flag	Effect
<code>-H[=DELAY]</code>	highlight new/old processes
<code>--no-meters, --no-function-bar, --readonly</code>	hide meters / bar; disable state-changing features

Extension keys (bound to keys htop leaves free): `f` fuzzy finder, `r` regex filter (saved), `d` snapshot/diff, `o` export JSON+CSV, `A` threshold alerts, `E` memory-exhaustion forecast, `G` braille CPU history, `y` aggregate/pivot, `:` command palette, `v` per-PID sparkline, `z/~` theme chooser/editor.

storageshower

Single invocation; CLI flags override the config, and most booleans have a hidden `--no-` inverse.

Group	Flags
Sorting	<code>-s name\ pct\ size, -R/--reverse, -l/--local-only, --virtual/--no-virtual</code>
Display	<code>-b gradient\ solid\ thin\ ascii, --color PALETTE (30), --theme NAME, -u human\ gib\ mib\ bytes, -k/--compact, -f/--full-mount, --bars/--border/--header/--used/--tooltips</code>
Thresholds	<code>-w PCT (warn), -C PCT (crit)</code>
Columns	<code>--col-mount, --col-bar-end, --col-pct (u16 width, 0 = auto)</code>
System	<code>-r SECS (refresh), -c PATH (config), -h, -V</code>
Introspection	<code>--list-colors, --export-theme</code>

In-TUI keys reach the same knobs; note the layering — `-f` on the command line is `--full-mount`, while the interactive `f` key cycles the refresh rate. Thresholds cycle in fixed steps (`t` walks warn 50 → 60 → 70 → 80, `z` walks crit 80 → 85 → 90 → 95).

zcolorizer

Streaming colouriser; reads stdin line-buffered with no file arguments.

Flag	Effect
<code>--theme NAME</code>	select one of 32 themes (<code>cyberpunk/default</code> alias <code>neon-sprawl</code>)
<code>-m MODULE</code>	enable a format module (repeatable; <code>-m all, -m auto</code>)
<code>-C/--force-color, --no-color</code>	force colour on / force passthrough
<code>-g PATTERN</code>	filter to matching lines, keeping colour

Flag	Effect
<code>-H PATTERN</code>	keep all lines, dim the non-matches
<code>--novelty</code>	statistical-novelty intensity; noise self-dims
<code>--novelty-decay D</code>	age counts ($0 < D \leq 1$); implies <code>--novelty</code>
<code>-w/--watch</code>	live-reload config on mtime change
<code>--list-themes/--list-modules/--list-rules</code>	enumerate the resolved sets
<code>--dump-config, --themes-json</code>	re-emit config / emit theme structs for a GUI picker

grcrs — grc / grcat

`grc` <command> (launcher) options: `-s/--stdout`, `-e/--stderr`, `-c/--config FILE`, `--colour=on|off|auto`, `--pty` (experimental), `--` (end option scanning). Abbreviated long options resolve by unique prefix (`--conf` → `--config`), and `--version` is not a flag — it falls through `resolve_long` to `help()`, whose banner leads `Generic Colouriser 1.13`.

`grcat` <colourfile> (colouriser) is driven from a pipeline (`cmd | grcat conf.log`). Its colourfile keywords: `regexp`, `colours` (`color/colors/colour` aliased), `count` (`more/once/stop/block/unblock/previous`), `command`, `skip`, `replace`, `concat`, plus the grcrs-only additive extension `trend/key/metric` for colouring a number by its motion over the stream.

zwire-host / zpwrchrome-host

`zwire-host` selects one of three run modes by first positional argument: bare (native-messaging stdio, what Chrome spawns), `serve` (NDJSON socket daemon, `--token/--peer` for TCP federation), and `call [--stream] JSON` (one-shot client). Its capability arms — advertised in the `hello` reply and gated by Cargo features — include `kv_*`, `fs_*`, `exec`, `job_*`, `bus` (`sub/pub`), `pty_*` (feature `pty`), `sysinfo_*`/`ps/kill/which` (feature `sysinfo-caps`), and `open/clipboard/notify/hostinfo`.

`zpwrchrome-host` speaks browserpass PROTOCOL.md v3.1.2 (`configure`, `list`, `tree`, `fetch`, `save`, `delete`, `echo`, error codes 10–32) and adds `otp`, `search`, `run.spawn`, `host.crawl/host.exec` (which call into the embedded `zwire_host` library), `zcite.save`, and the `dl.*` segmented download family (`dl.add/pause/resume/cancel/remove/clear/list`, worker as `--dl-worker GID`). CLI surface: `--install <ext-id>` (writes the native-messaging manifests), `--help`, `-version` (prints the browserpass protocol version 3.1.2).

Appendix B — the palette and theme catalog

The theme-engine chapter traced the shared palette set as a lineage; this appendix is the roster itself, gathered so a reader configuring a tool can see the whole set at once. The names below are the ones the per-tool chapters enumerate; the count differs by tool for the reasons the theme chapter gave (a parity palette added here, a hand-written classic there), and the differences are listed rather than averaged away.

The 31 generated palettes (zcolorizer's kebab ids)

zcolorizer enumerates the shared set in full and in declaration order, which fixes the canonical sequence — `neon-sprawl` is first and is the default (aliased `cyberpunk/default`):

neon-sprawl	acid-rain	ice-breaker	synth-wave	rust-belt
ghost-wire	red-sector	sakura-den	data-stream	solar-flare
neon-noir	chrome-heart	blade-runner	void-walker	toxic-waste
cyber-frost	plasma-core	steel-nerve	dark-signal	glitch-pop
holo-shift	night-city	deep-net	laser-grid	quantum-flux
bio-hazard	darkwave	overlock	megacorp	zaibatsu
iftopcolor				

The thirty-second zcolorizer theme, `ccze-classic`, is not from this set — it is a hand-written 16-colour map approximating the original ccze look (cyan dates, bold-red errors, bold-yellow warnings) for parity. Name resolution is forgiving: `normalize` lowercases and collapses spaces/underscores to hyphens, so "`Neon Sprawl`", `neon_sprawl`, and `NEON-SPRAWL` all match one palette.

The roster across the fleet

Tool	Count	Set
<code>storageshower</code>	30	the origin; <code>ColorMode</code> display names "Neon Sprawl" (<code>Default</code>) ... "Acid Rain" (<code>Green</code>) ... "Zaibatsu"
<code>iftoprs</code>	31	the 30 plus the parity palette <code>Iftopcolor</code> ; <code>ThemeName::ALL.len() == 31</code> asserted
<code>lsofrs</code>	31	NeonSprawl ... Zaibatsu; <code>ThemeName::ALL.len() == 31</code> asserted
<code>htoprs</code>	31	31 six-colour palettes, applied at <code>Ncurses::to_color</code>

Tool	Count	Set
<code>zcolorizer</code>	31 + 1	31 generated (above) + hand-written <code>ccze-classic</code> = 32; confirmed at runtime

`iftoprs`, `lsofrs`, `htoprs`, and `storageshower` all check in a `*.default.conf` (or `themes/` copies) that enumerates the theme names for hand-editing.

The six-colour custom-theme model

Every tool that carries the set also lets a user define a custom theme as six 256-colour indices, edited either by hand in the config or through a live in-app six-slot editor (bound to `C` in `lsofrs/iftoprs/storageshower`, `~` in `htoprs`). The six slots carry a fixed meaning — `zcolorizer` names them *primary*, *brightest accent*, *secondary*, *mid*, *dim*, *darkest*; `storageshower` names the channels `blue`, `green`, `purple`, `light_purple`, `royal`, `dark_purple`. A `storageshower` custom theme in TOML:

```
[custom_themes.neonpink]
blue      = 199
green     = 46
purple    = 201
light_purple = 213
royal     = 196
dark_purple = 161

active_theme = "neonpink"
```

Two collision-avoidance and generation details from the theme chapter belong here as the catalog's fine print: `shift_lighter` (an index-cube arithmetic helper ported from `iftoprs` into `zcolorizer`) steps a 256-colour index one notch brighter, clamping at the cube and grayscale-ramp edges, and is used to derive a distinguishable extra shade when two tokens would otherwise land on the same slot (`process` vs `host`); and `zcolorizer`'s 31 non-classic themes are *generated* from the six palette colours by `theme_from_palette` rather than authored token-by-token, layering bold/italic/underline over the six roles so a six-colour palette styles roughly sixty tokens.

zcolorizer token vocabulary and modules

`zcolorizer`'s themes key on token names, and the well-known ones the builtin rules emit are worth listing because a config can restyle any of them: the generic vocabulary (`date`, `time`, `host`, `ip`, `mac`, `pid`, `process`, `error`, `warning`, `debug`, `info`, `good`, `bad`, `system`, `keyword`, `email`, `uri`, `dir`, `file`, `size`, `version`, `number`, `address`, `percentage`, `signal`, `http_method`, `http_code`, `string`), the ccze-plugin tokens (`gettime`, `getsize`, `ftp_code`, `ident`, the eleven `proxy_*` squid actions, and the rest), and the modern-format tokens (`thread`, `level`, `facility`, `duration`, `latency`, `hash`, `json_key`). The 47 format modules split 20 classic ccze ports (`syslog`, `httpd`, `squid`, `dpkg`, `postfix`, `exim`, `procmail`, `proftpd`, `vsftpd`, `ftpstats`, `xferlog`, `php`, `oops`, `icecast`, `fetchmail`, `apm`, `distcc`, `sulog`, `super`, `ulogd`) and 27 modern ones (`json`, `logfmt`, `journald`, `authlog`, `kernel`, `cron`, `auditd`, `nginx`, `haproxy`, `caddy`, `traefik`, `postgres`, `mysql`, `redis`, `mongodb`, `klog`, `docker`, `spring`, `python`, `rust-log`, `rails`, `named`, `dnsmasq`, `dhcpcd`, `fail2ban`, `ufw`, `dovecot`).

Appendix C – the provisional-numbers ledger

The book marks every version, test count, and benchmark figure as provisional and states, at each site, which tool produced it. This appendix consolidates those numbers into one place so a reader can see the whole ledger at a glance — and so the provisional caveat is impossible to miss. **Read every cell in this appendix under the methodology chapter’s rule: a test count is a count of `#[test]` attributes found by `grep` (or, where noted, a runtime enumeration), not a `test result` line; a benchmark is the author’s own measurement on one machine, not reproduced for this book; a version is read from a `Cargo.toml/manifest`, not confirmed against a compiled artifact.** The numbers are the counts the sources reported at the time of writing, and they will drift.

Versions and editions

Tool	Version	Edition	Licence
<code>lsofrs</code>	4.9.1	2024	MIT
<code>nmaprs</code>	0.1.9	2021	MIT
<code>temprs</code>	2.9.12	2024	MIT
<code>iftoprs</code>	2.22.9	2024 (Rust $\geq$ 1.85)	MIT
<code>htoprs</code>	0.5.7	2021	MIT
<code>storageshower</code>	0.28.8	2024	MIT
<code>zcolorizer</code>	0.2.2	2021	MIT
<code>grcrs</code>	1.13.0	—	MIT
<code>zwire-host</code>	0.3.14	—	MIT
<code>zpwrchrome-host</code>	0.10.2	—	MIT

Test-attribute counts (provisional; `grep` inventory unless noted)

Tool	Reported test surface
<code>lsofrs</code>	1,029 <code>#[test]</code> in <code>src/</code> + 710 in <code>tests/</code> + 21 shell gate scripts
<code>nmaprs</code>	~1,468 test fns total; <code>tests/cli.rs</code> alone 204
<code>temprs</code>	5,059 unit (4,449 <i>generated</i> flag tests in <code>opts.rs</code>) + 585 integration + 21 shell
<code>iftoprs</code>	2,275 annotations (1,730 in <code>src/</code> , 545 in <code>tests/</code>)
<code>htoprs</code>	1,411 <code>#[test]</code> ; plus the <code>tests/parity/</code> byte-diff suite

Tool	Reported test surface
<code>storageshower</code>	3,101 in <code>tests/</code> + 596 in <code>src/</code> ≈ 3,697 (counts verified in-session; passing count not)
<code>zcolorizer</code>	94 unit + 17 integration + 1 doc = 112 (reported passing)
<code>grcrs</code>	71 total: 17 launcher + 27 colouriser + 27 end-to-end
<code>zpwchrome-host</code>	121 (from README, not built here)

Port-coverage and parity figures

Tool	Figure
<code>htoprs</code>	1,069 / 1,093 C functions ported (97.8%), 130/131 files, 1 stub; 1,930 <code>/// Port of</code> citations; 4,132-entry C-name index; empty allowlist
<code>ztmux</code> (Part I)	1,107 / 1,107 parity cases vs vendored tmux; <code>PROTOCOL_VERSION = 8</code>
<code>powerliners</code> (Part I)	134/137 upstream <code>.py</code> at DONE (97.8%); 2,436 lib tests; 462 parity tests
<code>nmaprs</code>	parity documented as a capability matrix, contract-tested
<code>grcrs</code>	end-to-end bytes cross-checked vs reference Python <code>grcat</code>
<code>zcolorizer</code>	47 modules, 32 themes (runtime-enumerated); precedence + sanitization pinned

Benchmark claims (author's own, one machine, not re-produced here)

Tool	Claim	Conditions
<code>lsofrs</code>	5–21× vs lsof 4.91 / lsofng	macOS, ~470 procs / ~5,000 open files, hyperfine
<code>nmaprs</code>	1.5–5.1× vs nmap 7.99	macOS arm64, <code>127.0.0.1, -n -Pn --min-rtt-timeout 50ms --max-retries 0</code> ; ~74–82 μs Criterion connect microbench
<code>storageshower</code>	format ops tens of ns; 200-mount render ~15.7 μs	Apple Silicon
<code>awkrs</code> (Part I)	14–110× fusevm offload vs bytecode interp; first awk pairing a bytecode VM with a persistent on-disk bytecode cache	pure-numeric accumulator loops
<code>zshrs</code> (Part I)	717 ms cold vs 7 ms warm	same script, rkyv image reuse
<code>vimlrs</code> (Part I)	~15–100× JIT vs interpreter	20M-iteration loop

Source size and module counts (provisional; per-chapter figures)

The per-tool chapters cite source sizes in passing, usually to justify which module carries the load-bearing logic. Gathered here, they sketch the relative weight of the eight ports — with the same caveat as everything else in this appendix: these are `wc -l/module-count` figures the chapters reported at the time of writing.

Tool	Source size	Notable modules
<code>lsofrs</code>	29 modules under <code>src/</code>	<code>main.rs</code> dispatcher; <code>types.rs</code> data model; <code>cli.rs</code> normalizer; per-mode view modules
<code>nmaprs</code>	~24,000 lines / 35 files	<code>config.rs</code> 2,314; <code>os_scan.rs</code> 1,860; <code>syn.rs</code> 1,760; <code>lib.rs</code> 1,552; <code>vscan.rs</code> 1,499; <code>discovery.rs</code> 1,437
<code>temprs</code>	<code>opts.rs</code> ~47,000 (mostly generated tests)	<code>utils.rs</code> ~2,200; <code>model/state</code> , <code>model/app.rs</code> , <code>consts.rs</code>
<code>iftoprs</code>	~23,040 lines / 20 files	<code>app.rs</code> 3,805; <code>parser.rs</code> 3,074; <code>render.rs</code> 2,610; <code>cli.rs</code> 2,344; <code>resolver.rs</code> 1,733; <code>flow.rs</code> 1,685
<code>htoprs</code>	141 ported modules + 21 extension modules	per-OS backends (<code>darwin/...solaris/</code>); the full meter set; 1,930 /// <code>Port of citations</code>
<code>storageshower</code>	~13,700 lines / ~12 modules	<code>ui.rs</code> ~3,860; <code>keys.rs</code> ~1,930; <code>cli.rs</code> ~1,550; <code>system.rs</code> ~1,510; <code>app.rs</code> ~1,320
<code>zcolorizer</code>	47 format modules over a 3-layer engine	<code>rules.rs</code> / <code>theme.rs</code> / <code>engine.rs</code> ; <code>novelty.rs</code> ; <code>modules.rs</code> (20) + <code>modules_modern.rs</code> (27)
<code>grcirs</code>	2 binaries, 2 dependencies	<code>grcirs.rs</code> launcher; <code>grcatrs.rs</code> colouriser; each carries its own <code>translate_regex</code>

Two shapes stand out in the ledger and both are documented in their chapters. `temprs`'s `opts.rs` is an outlier at ~47,000 lines because it is dominated by the 4,449 machine-generated flag-recognition tests, not by hand-written logic — the same reason its test count leads the fleet. And `htoprs`'s module count (141 ported + 21 extension) is high because the port mirrors htop's own file layout one module per C file, including the seven per-OS backend trees htop itself carries; the count is a property of faithfulness to the upstream's structure, not of feature sprawl.

The single instruction

The single instruction that governs this entire appendix is the one the methodology chapter states and the preface states before it: where a number is provisional the text marks it rather than rounding it into a claim, because versions move, suites grow, and a book that pretends otherwise goes stale the day it prints. These are the counts the sources reported at the time of writing — no more, and marked as no more.